

Claude

Q </>

How to use Claude / HTML to JSON conversion assessment

Share

Hide sidebar

R Roman · Pro

Projects

Artifacts

Scheduled

Customize

More

Chats and tasks

🔗

- HTML to JSON conversion assessment
- Fixing HTML comment parsing bug
- Відповідальність за ремонт крана в
- Виправлення подвійного декодуван
- Xfinity trade-in promotion and plan t
- Вихідні у Сент Августин та Кокоа Бі
- Геометричний орел на стіну
- Цвіль у повітропроводі кондиціонер
- Пошук роботи junior розробника в І
- Building a task manager portfolio pro
- iPhone trade-in pricing explanation
- Тижневий план харчування для вагі
- Сумісність об'єктивів з Nikon F60
- Видалення сірої зони на сеас.state.
- Finding NIZUC Resort & Spa in Canc
- Вибір між рецептами на Zoryve
- Node vs Go: вибір напрямку розвит
- Що він має на увазі під картою
- Доступ до податкових декларацій
- Заповнення форми F1300306 у ДПС

View all

Look here is index.html and
html2json 2 files function
convertHtmlToJsonAndSet()
{ const htmlTextAreaValue = document.getElementById("html").value; const jsonObj = html2json.parse(htmlTextAreaValue); console.log(jsonObj); }

PASTED

Hi cloud, i received an assesment and i need you help to solve that task, i read description and i want to share it with you, read it carefully.

<https://github.com/jito-dev/jito-intern-test-task#expected-deliverables>

repo with assesmant already have 2 files index.html and html2json

I noticed 2 important comments inside html2json !

Maybe you will not have access to thats 2 files, so i attached them to our chat

Lets write together our plan what to do each step and teach me, i will write a code, you will review it and tell if something wrong, we need to do our best

Read memory, read a page >

I read the README and both files. Here's what matters most, then our plan, then your first exercise.

What they're really testing

Four rules shape everything we do:

- No DOM parser.** You can't use `DOMParser`, `innerHTML`, `<template>`, or libraries like `htmlparser2`. You'll write your own parser that walks through the string character by character.
- The code must never crash**, even on strange or broken HTML. They'll feed it inputs designed to break it.
- Reasoning matters as much as the code.** The comment in `showExample1` says you must be ready to defend your JSON structure, and a senior dev may quiz you on it later. So you need to understand every line you write, which is why doing it this way (you write, I review) is right.
- Missing deliverables mean an automatic fail.** You need `html2json.js`, an `html_samples/` folder, the unchanged `index.html`, and an `ai_help/` folder. Since you're using Claude and not ChatGPT, put this chat's share link in a text file there (for example `claude_chat.txt`). I'd also add a PDF export as backup. Keep all the work in this one conversation, and before submitting, open the link in an incognito window to make sure it works.

The plan

- Design the JSON format** and write down why you chose it. No code yet.
- Tokenizer, basic version:** walk the string and split it into tokens: text, opening tag, closing tag. No attributes yet.
- Tree builder:** turn the flat token list into a nested tree using a stack. This is the heart of the parser.
- Attributes:** handle double-quoted, single-quoted, unquoted, and boolean ones (`disabled`), spaces around `=`, and a `>` inside a quoted value.
- Void and self-closing tags:** `meta`, `link`, `br`, `img`, `input`, `hr`, and so on, plus the `<br/>` syntax.
- Comments and doctype:** `<!-- ... -->` and `<!DOCTYPE html>`.
- Special content:** inside `<script>` and `<style>`, text like `if (a < b)` is not a tag. Inside `<textarea>` and `<title>`, content is also plain text (that's why Example 2 has the long textarea).
- Entities:** decode `©`, `&`, `©`, `©` (Example 1 uses `©`).
- Whitespace policy:** Example 1 is full of indentation-only text. Decide whether to keep or drop it, and explain why.
- Broken HTML recovery:** unclosed tags, stray `</div>`, wrong nesting like `<b><i></b>`, a bare `<` in text (`5 < 6`), tag names in different cases (`<DIV></div>`), input ending in the middle of a tag or comment.

11. **Robustness:** empty or non-string input, very large input (we'll avoid recursion so deep nesting can't overflow the call stack), and a final safety net so the function never throws.
12. **Test samples:** build the `html_samples/` folder, with one file per category of edge case, and test them all.
13. **README and final checklist:** explain your reasoning and check every deliverable and link.

We'll do one step at a time. You write the code, paste it here, and I review it before we move on.

Step 1: choosing the JSON format

The first idea most people have is to use tag names as keys:

```
json
{ "div": { "p": "Hello world!", "button": "Click me!" } }
```

This fails in three ways, and knowing them is exactly the reasoning they want from you:

- **Repeated tags overwrite each other.** A JSON object can't have three `"li"` keys, so Example 1's nav would lose two links.
- **Order is lost.** HTML order matters, and object keys aren't a good way to show it.
- **Mixed content has nowhere to go.** `<p>Hi <b>there</b> friend</p>` has text, then an element, then more text.

The fix is to describe every piece of the document as a node with a `type`. Children go in an array, which keeps them in order and allows duplicates:

```
json
{
  "type": "element",
  "tag": "a",
  "attributes": { "href": "#home" },
  "children": [
    { "type": "text", "content": "Home" }
  ]
}
```

The other node types would be `"text"`, `"comment"`, and `"doctype"`. At the top sits a `"document"` node with a `children` array. We need that root because Example 2 is a fragment and not a full page. Input like `<p>a</p><p>b</p>` has two top-level elements, and a single root handles both cases the same way.

Your exercise

Before any code, write out by hand the full JSON you'd expect for **Example 2** using this format. Think about these three things while you do it:

1. What happens to the newlines and spaces between `</p>` and `<button>`?
2. Should `attributes` be `{}` or left out when an element has none? Pick one and give a reason.
3. Should tag names be stored as written or lowercased? What about `<DIV>`?

There's no single right answer to these. What matters is that you choose on purpose and can explain the choice. Paste your JSON here and we'll discuss it, then move on to Step 2, the tokenizer.

📄 🔍 ⬆️ ⬇️ ⌛ 3 days ago

- **Whitespace:** Browsers actually care about those spaces and newlines between tags like `</p>` and `<button>`. If you strip them automatically, you risk ruining formatted text inside `<pre>` tags or squishing inline elements like side-by-side `<span>`s. Keeping them as text nodes (like `"\n"`) is the safest choice for document accuracy.
- **Empty Attributes:** Always include `"attributes": {}`. If you leave `attributes` undefined when a tag has no attributes, anyone using your JSON will hit `TypeError: Cannot read properties of undefined` the second

they try to check `node.attributes.class`. An empty object keeps the schema predictable every time.

- Tag Casing: Always convert tag names to lowercase (`"div"`). HTML doesn't care if someone writes `<DIV>`, `<div>`, or `<DiV>`, but JavaScript sure does. Lowercasing everything upfront saves you from writing annoying checks like `if (tag === 'div' || tag === 'DIV')` all over your parser.

Weighing casing rules and planning the tokenizer walkthrough. >

Good decisions, and you gave a reason for each one, which is what they'll look for. A few refinements so your reasoning holds up if a senior dev pushes on it.

Whitespace: keep it, but use a stronger argument. Your conclusion is right, but "browsers care about the spaces" is only half true. In normal text, browsers collapse a newline plus eight spaces of indentation into a single space, or drop it entirely between block elements. The reviewer could push back on that. A stronger argument is that **the parser should be lossless**. The JSON records exactly what's in the source, and whoever uses the JSON can decide what to throw away. A parser that silently deletes content is making decisions it has no right to make, and `<pre>` and `<textarea>` prove why. Be ready for the counter-argument too: Example 1's output will have many `"\n"` nodes and look noisy. Your answer: "Correctness over prettiness; filtering is easy afterward, recovering deleted data is impossible."

Empty attributes: perfect. "Predictable schema" is the exact phrase to use in your README. The same rule means `children: []` on every element, including void elements like `<br>`, so every element node has the same shape.

Casing: right, with two extensions.

1. Lowercase **attribute names** too (`CLASS="x"` becomes `class`), but **never attribute values or text**. `href="/About"` must stay exactly as written.
2. Closing tags must be lowercased before matching, or `<DIV></div>` won't close.

One known limitation to note in your README: SVG is case-sensitive (`viewBox`, `<linearGradient>`), so lowercasing breaks it. It's fine not to handle this, but mentioning it shows you've thought beyond the basics.

Keep Example 2's expected JSON in mind. We'll use it as a test later.

Step 2: the tokenizer

The idea: the parser works in two stages.

```
"<div><p>Hi</p></div>"
  → tokenize() → [open div, open p, text "Hi", close p, close div] (flat list)
  → buildTree() → nested JSON (Step 3)
```

We keep them separate so each stage is simple and can be tested on its own. It's also easy to explain in an interview: "the tokenizer knows nothing about nesting, and the tree builder knows nothing about characters."

How it works: keep an index `i` and walk through the string. At each position, ask one question: is the current character `<` or not?

- **Not `<`:** this is text. Read until the next `<` (or the end of the string) and push a text token.
- **`<` followed by a letter:** an opening tag. Read the tag name (letters, digits, `-`) until you hit a space, `/`, or `>`. Then jump past the next `>`.
- **`<` followed by `/` and a letter:** a closing tag. Same process.
- **`<` followed by anything else** (a space, a digit, the end of the string): it's not a tag. Treat the `<` as text. This handles `5 < 6`.
- **No `>` found anywhere after a tag start** (input like `<p>hello <span`): treat the rest of the string as text. Never throw.

Token shapes for now:

```
js
```

```
{ type: "text", content: "Hi" }
{ type: "openTag", name: "div" }
{ type: "closeTag", name: "div" }
```

For now, skip over attributes by jumping to the next `>`. That breaks on `<a title="a">b">`, and I know it. We'll fix it properly in Step 4, so don't worry about it yet.

The #1 bug to avoid: every pass through your `while` loop **must** move `i` forward. If some branch forgets to, the page freezes, and a frozen page counts as a crash to them.

Your task: write `function tokenize(html)` that returns the array of tokens. To see results in the UI, temporarily make `html2json` return `tokenize(htmlText)`. Then test these inputs:

```
<div><p>Hello</p></div>
5 < 6 and <b>bold</b>
<p>unterminated <span
</div>
<DIV>Mixed</div>
(empty input)
```

For the first one you should get 5 tokens. Before running the others, predict what you expect to see, then check whether the output matches. Paste your code and the results, and I'll review.

lets be more focused on our project and start to work on it step by step

Got it. From now on, one small step at a time: short explanation, you write the code, I review it.

Step 2.1: the loop and text tokens

We'll start with the simplest possible tokenizer. It only understands text for now and ignores tags completely.

In `html2json.js`, above the `html2json` function, write:

```
js
function tokenize(html) {
  const tokens = [];
  let i = 0;

  while (i < html.length) {
    // your code here
  }

  return tokens;
}
```

Inside the loop, do this:

1. Find where the next `<` is, starting from `i`. (Hint: `html.indexOf("<", i)`.)
2. If there's no `<`, the rest of the string is text. Push it as `{ type: "text", content: ... }` and stop the loop.
3. If the `<` is exactly at `i`, just move `i` forward by 1 for now. We'll handle tags in the next step.
4. Otherwise, the characters from `i` up to the `<` are text. Push them and move `i` to the `<`.

Then make `html2json` return `tokenize(htmlText)` so you can see the output in the page.

Test with: `Hello <b>world</b>`

Before you run it, write down what you expect the output to look like. Then paste your code and the actual output here.

```
function tokenize(html) { const tokens = []; let i = 0; while (i < html.length) { const
nextTag = html.indexOf("<", i); if (nextTag === -1) { // No more '<' found; push
remaining text and exit tokens.push({ type: "text", content: html.slice(i) }); break; }
if (nextTag === i) { // Temporary logic: skip '<' character for now i++; } else { // Text
up to the next '<' tokens.push({ type: "text", content: html.slice(i, nextTag) }); i =
nextTag; } } return tokens; }
```

Reviewing code and tracing sample output for HTML handling.

The logic is correct. The loop always moves `i` forward, so it can't freeze, and it handles the "no more `<`" case properly.

One note on formatting: your code arrived as a single line. If it's really one line in your file, the `// comment` will comment out everything after it. It's probably just how it pasted, but paste inside triple backticks (```) from now on so I see the real formatting.

The output you should see for `Hello <b>world</b>`:

```
json
[
  { "type": "text", "content": "Hello " },
  { "type": "text", "content": "b>world" },
  { "type": "text", "content": "/b>" }
]
```

`"b>world"` looks wrong, but it's expected for now. We only skip the `<` and don't understand tags yet. Does your output match?

Step 2.2: recognizing tags

Replace the temporary `i++` branch. When `nextTag === i`, look at the character right after the `<`:

1. **A letter:** it's an opening tag.
2. **A `/` followed by a letter:** it's a closing tag.
3. **Anything else** (a space, a digit, the end of the string): it's not a tag. Push `{ type: "text", content: "<" }` and move `i` forward by 1.

For cases 1 and 2:

- Find the closing `>` with `html.indexOf(">", i)`.
- **If there's no `>`**, treat the rest of the string as text and stop. This covers input like `<p>hello <span`.
- Read the tag name: start after `<` (or after `</`) and keep reading while the character is a letter, digit, or `-`. Lowercase it.
- Push `{ type: "openTag", name }` or `{ type: "closeTag", name }`.
- Move `i` to just after the `>`.

Hint: a small helper keeps the loop readable:

```
js
function isLetter(ch) {
  return /[a-zA-Z]/.test(ch);
}
```

`ch` can be `undefined` at the end of the string. Check what `/[a-zA-Z]/.test(undefined)` returns in the console before you rely on it.

Test with:

```
Hello <b>world</b>
5 < 6
<p>unterminated <span
```

Paste your code (in backticks) and the output for all three.

hey iam confused now, lets create clear plan - each step what we do to build our app

Drafting a clear, step-by-step roadmap for the request.

You're right, I jumped into details too fast. Here's the full roadmap in one place.

The big picture

Our function works in two stages:

```
HTML string → tokenize() → flat list of pieces → buildTree() → nested JSON
```

The tokenizer cuts the string into pieces (text, opening tag, closing tag). The tree builder puts those pieces together into the nested structure. We build the simplest working version first, then improve it one feature at a time.

The plan

#	What we build	Done when
Part 1: first working version		
1	Tokenizer: text only	<code>Hello</code> gives one text token done
2	Tokenizer: tag names	<code>hi</code> gives open b, text, close b we are here
3	Tree builder	<code><div><p>Hi</p></div></code> shows nested JSON on the page
Part 2: real HTML features		
4	Attributes	<code></code> gives <code>{ href: "#home", disabled: "" }</code>
5	Void and self-closing tags	<code>
</code> , <code></code> , <code>
</code> don't swallow the rest of the page
6	Comments and doctype	<code><!-- note --></code> and <code><!DOCTYPE html></code> become their own nodes
7	Script, style, textarea, title	<code><script>if (a < b)</script></code> stays as plain text
8	Entities	<code>&copy;</code> becomes <code>©</code> , <code>&amp;</code> becomes <code>&</code>
Part 3: never crash		
9	Broken HTML	unclosed tags, stray <code></div></code> , wrong nesting all give sensible output
10	Safety net	empty input, non-string input, huge input: no crash, no freeze
Part 4: delivery		
11	Test samples	<code>html_samples/</code> folder with a file for each tricky case, all passing
12	README	our design decisions explained (the JSON format, whitespace, casing)
13	Final checklist	all folders present, chat link works in incognito, everything retested

Once Step 3 is done, the app actually works for simple HTML. Every step after that adds one feature, and we test before moving on.

How each step goes

1. I explain the goal in a few lines.
2. You write the code.
3. You paste it (in triple backticks) plus the output of the test.
4. I review it, and we fix anything before the next step.

Where we are now

Step 2: making the tokenizer recognize tag names. My previous message has the instructions for it. If it felt like too much at once, we can split it further: first opening tags only (`<b>`), then closing tags (`</b>`). Want to start with just opening tags?

```
I wrote next code Part 1 . 2
look and give your feedback

function tokenize(html) {
  const tokens = [];
  let i = 0;

  while (i < html.length) {
    const nextTag = html.indexOf('<', i);

    if (nextTag === -1) {
      const textContent = html.slice(i);
      if (textContent.length > 0) {
        tokens.push({ type: 'text', content: textContent });
      }
      break;
    }

    // 1. Text before '<'
    if (nextTag > i) {
      tokens.push({ type: 'text', content: html.slice(i, nextTag) });
      i = nextTag;
    }

    // 2. We are at '<'. Find matching '>'
    const closeBracket = html.indexOf('>', i);
    if (closeBracket === -1) {
      // Unclosed tag at end of string
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    const tagContent = html.slice(i + 1, closeBracket).trim();
    if (tagContent.startsWith('/')) {
      // End tag (e.g., </b>)
      const tagName = tagContent.slice(1).trim().toLowerCase();
      tokens.push({ type: 'endTag', tag: tagName });
    } else {
      // Start tag (e.g., <b> or <div class="card">)
      const spaceIndex = tagContent.search(/\s/);
      const tagName = (spaceIndex === -1 ? tagContent : tagContent.slice(0,
spaceIndex)).toLowerCase();
      tokens.push({ type: 'startTag', tag: tagName });
    }

    // Move pointer past the '>'
    i = closeBracket + 1;
  }

  return tokens;
}
```

Checking that the parsing loop always makes progress.

Good work. The structure is clear, the naming is consistent (`startTag` / `endTag` are the names the HTML spec uses), and every path through the loop either moves `i` forward or breaks, so it can't freeze. The unclosed-tag case at the end is handled too.

There are two bugs to fix before Step 3.

Bug 1: a < that isn't a tag breaks everything after it

Input: 5 < 6 and bold

Your code finds the < after 5, then searches for the next >, which belongs to . So tagContent becomes "6 and <b", and you get:

```
json
{ "type": "startTag", "tag": "6" }
```

The text 6 and disappears, and so does the tag. The same problem hits <div>, <> and </>. Browsers treat all of these as plain text.

Fix: before looking for >, check the character right after <:

- a letter means a start tag
- / followed by a letter means an end tag
- anything else means the < is just text: push { type: "text", content: "<" }, do i++, and continue

Bug 2: the tag name includes too much

Your code cuts the name at the first whitespace, but whitespace isn't the only thing that ends a name:

Input	You get	Should be
 	"br/"	"br"
</div foo>	"div foo"	"div"

Fix: instead of cutting at whitespace, read characters one by one while they're valid name characters (letters, digits, -). The first character that isn't valid ends the name. A small helper keeps the loop clean:

```
js
function readTagName(html, start) {
  // loop from start while the char is a letter, digit or "-"
  // return the name (lowercased)
}
```

Use it for both start and end tags. That also lets you drop the .trim() calls.

One side effect to know about

<!DOCTYPE html> and <!-- comments --> will now become text, because ! isn't a letter. That's fine for now; we handle them properly in Step 6.

Test after fixing

Input	Expected tokens
5 < 6 and bold	text "5 ", text "<", text " 6 and ", start b, text "bold", end b
 	start br
</div foo>	end div
< div>	text "<", text " div>"
<DIV>Hi</div>	start div, text "Hi", end div

Notice that 5 < 6 becomes three separate text tokens. That works, but it would give us three text nodes in the JSON where there should be one. Optional improvement: write a small pushText(tokens, text) helper. If the last token is already text, append to it instead of pushing a new token. Then the first test gives a single "5 < 6 and " token.

Paste your updated code and the results for these inputs.

what you think about this solution?

```

function isValidTagChar(ch) {
  return (ch >= 'a' && ch <= 'z') ||
    (ch >= 'A' && ch <= 'Z') ||
    (ch >= '0' && ch <= '9') ||
    ch === '-';
}

function tokenize(html) {
  const tokens = [];
  let i = 0;

  while (i < html.length) {
    const nextTag = html.indexOf('<', i);

    if (nextTag === -1) {
      const textContent = html.slice(i);
      if (textContent.length > 0) {
        tokens.push({ type: 'text', content: textContent });
      }
      break;
    }

    // 1. Collect text before '<'
    if (nextTag > i) {
      tokens.push({ type: 'text', content: html.slice(i, nextTag) });
    }
    i = nextTag;

    // 2. Look ahead past '<' to verify if it's a valid tag start
    const charAfterLt = html[i + 1];

    // Check for End Tag: </a
    const isEndTag = charAfterLt === '/' && isValidTagChar(html[i + 2]);

    // Check for Start Tag: <a
    const isStartTag = isValidTagChar(charAfterLt);

    if (!isStartTag && !isEndTag) {
      // Not a real tag (e.g. "5 < 6" or "<>"), treat '<' as plain
      // text
      tokens.push({ type: 'text', content: '<' });
      i++;
      continue;
    }

    // 3. Find closing '>'
    const closeBracket = html.indexOf('>', i);
    if (closeBracket === -1) {
      // Unclosed tag: treat remaining input as text
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    // 4. Extract valid tag name character-by-character
    let nameStart = isEndTag ? i + 2 : i + 1;
    let nameEnd = nameStart;

    while (nameEnd < closeBracket && isValidTagChar(html[nameEnd])) {
      nameEnd++;
    }

    const tagName = html.slice(nameStart, nameEnd).toLowerCase();

    if (isEndTag) {
      tokens.push({ type: 'endTag', tag: tagName });
    } else {

```

```
tokens.push({ type: 'startTag', tag: tagName });
}

// Move pointer past '>'
i = closeBracket + 1;
}

return tokens;
}
```

Show more

Auditing tag-parsing code for edge case bugs.

This is a solid solution. The look-ahead check works, and the name-reading loop is exactly the right idea. `isValidTagChar(undefined)` safely returns `false`, so checking `html[i + 2]` at the end of the string can't crash. Every path moves `i` forward or breaks, so no freezes.

One real bug, and two tiny cleanups.

Bug: a tag can't start with a digit or `-`

Your start check uses `isValidTagChar`, which allows digits and `-`. But in HTML, **the first character of a tag name must be a letter**. Digits and `-` are only allowed after it. So right now:

Input	You get	Should be
<code>x <5 y</code>	start tag <code>5</code>	text
<code>a <- b</code>	start tag <code>-</code>	text

`a <- b` looks rare, but arrows like that are common in real text and code snippets, so reviewers may test it.

Fix: add an `isLetter(ch)` helper and use it only for the **first** character: `isLetter(charAfterLt)` for start tags, `isLetter(html[i + 2])` for end tags. Keep `isValidTagChar` for the rest of the name.

Small cleanups

- In the `nextTag === -1` branch, `textContent.length > 0` is always true, because the loop only runs while `i < html.length`. You can remove the check.
- `let nameStart` is never reassigned, so it should be `const`.

After the fix, test `x <5 y` and `a <- b`. Both should come out as text only. Once they do, **Step 2 is done**.

Step 3: the tree builder (just the idea for now)

This is where the flat token list becomes nested JSON. The key tool is a **stack** that tracks which element we're currently inside.

- Start with a root node `{ type: "document", children: [] }` and a stack containing only the root.
- The **top of the stack** is the current parent.
- **Text token:** add a text node to the current parent's `children`.
- **Start tag:** create `{ type: "element", tag, attributes: {}, children: [] }`, add it to the current parent's `children`, then push it onto the stack. It becomes the new parent.
- **End tag:** if the top of the stack has the same tag name, pop it. If not, ignore the token for now (Step 9 handles broken HTML properly). **Never pop the root.**

Walk through `<div><p>Hi</p></div>` on paper with this and you'll see the stack go `[root]` → `[root, div]` → `[root, div, p]` → back down to `[root]`.

Your task: write `function buildTree(tokens)` that returns the root. Then change `html2json` to return `buildTree(tokenize(htmlText))`.

Test with:

1. `<div><p>Hi</p></div>` : `p` is inside `div`, which is inside the root
2. `<p>a</p><p>b</p>` : two `p` elements side by side in the root's children
3. `</div>hello` : no crash, just a text node `"hello"`

Paste the code and the output of the first test.

```
// Helper: Only letters are valid for the FIRST character of a tag name
function isLetter(ch) {
  return (ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z');
}

// Helper: Letters, numbers, and hyphens are valid for subsequent characters
function isValidTagChar(ch) {
  return isLetter(ch) || (ch >= '0' && ch <= '9') || ch === '-';
}

function tokenize(html) {
  const tokens = [];
  let i = 0;

  while (i < html.length) {
    const nextTag = html.indexOf('<', i);

    if (nextTag === -1) {
      // Cleanup: Since i < html.length, html.slice(i) is guaranteed non-empty
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    // 1. Collect text before '<'
    if (nextTag > i) {
      tokens.push({ type: 'text', content: html.slice(i, nextTag) });
      i = nextTag;
    }

    // 2. Look ahead past '<' to verify if it's a valid tag start
    const charAfterLt = html[i + 1];

    // End tag must start with </ followed by a letter (e.g., </a)
    const isEndTag = charAfterLt === '/' && isLetter(html[i + 2]);
    // Start tag must start with < followed by a letter (e.g., <a)
    const isStartTag = isLetter(charAfterLt);

    if (!isStartTag && !isEndTag) {
      // Not a real tag (e.g., "x <5 y" or "a <- b"), treat '<' as plain text
      tokens.push({ type: 'text', content: '<' });
      i++;
      continue;
    }

    // 3. Find closing '>'
    const closeBracket = html.indexOf('>', i);
    if (closeBracket === -1) {
      // Unclosed tag: treat remaining input as text
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    // 4. Extract valid tag name character-by-character
    // Cleanup: nameStart is never reassigned, using const
    const nameStart = isEndTag ? i + 2 : i + 1;
    let nameEnd = nameStart;

    while (nameEnd < closeBracket && isValidTagChar(html[nameEnd])) {
      nameEnd++;
    }

    const tagName = html.slice(nameStart, nameEnd).toLowerCase();

    if (isEndTag) {
      tokens.push({ type: 'endTag', tag: tagName });
    } else {
      tokens.push({ type: 'startTag', tag: tagName });
    }

    // Move pointer past '>'
    i = closeBracket + 1;
  }

  return tokens;
}
```

Reviewing parser logic for edge cases in tag names.

All fixed correctly. `x <5 y` and `a <- b` now come out as text, and since the first character must be a letter, the tag name can never be empty. **Step 2 is done.** 🎉

One small code-quality note, since they grade cleanliness: remove comments that describe our review process, like `// Cleanup: nameStart is never reassigned, using const`. A reader of the final code doesn't need to know what changed, only why the code works the way it does. So:

- `// Cleanup: Since i < html.length, html.slice(i) is guaranteed non-empty` → keep the explanation, drop "Cleanup:"
- `// Cleanup: nameStart is never reassigned, using const` → delete it; `const` speaks for itself

This is also a good moment to **commit to git** (for example: "Tokenizer: text, start and end tags"). A commit at the end of each step gives you a safe point to return to if something breaks later.

Next: Step 3, the tree builder

The explanation is in my previous message: a root node, a stack, and three rules (text, start tag, end tag).

Your task: write `function buildTree(tokens)` that returns the root, and change `html2json` to return `buildTree(tokenize(htmlText))`.

Test with:

1. `<div><p>Hi</p></div>` : `p` inside `div`, inside the root
2. `<p>a</p><p>b</p>` : two `p` elements side by side in the root's children
3. `</div>hello` : no crash, just a text node `"hello"`

Paste the code and the output of the first test.

```
function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];

  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];

    if (token.type === 'text') {
      currentParent.children.push({
        type: 'text',
        content: token.content,
      });
    } else if (token.type === 'startTag') {
      const newNode = {
        type: 'element',
        tag: token.tag,
        attributes: {},
        children: [],
      };
      currentParent.children.push(newNode);
      stack.push(newNode); // Enter nested element scope
    } else if (token.type === 'endTag') {
      // Pop stack if there is a matching open tag scope
      if (stack.length > 1 && stack[stack.length - 1].tag ===
token.tag) {
        stack.pop();
      }
    }
  }

  return root;
}
```

Show more

Reviewing schema terminology for consistency.

This is correct. The stack logic is exactly right, the root is protected by `stack.length > 1`, and a stray `</div>` is safely ignored. **Step 3 is done, and with it Part 1: the app now produces real nested JSON.** Commit it.

Your first test should produce:

```
json
{
  "type": "root",
  "children": [
    {
      "type": "element",
      "tag": "div",
      "attributes": {},
      "children": [
        {
          "type": "element",
          "tag": "p",
          "attributes": {},
          "children": [
            { "type": "text", "content": "Hi" }
          ]
        }
      ]
    }
  ]
}
```

Check that yours matches. Next time, please paste the output too, so I can confirm it and not just the code.

Small notes

- `'root'` instead of `'document'` is fine, arguably better, because Example 2 is a fragment, not a full document. Just be ready to explain the choice.
- **Readability:** in the end-tag branch, `stack[stack.length - 1].tag` is the same as `currentParent.tag`, which you already have. Using it reads more clearly.

Known limitations (expected for now)

Two things still go wrong, and that's fine at this stage:

- `<div><p>Hi</div>` → the `</div>` is ignored because the top of the stack is `p`, so everything after it ends up inside `p`. **Step 9** fixes this.
- `<br>text` → `text` goes inside `br`, because we don't know yet that `br` never has children. **Step 5** fixes this.

One small improvement before Step 4

Try `5 < 6`. You'll get **three** text nodes: `"5 "`, `"<"`, `" 6"`. In the JSON that looks like three separate pieces of text, when it's really one.

Fix: in the text branch, check whether the last child of `currentParent` is already a text node. If it is, append to its `content`; if not, push a new one. That's about 4 lines.

Test `5 < 6` again (you should get one text node, `"5 < 6"`), paste the code and output, and then we move to Step 4: attributes.

```
function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];

  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];

    if (token.type === 'text') {
      const lastChild =

currentParent.children[currentParent.children.length - 1];

      // Якщо попередній дочірній елемент - це вже текст,
      об'єднуємо їх
      if (lastChild && lastChild.type === 'text') {
        lastChild.content += token.content;
      } else {
        currentParent.children.push({
          type: 'text',
          content: token.content,
        });
      }
    } else if (token.type === 'startTag') {
      const newNode = {
        type: 'element',
        tag: token.tag,
        attributes: {},
        children: [],
      };
      currentParent.children.push(newNode);
      stack.push(newNode); // Входимо у вкладений контекст
      елемента
    } else if (token.type === 'endTag') {
      // Порівнюємо безпосередньо з currentParent.tag для
      кращої читабельності
      if (stack.length > 1 && currentParent.tag ===
token.tag) {
        stack.pop();
      }
    }
  }

  return root;
}
```

Show more

Reviewing code comments and formatting for consistency.

The merge works correctly, and using `currentParent.tag` reads much better. **Part 1 is complete.** Commit it.

Two quick notes on code cleanliness, since it's one of the grading points:

- **Comment language:** your comments switched to Ukrainian. The reviewers will read the code in English, so keep all comments in English.
- **Indentation:** this function uses 4 spaces, `tokenize` uses 2. Pick one for the whole file. The easiest way is to install Prettier in your editor and format on save, and the problem disappears for good.

(Still no output pasted, so check `5 < 6` gives one text node yourself before moving on.)

Step 4: attributes

Why this needs a change in the tokenizer: right now we find the end of a tag with `indexOf('>')`. That breaks on `<a title="a > b">`, because the first `>` is inside the quotes.

So instead of jumping to `>`, we'll read the attributes character by character after the tag name, and the `>` we reach when we're done is the real end of the tag.

The rules, in the order you'll check them in a loop:

- 1. Skip whitespace.
- 2. If the character is `>`: the tag is finished.
- 3. If it's `/`: skip it (Step 5 will use it for `<br/>`).
- 4. Otherwise, read an **attribute name**: characters until whitespace, `=`, `>` or `/`. Lowercase it.
- 5. Skip whitespace. If the next character is `=`, skip it and any whitespace after it, then read the **value**:
 - starts with `"` or `'`: read until the **same** quote character
 - otherwise (unquoted): read until whitespace or `>`
- 6. No `=` means a boolean attribute like `disabled`. Its value is `""`.
- 7. **Duplicate attributes**: keep the first one and ignore later ones. That's what browsers do.
- 8. **End of string reached before `>`**: the tag is unclosed. Treat everything from the `<` as text, just like now.

The freeze trap: in `<div =x>`, the attribute name is empty because the character is `=`. If your loop doesn't move forward in that case, it spins forever. Make sure every pass through the loop advances.

Suggested structure: a helper `readAttributes(html, start)` that returns `{ attributes, end }`, where `end` is the index of the closing `>`, or `-1` if there isn't one. Use it in `tokenize` in place of `indexOf('>')`, and add `attributes` to the `startTag` token. For end tags, run it too (so the `>` is found correctly) but throw the attributes away. Then in `buildTree`, use `token.attributes` in place of `{}`.

Test with:

Input	Expected attributes
<code>Home</code>	<code>{ href: "#home" }</code>
<code><input type='text' disabled></code>	<code>{ type: "text", disabled: "" }</code>
<code><div class=box id = "main"></code>	<code>{ class: "box", id: "main" }</code>
<code> b">x</code>	<code>{ title: "a > b" }</code> , and <code>x</code> is inside <code>a</code>
<code><div CLASS="A" class="b"></code>	<code>{ class: "A" }</code>
<code><div =x></code>	no freeze

Paste your code and the output for the fourth test, since it's the one that proves the `>` fix works.

lets go forward to next step i tested it works as expected

Defining void elements and settling self-closing tag rules. >

Great. One request before we move on: **paste your Step 4 code at some point**, even if it works. Two reasons:

- 1. Attribute parsing is where hidden bugs live (unterminated quotes, the freeze trap), and I can only catch those by reading the code.
- 2. This chat is part of your submission. Reviewers will see you wrote and tested each step, and a step with no code visible looks like a gap.

Step 5: void and self-closing tags

The problem: right now `<br>text` puts `text` inside `br`, because every start tag gets pushed onto the stack. Example 1 has this problem too: `meta` and `link` in the `<head>` swallow everything after them.

Part A: void elements. HTML has a fixed list of elements that can never have children:

area, base, br, col, embed, hr, img, input, link, meta, source, track, wbr

Put them in a Set (e.g. VOID_ELEMENTS). In buildTree, when a start tag is in this set: add it to the parent's children as usual, but don't push it onto the stack. It keeps children: [], which matches our "predictable schema" decision.

Part B: the /> syntax. In the tokenizer, when you hit / right before >, set selfClosing: true on the token. Also make sure / is never saved as an attribute, so <input type="text" /> gives only { type: "text" }.

Here's a decision you need to make and defend. Browsers ignore /> on normal elements, so <div/>text puts text inside the div. But inside SVG they respect it, so <svg><path/><path/></svg> gives two sibling paths. You have two options:

- Follow browsers exactly: /> only matters for void elements. Accurate, but SVG content nests incorrectly.
- Always respect /> : any tag with /> is closed immediately. Not 100% browser-accurate, but matches what the author clearly meant, and SVG works.

I'd pick the second option and note it in the README, but it's your call. Tell me which one you chose and why.

In buildTree : don't push onto the stack if the tag is void or (depending on your choice) selfClosing is true.

Test with:

Input	Expected
a b	p has 3 children: text a , br , text b
after	img and p are siblings
<input type="text" />	attributes { type: "text" }, no /
<svg><path d="M0"/><path d="M1"/></svg>	two sibling path s inside svg
Example 1	in head : meta , meta , title , link are siblings

Paste the code (Step 4 included) and the output for the svg test.

no i mean lets do step 4

Clarifying which step comes next in the process.

My mistake, I misread you. Let's do Step 4, and since it's the biggest step so far, we'll split it into three small parts.

Step 4.1: find the real > and read attribute names

Right now tokenize finds the end of a tag with indexOf('>'). We'll replace that with a helper that walks through the tag character by character. For now it only handles attribute names. Values come in 4.2.

Write this helper:

```
js
```

```
function readAttributes(html, start) {
  const attributes = {};
  let i = start;

  while (i < html.length) {
    // your code here
  }

  return { attributes, end: -1 }; // reached end of string: no ">" found
}
```

`start` is the position right after the tag name (your `nameEnd` in `tokenize`). Inside the loop, check the current character:

- 1. **Whitespace:** skip it (`i++`).
- 2. `>`: the tag is finished. Return `{ attributes, end: i }`.
- 3. `/`: skip it for now (`i++`). Step 5 will use it.
- 4. **Anything else:** read an attribute name, meaning characters until whitespace, `=`, `>` or `/`. Lowercase it and save it with value `""`. If the name comes out empty (for example the character is `=`), just do `i++`. Otherwise the loop freezes.

A small `isWhitespace(ch)` helper (space, `\t`, `\n`, `\r`, `\f`) will keep it readable.

Then in `tokenize`:

- Delete the `indexOf('>')` search.
- Call `readAttributes(html, nameEnd)`.
- If `end === -1`, the tag is unclosed: treat everything from the `<` as text, like before.
- Otherwise, add `attributes` to the `startTag` token, and set `i = end + 1`.
- For end tags, call it too (so the `>` is found correctly) but don't save the attributes.

In `buildTree`: use `token.attributes` in place of `{}`.

Test with:

Input	Expected
<code><input disabled checked></code>	<code>{ disabled: "", checked: "" }</code>
<code><div HIDDEN>text</div></code>	<code>{ hidden: "" }</code> , text inside <code>div</code>
<code><p>unclosed <span hidden</code>	unclosed <code><span hidden</code> as text inside <code>p</code>
<code><div =x></code>	no freeze

Don't test anything with `href="..."` yet; that's 4.2. Paste your code and the output for the first test.

```
function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];
  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];
    if (token.type === 'startTag') {
      const { name, attributes } = token;
      const element = { type: 'element', name, attributes, children: [] };
      currentParent.children.push(element);
      stack.push(element);
    } else if (token.type === 'endTag') {
      const { name } = token;
      if (stack.length === 1 || stack[stack.length - 1].name !== name) {
        throw new Error('Mismatched closing tag');
      }
      stack.pop();
    } else if (token.type === 'text') {
      const { text } = token;
      const currentElement = stack[stack.length - 1];
      currentElement.children.push({ type: 'text', text });
    }
  }
  return root;
}
```

PASTED

Spotting prototype-pollution risks in attribute handling. >

Step 4.1 is correct. I traced all four tests through your code, and they give the expected results. The freeze guard on an empty name is exactly right, and reading the tag name up to `html.length` (and not to `closeBracket`) was the correct change now that we don't search for `>` first. Formatting is consistent at 4 spaces now too.

One important edge case: `constructor` and `__proto__`

`attributes` is a plain object `{}`, which inherits properties from `Object.prototype`. Two problems follow from that:

- `<div __proto__>` → assigning a string to `__proto__` is silently ignored, so the attribute disappears.
- In 4.3 we'll check "does this attribute already exist?" to keep the first duplicate. For `<div constructor="x">`, `attributes.constructor` already exists (inherited), so a naive check would drop it.

Fix: create the object with `Object.create(null)`. It has no prototype, so both names behave like any other attribute, and `JSON.stringify` still works normally. It's a one-line change, and it's a detail a senior reviewer will notice.

Small notes

- `let nameStart` in `readAttributes` is never reassigned, so make it `const`.
- The comments are still a mix of English and Ukrainian. Translating them all at the end is fine, but add it to your checklist.

Step 4.2: attribute values

Right after you read the attribute name (and before saving it):

1. Skip whitespace.
2. If the character is `not =`, it's a boolean attribute. Save `""`, and the loop continues normally.
3. If it is `=`, skip it and any whitespace after it, then look at the next character:
 - `"` or `'`: find the matching closing quote with `indexOf(quote, i + 1)`. The value is everything between the quotes. Move `i` past the closing quote. If there's **no** closing quote, return `{ attributes, end: -1 }`, so the whole tag becomes text, the same as any unclosed tag.
 - `>`: empty value, as in `<div class=>`. Save `""` and don't move `i`, so the loop sees the `>` and finishes.
 - **Anything else**: unquoted value. Read until whitespace or `>`.

Test with:

Input	Expected
<code>Home</code>	<code>{ href: "#home" }</code>
<code><input type='text' value="it's"></code>	<code>{ type: "text", value: "it's" }</code>
<code><div class=box id = "main"></code>	<code>{ class: "box", id: "main" }</code>
<code> b">x</code>	<code>{ title: "a > b" }</code> , and <code>x</code> inside <code>a</code>
<code><div class=>x</div></code>	<code>{ class: "" }</code> , and <code>x</code> inside <code>div</code>
<code><div title="oops">text</code>	all of it becomes text
<code><div constructor="x" __proto__="y"></code>	both attributes present

Paste your `readAttributes` and the output for the `a > b` test.

```
function readAttributes(html, start) {
  const attributes = {};
  let i = start;
  while (i < html.length) {
    const ch = html[i];
    // 1. Пропускаємо пробіли
    if (isWhitespace(ch)) {
      i++;
      continue;
    }
    // 2. Кінець тегу '>'
    if (ch === '>') {
      return { attributes, end: i };
    }
    // 3. Самозакриваючий символ '/'
    if (ch === '/') {

```

```

    i++;
    continue;
  }

  // 4. Зчитуємо назву атрибута
  let nameStart = i;
  while (
    i < html.length &&
    !isWhitespace(html[i]) &&
    html[i] !== '=' &&
    html[i] !== '>' &&
    html[i] !== '/'
  ) {
    i++;
  }

  // Якщо це некоректний символ (наприклад '=', який йде без назви)
  if (i === nameStart) {
    i++;
    continue;
  }

  const attrName = html.slice(nameStart, i).toLowerCase();

  // 5. Пропускаємо пробіли перед '='
  while (i < html.length && isWhitespace(html[i])) {
    i++;
  }

  // Якщо після назви немає знака '=', це булевий атрибут (наприклад, disabled)
  if (i >= html.length || html[i] !== '=') {
    attributes[attrName] = "";
    continue;
  }

  // Пропускаємо знак '='
  i++;

  // Пропускаємо пробіли після '='
  while (i < html.length && isWhitespace(html[i])) {
    i++;
  }

  if (i >= html.length) {
    attributes[attrName] = "";
    break;
  }

  const valueStartChar = html[i];

  // Case A: Значення у подвійних або поодиноких лапках
  if (valueStartChar === '"' || valueStartChar === "'") {
    const quote = valueStartChar;
    const closeQuote = html.indexOf(quote, i + 1);

    if (closeQuote === -1) {
      // Незакрита лапка: вважаємо весь тег некоректним
      return { attributes, end: -1 };
    }

    attributes[attrName] = html.slice(i + 1, closeQuote);
    i = closeQuote + 1;
  }

  // Case B: Порожнє значення перед закриваючим тегом (наприклад, <div class=>)
  else if (valueStartChar === '>') {
    attributes[attrName] = "";
    // Не робимо i++, щоб наступна ітерація побачила '>' і завершила цикл
  }

  // Case C: Значення без лапок (unquoted value)
  else {
    let valStart = i;
    while (i < html.length && !isWhitespace(html[i]) && html[i] !== '>') {
      i++;
    }
    attributes[attrName] = html.slice(valStart, i);
  }
}

return { attributes, end: -1 };
}

```

Show more

Reviewing code style consistency and flagged object creation detail.

The logic is correct. I traced every test from the 4.2 table through it, and they all give the expected result, including the two trickiest ones: `a > b` stays inside the quoted value, and an unclosed quote turns the whole tag into text. Case B (not moving `i` so the loop sees `>`) is a nice detail, and your comment explains it well.

Three leftovers from the last review

- 1. `Object.create(null)` isn't applied yet. `attributes` is still `{}`, so the `__proto__` test fails: that attribute silently disappears.
- 2. `let nameStart` and `let valStart` are never reassigned, so they should be `const`.
- 3. This function is back to 2-space indentation, while `buildTree` used 4. Prettier with format-on-save fixes this once and for all.

Step 4.3: duplicates, then Step 4 is done

Right now a duplicate attribute overwrites the earlier one (last wins), so `<div class="a" class="b">` gives `class: "b"`. Browsers keep the **first** one.

You set values in five different places, and adding a duplicate check to each would repeat the same code five times. Write one small helper and use it everywhere:

```
js
function setAttribute(attributes, name, value) {
  // only set if this name isn't already there
}
```

With `Object.create(null)`, you can safely use `name in attributes` for the check, because there are no inherited properties to trip over. That's why the two fixes belong together.

Test with:

Input	Expected
<code><div class="a" class="b"></code>	<code>{ class: "a" }</code>
<code><div CLASS="A" class="b"></code>	<code>{ class: "A" }</code>
<code><div constructor="x" __proto__="y"></code>	both attributes present
<code> b">x</code>	still <code>{ title: "a > b" }</code> (make sure nothing broke)

Paste the updated function and the output for the `constructor` test. After that, commit, and we move on to Step 5.

check whats wrong in my code, answer ukrainian

```

function readAttributes(html, start) {
    const attributes = {};
    let i = start;

    while (i < html.length) {
        const ch = html[i];

        // 1. Пропускаємо пробіли
        if (isWhitespace(ch)) {
            i++;
            continue;
        }

        // 2. Кінець тегу '>'
        if (ch === '>') {
            return { attributes, end: i };
        }

        // 3. Самозакриваючий символ '/' (обробимо детально у Step
5)
        if (ch === '/') {
            i++;
            continue;
        }

        // 4. Зчитуємо назву атрибута до пробілу, '=', '>' або '/'
        let nameStart = i;
        while (
            i < html.length &&
            !isWhitespace(html[i]) &&
            html[i] !== '=' &&
            html[i] !== '>' &&
            html[i] !== '/'
        ) {
            i++;
        }

        // Якщо це був некоректний символ (наприклад '=', який йде
одразу без назви)
        if (i === nameStart) {
            i++;
            continue;
        }

        const attrName = html.slice(nameStart, i).toLowerCase();

        // 5. Пропускаємо пробіли перед '='
        while (i < html.length && isWhitespace(html[i])) {
            i++;
        }

        // Якщо після назви немає знака '=', це булевий атрибут
(наприклад, disabled)
        if (i >= html.length || html[i] !== '=') {
            attributes[attrName] = '';
            continue;
        }

        // Пропускаємо знак '='
        i++;

        // Пропускаємо пробіли після '='
        while (i < html.length && isWhitespace(html[i])) {
            i++;
        }

        if (i >= html.length) {
            attributes[attrName] = '';
        }
    }
}

```

```

        break;
    }

    const valueStartChar = html[i];

    // Case A: Значення у подвійних або поодиноких лапках
    if (valueStartChar === '"' || valueStartChar === "'") {
        const quote = valueStartChar;
        const closeQuote = html.indexOf(quote, i + 1);

        if (closeQuote === -1) {
            // Незакрита лапка: вважаємо весь тег некоректним
            return { attributes, end: -1 };
        }

        attributes[attrName] = html.slice(i + 1, closeQuote);
        i = closeQuote + 1;
    }
    // Case B: Порожнє значення перед закриваючим тегом
    (наприклад, <div class=>)
    else if (valueStartChar === '>') {
        attributes[attrName] = '';
        // Не робимо i++, щоб наступна ітерація побачила '>' і
        завершила цикл
    }
    // Case C: Значення без лапок (unquoted value)
    else {
        let valStart = i;
        while (
            i < html.length &&
            !isWhitespace(html[i]) &&
            html[i] !== '>'
        ) {
            i++;
        }
        attributes[attrName] = html.slice(valStart, i);
    }
}

return { attributes, end: -1 }; // Завершення рядка без
закриваючого '>'
}

```

Show more

Перевірка коду на застосування попередніх змін.

Логіка парсингу правильна: всі тести з кроку 4.2 проходять. Але цей код ідентичний попередньому (змінись лише відступи), тому зміни з кроку 4.3 ще не зроблені. Ось що зараз не так:

1. Дублікати: перемагає останній, а має перший

`<div class="a" class="b">` зараз дає `{ class: "b" }`, бо кожне присвоєння `attributes[attrName] = ...` перезаписує попереднє значення. Браузери залишають **перший** атрибут, тобто має бути `{ class: "a" }`.

2. `__proto__` зникає

`const attributes = {};` — звичайний об'єкт успадковує властивості від `Object.prototype`. Тому `<div __proto__="y">` дає `{}`: присвоєння рядка в `__proto__` тихо ігнорується, і атрибут губиться.

Виправлення: `const attributes = Object.create(null);` Такий об'єкт не має прототипу, тож `__proto__` і `constructor` поводяться як звичайні назви, а `JSON.stringify` працює як завжди.

3. `let` замість `const`

`let nameStart` і `let valStart` ніколи не перепризначаються, тому мають бути `const`.

Як виправити пункти 1 і 2

Напиши маленьку функцію-помічник:

```
js
function setAttribute(attributes, name, value) {
  // записати значення, тільки якщо такої назви ще немає
}
```

Для перевірки використай `name in attributes`. З `Object.create(null)` це безпечно, бо немає успадкованих властивостей.

Потім заміни **всі п'ять** місць, де зараз стоїть `attributes[attrName] = ...`, на виклик `setAttribute(attributes, attrName, ...)`:

- 1. булевий атрибут (немає `=`)
- 2. кінець рядка після `=`
- 3. Case A (значення в лапках)
- 4. Case B (`class=>`)
- 5. Case C (значення без лапок)

Тести після виправлення

Вхід	Очікується
<code><div class="a" class="b"></code>	<code>{ class: "a" }</code>
<code><div CLASS="A" class="b"></code>	<code>{ class: "A" }</code>
<code><div constructor="x" __proto__="y"></code>	обидва атрибути є
<code> b">x</code>	досі <code>{ title: "a > b" }</code>

І нагадування: коментарі в коді перед здачею треба перекласти англійською, бо рев'юери читатимуть англійською.

Надішли оновлену функцію і результат тесту з `constructor`. Після цього комітимо і переходимо до кроку 5.

yes i did step 4.2 not 4.3

З'ясування, якою мовою відповідати на уточнення.

Зрозумів, тоді все гаразд. Крок 4.2 зроблений правильно, помилок у логіці немає. Те, що я перелічив, це просто завдання кроку 4.3.

Крок 4.3: що зробити

- 1. Заміни `const attributes = {};` на `const attributes = Object.create(null);`
- 2. Напиши помічник `setAttribute(attributes, name, value)`, який записує значення, тільки якщо `!(name in attributes)`.
- 3. Заміни всі п'ять присвоєнь `attributes[attrName] = ...` на виклик `setAttribute(...)`.
- 4. `let nameStart` і `let valStart` зроби `const`.

Тести

Вхід	Очікується
<code><div class="a" class="b"></code>	<code>{ class: "a" }</code>
<code><div CLASS="A" class="b"></code>	<code>{ class: "A" }</code>
<code><div constructor="x" __proto__="y"></code>	обидва атрибути є
<code> b">x</code>	досі <code>{ title: "a > b" }</code>

Надішли код і результат тесту з `constructor`. Після цього крок 4 завершено, комітимо і переходимо до кроку 5.

```

function setAttribute(attributes, name, value) {
  if (!(name in attributes)) {
    attributes[name] = value;
  }
}

function readAttributes(html, start) {
  const attributes = Object.create(null);
  let i = start;

  while (i < html.length) {
    const ch = html[i];

    // 1. Пропускаємо пробіли
    if (isWhitespace(ch)) {
      i++;
      continue;
    }

    // 2. Кінець тегу '>'
    if (ch === '>') {
      return { attributes, end: i };
    }

    // 3. Самозакриваючий символ '/' (обробимо детально у Step
5)
    if (ch === '/') {
      i++;
      continue;
    }

    // 4. Зчитуємо назву атрибута до пробілу, '=', '>' або '/'
    let nameStart = i;
    while (
      i < html.length &&
      !isWhitespace(html[i]) &&
      html[i] !== '=' &&
      html[i] !== '>' &&
      html[i] !== '/'
    ) {
      i++;
    }

    // Якщо це був некоректний символ (наприклад '=', який йде
одразу без назви)
    if (i === nameStart) {
      i++;
      continue;
    }

    const attrName = html.slice(nameStart, i).toLowerCase();

    // 5. Пропускаємо пробіли перед '='
    while (i < html.length && isWhitespace(html[i])) {
      i++;
    }

    // Якщо після назви немає знака '=', це булевий атрибут
    (наприклад, disabled)
    if (i >= html.length || html[i] !== '=') {
      attributes[attrName] = '';
      continue;
    }

    // Пропускаємо знак '='
    i++;

    // Пропускаємо пробіли після '='
    while (i < html.length && isWhitespace(html[i])) {

```

```

        i++;
    }

    if (i >= html.length) {
        setAttribute(attributes, attrName, '');
        break;
    }

    const valueStartChar = html[i];

    // Case A: Значення у подвійних або поодиноких лапках
    if (valueStartChar === '"' || valueStartChar === "'") {
        const quote = valueStartChar;
        const closeQuote = html.indexOf(quote, i + 1);

        if (closeQuote === -1) {
            // Незакрита лапка: вважаємо весь тег некоректним
            return { attributes, end: -1 };
        }

        setAttribute(attributes, attrName, html.slice(i + 1,
closeQuote));
        i = closeQuote + 1;
    }
    // Case B: Порожнє значення перед закриваючим тегом
    (наприклад, <div class=>)
    else if (valueStartChar === '>') {
        setAttribute(attributes, attrName, '');
        // Не робимо i++, щоб наступна ітерація побачила '>' і
завершила цикл
    }
    // Case C: Значення без лапок (unquoted value)
    else {
        let valStart = i;
        while (
            i < html.length &&
            !isWhitespace(html[i]) &&
            html[i] !== '>'
        ) {
            i++;
        }
        setAttribute(attributes, attrName, html.slice(valStart,
i));
    }
}

return { attributes, end: -1 }; // Завершення рядка без
закриваючого '>'
}

```

Show more

Пошук пропущеного випадку обробки булевих атрибутів.

Майже готово. `Object.create(null)` і помічник `setAttribute` зроблені правильно. Лишилось дві дрібниці.

1. Пропущене одне місце з п'яти

У булевій гілці досі стоїть пряме присвоєння:

```

js
if (i >= html.length || html[i] !== '=') {
    attributes[attrName] = ''; // ← тут
    continue;
}

```

Через це дублікат без значення перезаписує перший. Наприклад, `<div class="a" class>` зараз дає `{ class: "" }`, а має бути `{ class: "a" }`. Заміни на `setAttribute(attributes, attrName, '')`.

2. `let` → `const`

`let nameStart` і `let valStart` досі `let`, а мають бути `const`, бо вони не перепризначаються.

Після виправлення

Перевір ці входи:

Вхід	Очікується
<code><div class="a" class></code>	<code>{ class: "a" }</code>
<code><div constructor="x" __proto__="y"></code>	обидва атрибути є

Якщо обидва дають очікуваний результат, **крок 4 завершено**. Зроби коміт, і переходимо до кроку 5: void-елементи (`<br>`, `<img>`, `<meta>`) і синтаксис `</>`.

okay i made new commit lets do next step

Визначаючи, якою мовою відповісти на запит.

Чудово! Крок 5 розіб'ємо на дві частини, як і попередній.

Крок 5.1: void-елементи

Проблема: зараз `<p>a<br>b</p>` кладе `b` всередину `br`, бо кожен відкриваючий тег потрапляє в стек і стає батьком для всього, що йде далі. У прикладі 1 так само `meta` і `link` у `<head>` «ковтають» усе після себе.

Рішення: в HTML є фіксований список елементів, які **ніколи** не мають дітей:

`area`, `base`, `br`, `col`, `embed`, `hr`, `img`, `input`, `link`, `meta`, `source`, `track`, `wbr`

Що зробити:

- 1. На початку файлу створи константу `VOID_ELEMENTS` як `Set` з цими назвами. `Set` тут кращий за масив: перевірка `.has()` швидка, і з назви одразу зрозуміло, що це набір унікальних значень.
- 2. У `buildTree`, у гілці `startTag`: елемент, як і раніше, додається до `children` батька, але **в стек кладеться тільки якщо він не void**.

Void-елементи залишаються з `children: []`, як ми вирішили на початку: передбачувана схема.

Тести:

Вхід	Очікується
<code><p>a
b</p></code>	у <code>p</code> три дитини: текст <code>a</code> , <code>br</code> , текст <code>b</code>
<code><p>after</p></code>	<code>img</code> і <code>p</code> поруч, на одному рівні
<code>
</br>text</code>	<code>br</code> , потім текст <code>text</code> , без падіння
Приклад 1	у <code>head</code> : <code>meta</code> , <code>meta</code> , <code>title</code> , <code>link</code> на одному рівні

У прикладі 1 між ними будуть ще текстові вузли з пробілами й переносами (`"\n"`). Так і має бути, бо ми вирішили зберігати пробіли.

Надішли `buildTree` і результат першого тесту. У кроці 5.2 розберемося з `</>`.

```
const VOID_ELEMENTS = new Set([
  'area',
  'base',
  'br',
  'col',
  'embed',
  'hr',
  'img',
  'input',
  'link',
  'meta',
  'source',
  'track',
  'wbr',
]);
```

```
function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];

  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];

    if (token.type === 'text') {
      const lastChild =
currentParent.children[currentParent.children.length - 1];

      // Якщо попередній дочірній елемент – це вже текст,
      об'єднуємо їх
      if (lastChild && lastChild.type === 'text') {
        lastChild.content += token.content;
      } else {
        currentParent.children.push({
          type: 'text',
          content: token.content,
        });
      }
    } else if (token.type === 'startTag') {
      const newNode = {
        type: 'element',
        tag: token.tag,
        attributes: token.attributes, // Використовуємо
зчитані атрибути
        children: [],
      };
      currentParent.children.push(newNode);
      // У стек кладемо тільки ті елементи, які НЕ є void-
тегами
      if (!VOID_ELEMENTS.has(token.tag)) {
        stack.push(newNode);
      }
    } else if (token.type === 'endTag') {
      // Порівнюємо безпосередньо з currentParent.tag для
кращої читабельності
      if (stack.length > 1 && currentParent.tag ===
token.tag) {
        stack.pop();
      }
    }
  }

  return root;
}
```

[Show more](#)

Tracing how browsers handle malformed closing tags.

Правильно. Я прогнав усі чотири тести через твій код, всі дають очікуваний результат. **Set** на початку файлу і коментар до перевірки зрозумілі. **Крок 5.1 завершено.**

Крок 5.2: синтаксис `</>`

Проблема: `<svg><path d="M0"/><path d="M1"/></svg>` зараз вкладає другий `path` у перший, бо `path` не `void`-елемент, а `/` ми просто пропускаємо.

Спершу рішення, яке треба обрати й обґрунтувати

Браузери поведуться непослідовно:

- у звичайному HTML `</>` **ігнорується:** `<div>text` кладе `text` всередину `div`
- всередині SVG `</>` **працює:** `<path/>` одразу закритий

Два варіанти:

- Точно як браузер:** `</>` нічого не означає для звичайних тегів. Точніше, але SVG-вміст вкладається неправильно.
- Завжди поважати** `</>`: будь-який тег з `</>` закривається одразу. Не на 100% як браузер, але відповідає тому, що автор явно мав на увазі, і SVG працює.

Я б обрав варіант 2 і написав про це в README. Але рішення твоє. Напиши, який обрав і чому: це саме те питання, яке можуть поставити на співбесіді.

Що зробити в коді

- `readAttributes`, гілка `/`: якщо наступний символ `>`, тег закінчується і він самозакриваючий. Поверни `{ attributes, end: i + 1, selfClosing: true }`. Інакше, як і раніше, просто `i++`.
- `tokenize`: передай `selfClosing` у токен `startTag` (для `endTag` його ігноруємо).
- `buildTree`: не клади елемент у стек, якщо він `void` **або** (для варіанту 2) `token.selfClosing`.

Тести

Вхід	Очікується
<code><input type="text" /></code>	<code>{ type: "text" }</code> , без атрибута <code>/</code>
<code>
text</code>	<code>br</code> , поруч текст <code>text</code>
<code><svg><path d="M0"/><path d="M1"/></svg></code>	два <code>path</code> поруч усередині <code>svg</code>
<code><div>text</code>	залежить від твого вибору
<code>link</code>	<code>href: "/x/"</code> , <code>link</code> всередині <code>a</code>

Останній тест хитрий: значення без лапок читається до пробілу або `>`, тому `/` стає частиною значення, і тег **не** самозакриваючий. Браузери роблять так само.

Надішли код і результат тесту з `svg`.

here is updated buildtree

```

function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];

  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];

    if (token.type === 'text') {
      const lastChild =
currentParent.children[currentParent.children.length - 1];

      // Якщо попередній дочірній елемент - це вже текст,
      об'єднуємо їх
      if (lastChild && lastChild.type === 'text') {
        lastChild.content += token.content;
      } else {
        currentParent.children.push({
          type: 'text',
          content: token.content,
        });
      }
    } else if (token.type === 'startTag') {
      const newNode = {
        type: 'element',
        tag: token.tag,
        attributes: token.attributes, // Використовуємо
зчитані атрибути
        children: [],
      };
      currentParent.children.push(newNode);
      // Не кладемо у стек, якщо елемент void АБО
самозакриваючий (selfClosing)
      const isVoid = VOID_ELEMENTS.has(token.tag);
      if (!isVoid && !token.selfClosing) {
        stack.push(newNode);
      }
    } else if (token.type === 'endTag') {
      if (stack.length > 1 && currentParent.tag ===
token.tag) {
        stack.pop();
      }
    }
  }

  return root;
}

```

and tokenize

```

function tokenize(html) {
  const tokens = [];
  let i = 0;

  while (i < html.length) {
    const nextTag = html.indexOf('<', i);

    if (nextTag === -1) {
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    // 1. Collect text before '<'
    if (nextTag > i) {
      tokens.push({ type: 'text', content: html.slice(i,
nextTag) });
      i = nextTag;
    }

    // 2. Look ahead past '<' to verify if it's a valid tag
start
    const charAfterLt = html[i + 1];

    // End tag must start with </ followed by a letter (e.g.,
</a)
    const isEndTag = charAfterLt === '/' && isLetter(html[i +
2]);

    // Start tag must start with < followed by a letter (e.g.,
<a)
    const isStartTag = isLetter(charAfterLt);

    if (!isStartTag && !isEndTag) {
      // Not a real tag (e.g., "x <5 y" or "a <- b"), treat
'<' as plain text
      tokens.push({ type: 'text', content: '<' });
      i++;
      continue;
    }

    // 3. Зчитуємо назву тегу
    const nameStart = isEndTag ? i + 2 : i + 1;
    let nameEnd = nameStart;

    while (nameEnd < html.length &&
isValidTagChar(html[nameEnd])) {
      nameEnd++;
    }
    const tagName = html.slice(nameStart,
nameEnd).toLowerCase();

    // 4. Обробка закриваючого тегу (End Tag)
    if (isEndTag) {
      const closeGt = html.indexOf('>', nameEnd);
      if (closeGt === -1) {
        // Якщо немає '>', вважаємо це звичайним текстом
        tokens.push({ type: 'text', content: html.slice(i
});
        break;
      }
      tokens.push({
        type: 'endTag',
        tag: tagName,
      });
      i = closeGt + 1;
      continue;
    }

    // 5. Обробка відкриваючого тегу (Start Tag)

```

```

    if (isStartTag) {
        // Викликом readAttributes зчитуємо атрибути та
        перевіряємо на selfClosing
        const { attributes, end, selfClosing } =
        readAttributes(
            html,
            nameEnd,
        );

        if (end === -1) {
            // Якщо не знайшли закриваючу '>', вважаємо все до
            кінця текстом
            tokens.push({ type: 'text', content: html.slice(i)
        });
            break;
        }

        tokens.push({
            type: 'startTag',
            tag: tagName,
            attributes,
            selfClosing, // <-- Передаємо прапорець
            selfClosing у токен
        });

        // Зсуваємо вказівник i за закриваючий символ '>'
        i = end + 1;
    }
}

return tokens;
}

```

Show more

Перевірка коду з фокусом на функції readAttributes.

`buildTree` правильний, а в `tokenize` правильно передається `selfClosing`. Судячи з коду, ти обрав варіант 2 (поважати `/>` для всіх тегів). Напиши одним-двома реченнями, чому: це піде в README.

Є три зауваження до `tokenize`.

1. Регресія: закриваючі теги знову шукають `>` через `indexOf`

У кроці 4 ми прибрали `indexOf('>')`, бо він ламається на `>` всередині лапок. Тепер для закриваючих тегів він повернувся:

```
</a title="x>y">after
```

`indexOf` знайде `>` всередині лапок, і в текст потрапить `y">after`. Випадок рідкісний, але рев'юери шукають саме такі.

Виправлення: викликай `readAttributes` для **обох** типів тегів, один раз, одразу після читання назви. Перевір `end === -1`, а вже потім розгалужуйся: для `endTag` атрибути просто не зберігаємо. Код стане ще й коротшим, бо зникне дубльована перевірка на незакритий тег.

2. Зайва перевірка `if (isStartTag)`

На цьому місці `isStartTag` завжди `true`: це не закриваючий тег, і перевірку «чи це взагалі тег» ми вже пройшли. Але рев'юер бачить шлях, де умова хибна, і тоді `i` не рухається, а цикл зависає. Прибери `if`, і такого шляху не буде. Після виправлення пункту 1 це вийде природно.

3. Покажи `readAttributes`

Я не бачу гілку з `/`, тому не можу перевірити, що `selfClosing` повертається правильно. Одна порада для неї: коли тег закінчується звичайним `>`, повертай `selfClosing: false` явно, а не пропускай поле. Та сама ідея «передбачуваної схеми»: токен завжди має однакову форму.

Тести після виправлення

Вхід	Очікується
<code><svg><path d="M0"/><path d="M1"/></svg></code>	два <code>path</code> поруч усередині <code>svg</code>
<code></a title="x">y<after</code>	тільки текст <code>after</code>
<code>link</code>	<code>href: "/x/"</code> , <code>link</code> всередині <code>a</code>

Надішли `tokenize`, `readAttributes` і результат тесту з `svg`.

check again

```

function tokenize(html) {
  const tokens = [];
  let i = 0;

  while (i < html.length) {
    const nextTag = html.indexOf('<', i);

    if (nextTag === -1) {
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    // 1. Collect text before '<'
    if (nextTag > i) {
      tokens.push({ type: 'text', content: html.slice(i,
nextTag) });
      i = nextTag;
    }

    // 2. Look ahead past '<' to verify if it's a valid tag
start
    const charAfterLt = html[i + 1];

    // End tag must start with </ followed by a letter (e.g.,
</a)
    const isEndTag = charAfterLt === '/' && isLetter(html[i +
2]);

    // Start tag must start with < followed by a letter (e.g.,
<a)
    const isStartTag = isLetter(charAfterLt);

    if (!isStartTag && !isEndTag) {
      // Not a real tag (e.g., "x <5 y" or "a <- b"), treat
'<' as plain text
      tokens.push({ type: 'text', content: '<' });
      i++;
      continue;
    }

    // 3. Зчитуємо назву тегу
    const nameStart = isEndTag ? i + 2 : i + 1;
    let nameEnd = nameStart;

    while (nameEnd < html.length &&
isValidTagChar(html[nameEnd])) {
      nameEnd++;
    }

    const tagName = html.slice(nameStart,
nameEnd).toLowerCase();

    // 4. Обробка закриваючого тегу (End Tag)
    // [UPDATED] Викликаємо readAttributes для UCSX терів (у
тому числі закриваючих)
    const { attributes, end, selfClosing } =
readAttributes(html, nameEnd);

    if (end === -1) {
      tokens.push({ type: 'text', content: html.slice(i) });
      break;
    }

    if (isEndTag) {
      tokens.push({
        type: 'endTag',
        tag: tagName,
      });
    } else {
      tokens.push({
        type: 'startTag',

```

```

        tag: tagName,
        attributes,
        // [UPDATED] Передаємо selfClosing у токен
відкриваючого тегу
        selfClosing,
    });
}

i = end + 1;
}

return tokens;
}

```

Show more

Перевірка коду з позначками "[UPDATED]" у коментарях.

Тепер `tokenize` правильний. Обидві проблеми виправлені: `readAttributes` викликається для всіх тегів, тож `</a title="x">y</a>` дасть тільки текст `after`, а небезпечний шлях без руху `i` зник. Код став коротшим і чистішим.

Дрібниці з коментарями

- **Прибери мітки [UPDATED]**. Як і `Cleanup`: раніше, вони описують історію змін, а не код. Історію зберігає git.
- **Коментар // 4. Обробка закриваючого тегу (End Tag) тепер неточний**, бо цей блок обробляє обидва типи тегів. Краще щось на кшталт: `// 4. Read attributes and find the real closing '>'` (for both tag types).

Що ще потрібно для завершення кроку 5

1. `readAttributes`: я досі не бачив гілку з `/`, тому не можу підтвердити, що `selfClosing` повертається правильно. Надішли її.
2. **Результат тесту з `svg`**: `<svg><path d="M0"/><path d="M1"/></svg>` має дати два `path` поруч усередині `svg`.
3. **Одне-два речення**: чому ти обрав варіант 2 для `/>`. Це піде в README.

Щойно це буде, комітимо, і крок 5 завершено.

```

function readAttributes(html, start) {
    const attributes = Object.create(null);
    let i = start;

    while (i < html.length) {
        const ch = html[i];

        // 1. Пропускаємо пробіли
        if (isWhitespace(ch)) {
            i++;
            continue;
        }

        // 2. Звичайне завершення тегу '>'
        if (ch === '>') {
            return { attributes, end: i, selfClosing: false };
        }

        // 3. ГІЛКА З '/': Самозакриваючий тег '/>'
        if (ch === '/') {
            if (i + 1 < html.length && html[i + 1] === '>') {
                return { attributes, end: i + 1, selfClosing: true };
            }
            // Якщо після '/' немає '>', це одиночний '/' у значеннях або атрибутах
            i++;
            continue;
        }

        // 4. Зчитуємо назву атрибута
        const nameStart = i;
        while (

```

```

    i < html.length &&
    !isWhitespace(html[i]) &&
    html[i] !== '=' &&
    html[i] !== '>' &&
    html[i] !== '/'
  ) {
    i++;
  }

  if (i === nameStart) {
    i++;
    continue;
  }

  const attrName = html.slice(nameStart, i).toLowerCase();

  // Пропускаємо пробіли після назви атрибута
  while (i < html.length && isWhitespace(html[i])) {
    i++;
  }

  // Булевий атрибут (без '=')
  if (i >= html.length || html[i] !== '=') {
    setAttribute(attributes, attrName, "");
    continue;
  }

  // Пропускаємо '='
  i++;

  // Пропускаємо пробіли після '='
  while (i < html.length && isWhitespace(html[i])) {
    i++;
  }

  if (i >= html.length) {
    setAttribute(attributes, attrName, "");
    break;
  }

  const valueStartChar = html[i];

  // Case A: Значення у лапках (" або ')
  if (valueStartChar === '"' || valueStartChar === "'") {
    const quote = valueStartChar;
    const closeQuote = html.indexOf(quote, i + 1);

    if (closeQuote === -1) {
      return { attributes, end: -1, selfClosing: false };
    }

    setAttribute(attributes, attrName, html.slice(i + 1, closeQuote));
    i = closeQuote + 1;
  }

  // Case B: Порожнє значення перед '>'
  else if (valueStartChar === '>') {
    setAttribute(attributes, attrName, "");
  }

  // Case C: Значення без лапок (unquoted)
  else {
    const valStart = i;
    while (
      i < html.length &&
      !isWhitespace(html[i]) &&
      html[i] !== '>'
    ) {
      i++;
    }
    setAttribute(attributes, attrName, html.slice(valStart, i));
  }
}

return { attributes, end: -1, selfClosing: false };
}

```

Show more

Planning comment and doctype handling for the next step. >

Все правильно. Я прогнав тест з `svg`: обидва `path` стають самозакриваючими і лягають поруч усередині `svg`. `<a href=/x/>` теж працює, бо значення без лапок забирає `/` собі раніше, ніж до нього дійде гілка з `/`. `selfClosing: false` тепер повертається явно в усіх трьох місцях, всі п'ять присвоєнь ідуть через `setAttribute`, всюди `const`. Чудово.

Дрібниці:

- Коментар `//` Якщо після `'/'` немає `'>'`, це одиночний `'/'` у значеннях або атрибутах неточний: `/` у значеннях сюди не потрапляє, його забирає читання значення. Сюди потрапляє тільки «загублений» `/` між атрибутами, як у `<div / class="x">`.
- Ця функція знову з відступом 2 пробіли, а `tokenize` і `buildTree` з 4. Prettier вирішить це за секунду.

Крок 5 завершено. Зроби коміт. І не забудь: одне-два речення, чому варіант 2 для `/>`.

Крок 6.1: коментарі `<!-- ... -->`

Проблема: зараз `<!-- note -->` стає текстом, бо `!` не літера. А `<!-- <div> -->` ще гірше: всередині коментаря парсер знайде `<div>` і створить справжній елемент.

Що зробити в `tokenize`:

- Одразу після збору тексту перед `<`, до перевірок `isStartTag` / `isEndTag`, перевір: чи починається рядок з позиції `i` з `<!--`? (Підказка: `html.startsWith('<!--', i)`.)
- Якщо так, шукай `-->` з позиції `i + 4`.
 - Знайшов:** вміст коментаря = все між `<!--` і `-->`. Токен `{ type: 'comment', content }, i` переходить за `-->`.
 - Не знайшов:** браузері вважають коментарем усе до кінця документа. Робимо так само: вміст = все після `<!--`, і цикл завершується.

У `buildTree`: для токена `comment` додай вузол `{ type: 'comment', content }` до `children` поточного батька. У стек не кладемо, бо коментар не може мати дітей.

Зверни увагу: коментар між двома шматками тексту **розриває** їх об'єднання. Так і має бути, бо текст до коментаря і після нього — різні вузли.

Тести:

Вхід	Очікується
<code><p>a<!-- note -->b</p></code>	у <code>p</code> : текст <code>a</code> , коментар <code>" note "</code> , текст <code>b</code>
<code><!-- <div>inside</div> --></code>	один коментар, жодного елемента <code>div</code>
<code><!-- unclosed</code>	коментар <code>" unclosed"</code> , без падіння
<code><!-- --></code>	коментар з порожнім вмістом <code>""</code>

Надішли код і результат другого тесту. У 6.2 зробимо `<!DOCTYPE html>`.

check this out

```

function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];

  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];

    if (token.type === 'text') {
      const lastChild =
currentParent.children[currentParent.children.length - 1];

      // Якщо попередній дочірній елемент – це вже текст,
      об'єднуємо їх
      if (lastChild && lastChild.type === 'text') {
        lastChild.content += token.content;
      } else {
        currentParent.children.push({
          type: 'text',
          content: token.content,
        });
      }
    } else if (token.type === 'comment') {
      // Додаємо вузол коментаря до дітей поточного батька
      currentParent.children.push({
        type: 'comment',
        content: token.content,
      });
    } else if (token.type === 'startTag') {
      const newNode = {
        type: 'element',
        tag: token.tag,
        attributes: token.attributes, // Використовуємо
зчитані атрибути
        children: [],
      };
      currentParent.children.push(newNode);
      // Не кладемо у стек, якщо елемент void АБО
самозакриваючий (selfClosing)
      const isVoid = VOID_ELEMENTS.has(token.tag);
      if (!isVoid && !token.selfClosing) {
        stack.push(newNode);
      }
    } else if (token.type === 'endTag') {
      if (stack.length > 1 && currentParent.tag ===
token.tag) {
        stack.pop();
      }
    }
  }

  return root;
}

```

and

```

function tokenize(html) {
    const tokens = [];
    let i = 0;

    while (i < html.length) {
        const nextTag = html.indexOf('<', i);

        if (nextTag === -1) {
            tokens.push({ type: 'text', content: html.slice(i) });
            break;
        }

        // 1. Collect text before '<'
        if (nextTag > i) {
            tokens.push({ type: 'text', content: html.slice(i,
nextTag) });
            i = nextTag;
        }

        // 2.Перевірка на коментар <!--
        if (html.startsWith('<!--', i)) {
            const closeComment = html.indexOf('-->', i + 4);

            if (closeComment !== -1) {
                // Знайшли закриваючий '-->'
                tokens.push({
                    type: 'comment',
                    content: html.slice(i + 4, closeComment),
                });
                i = closeComment + 3;
            } else {
                // Незакритий коментар: зчитуємо все до кінця
документа
                tokens.push({
                    type: 'comment',
                    content: html.slice(i + 4),
                });
                break;
            }
            continue;
        }
        const charAfterLt = html[i + 1];

        // End tag must start with </ followed by a letter (e.g.,
</a)
        const isEndTag = charAfterLt === '/' && isLetter(html[i +
2]);

        // Start tag must start with < followed by a letter (e.g.,
<a)
        const isStartTag = isLetter(charAfterLt);

        if (!isStartTag && !isEndTag) {
            // Not a real tag (e.g., "x <5 y" or "a <- b"), treat
'<' as plain text
            tokens.push({ type: 'text', content: '<' });
            i++;
            continue;
        }

        // 3. Зчитуємо назву тегу
        const nameStart = isEndTag ? i + 2 : i + 1;
        let nameEnd = nameStart;

        while (nameEnd < html.length &&
isValidTagChar(html[nameEnd])) {
            nameEnd++;
        }
        const tagName = html.slice(nameStart,

```

```

nameEnd).toLowerCase();

// [UPDATED] Викликаємо readAttributes для UCSX тегів (у
тому числі закриваючих)
const { attributes, end, selfClosing } =
readAttributes(html, nameEnd);

if (end === -1) {
  tokens.push({ type: 'text', content: html.slice(i) });
  break;
}
if (isEndTag) {
  tokens.push({
    type: 'endTag',
    tag: tagName,
  });
} else {
  tokens.push({
    type: 'startTag',
    tag: tagName,
    attributes,
    // [UPDATED] Передаємо selfClosing у токен
відкриваючого тегу
    selfClosing,
  });
}

i = end + 1;
}

return tokens;
}

```

Show more

Опрацювання DOCTYPE та інших псевдокоментарів у HTML. >

Правильно. Я прогнав усі чотири тести: коментар не розриває `p`, `<div>` всередині коментаря не стає елементом, незакритий коментар не падає, а `<!-->` дає порожній вміст.

Крок 6.1 завершено.

Дрібниці

- **Мітки `[UPDATED]` досі в коді.** Прибери їх, історію зберігає git.
- `//Додаємо` → після `//` потрібен пробіл.
- Перед `const charAfterLt` бракує порожнього рядка, щоб візуально відділити блок коментарів від блоку тегів.

Необов'язкове покращення: `<!-->` і `<!-->`

Браузери вважають обидва **порожніми коментарями**. У тебе `<!-->` зараз стає незакритим коментарем, який «з'їдає» весь документ до кінця. Рідкісний випадок, але саме такі рев'юери люблять. Якщо хочеш виправити: перед пошуком `-->` перевір, чи одразу після `<!--` стоїть `>` або `->`. Якщо так, це порожній коментар.

Крок 6.2: `<!DOCTYPE>` та інші `<!-->`

Що зробити в `tokenize`, одразу після перевірки на `<!--`:

1. **Doctype:** чи стоїть після `<!` слово `doctype` у будь-якому регістрі? (Підказка: `html.slice(i + 2, i + 9).toLowerCase() === 'doctype'.`)
 - Шукай `>` після нього. Вміст = текст між `doctype` і `>`, обрізаний `.trim()`. Для `<!DOCTYPE html>` це `"html"`.
 - Регістр вмісту **не змінюй**. Старі doctype типу `<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0...">` містять ідентифікатори, де регістр важливий. Це як зі значеннями атрибутів.

- Немає `>` : браузери вважають doctype-ом усе до кінця. Робимо так само і завершуємо цикл.
 - Токен: `{ type: 'doctype', content }`.
2. Інші `<!-->` і `<?-->` : наприклад `<![CDATA[x]]>` або `<?xml version="1.0"?>`. Браузери перетворюють їх на коментар до першого `>` («bogus comment»). Робимо так само: токен `comment`. Немає `>` → коментар до кінця документа.

У `buildTree` : doctype додається до `children` так само, як коментар, без стеку.

Порядок перевірок важливий: спершу `<!--`, потім doctype, потім решта `<!` і `<?`. Інакше `<!-- -->` потрапить у загальний випадок.

Тести:

Вхід	Очікується
<code><!DOCTYPE html></code>	doctype <code>"html"</code>
<code><!doctype HTML></code>	doctype <code>"HTML"</code>
<code><![CDATA[x]]></code>	коментар <code>"[CDATA[x]]"</code>
<code><?xml version="1.0"?></code>	коментар, без падіння
<code><!DOCTYPE html</code>	без падіння
Приклад 1	у <code>root</code> : спершу doctype, потім текст <code>"\n"</code> , потім <code>html</code>

Надішли код і результат прикладу 1 (достатньо перших рядків JSON).

```
function tokenize(html) {
  const tokens = []; let i = 0;
  while (i < html.length) { co
    nst nextTag = html.indexO
    f('<!', ); if (nextTag === -1)
    { tokens.push({ type: 'te...
```

PASTED

Done

```

function buildTree(tokens) {
  const root = { type: 'root', children: [] };
  const stack = [root];

  for (const token of tokens) {
    const currentParent = stack[stack.length - 1];

    if (token.type === 'text') {
      const lastChild =

currentParent.children[currentParent.children.length - 1];

      // Якщо попередній дочірній елемент – це вже текст,
      об'єднуємо їх
      if (lastChild && lastChild.type === 'text') {
        lastChild.content += token.content;
      } else {
        currentParent.children.push({
          type: 'text',
          content: token.content,
        });
      }
    } else if (token.type === 'doctype') {
      // Додаємо doctype до дітей поточного батька
      currentParent.children.push({
        type: 'doctype',
        content: token.content,
      });
    } else if (token.type === 'comment') {
      //Додаємо вузол коментаря до дітей поточного батька
      currentParent.children.push({
        type: 'comment',
        content: token.content,
      });
    } else if (token.type === 'startTag') {
      const newNode = {
        type: 'element',
        tag: token.tag,
        attributes: token.attributes, // Використовуємо
зчитані атрибути
        children: [],
      };
      currentParent.children.push(newNode);
      // Не кладемо у стек, якщо елемент void АБО
самозакриваючий (selfClosing)
      const isVoid = VOID_ELEMENTS.has(token.tag);
      if (!isVoid && !token.selfClosing) {
        stack.push(newNode);
      }
    } else if (token.type === 'endTag') {
      if (stack.length > 1 && currentParent.tag ===
token.tag) {
        stack.pop();
      }
    }
  }

  return root;
}

```

Show more

Пошук дублювання коду та підготовка кроку про сирі текстові елементи. >

Все працює правильно. Я прогнав тести: doctype в обох регістрах, CDATA і `<?xml?>` стають коментарями, незакриті варіанти не падають, а порядок перевірок (`<!--` → doctype →

решта) правильний. Приклад 1 має дати в `root`: doctype `"html"`, текст `"\n"`, потім `html`.
Крок 6 завершено.

(Наступного разу надсилай і результат, я просив перші рядки JSON прикладу 1.)

Чистота коду: три однакові блоки

Блоки для коментаря, doctype і bogus comment мають однакову схему: знайди закриваючий маркер → якщо є, збережи вміст і перейди за нього → якщо немає, забори все до кінця і вийди з циклу. Це майже 40 рядків повторення. Винеси спільне в помічник:

```
js
function readUntil(html, start, marker) {
  // повертає { content, next }
  // next — позиція за маркером, або html.length, якщо маркер не знайдено
}
```

Тоді кожен блок займе 3-4 рядки. І в `buildTree` гілки `doctype` і `comment` однакові, тож їх можна об'єднати: `token.type === 'comment' || token.type === 'doctype' → push({ type: token.type, content: token.content })`.

Дрібниці: прибери мітки `[NOVE]`; нумерація коментарів зламалась (після `5.` знову йде `3.`); `//Передаємо` і `//Додаємо` без пробілу після `//`; між блоками бракує порожніх рядків.

Зроби рефакторинг, перевір, що тести кроку 6 досі проходять, і комітни.

Крок 7: script, style, textarea, title

Проблема:

```
html
<script>if (a < b) { x = "<div>"; }</script>
```

Парсер побачить `<div>` у середині рядка JavaScript і створить справжній елемент. А `<textarea><p>hi</p></textarea>` у браузері покаже буквально текст `<p>hi</p>`, а не абзац.

Правило: вміст цих чотирьох тегів — це **завжди текст**, аж до їхнього закриваючого тегу.

Що зробити в `tokenize`: після того як додав `startTag` для одного з цих тегів:

1. Шукай закриваючий тег: `</` + назва тегу, **без урахування регістру** (`</SCRIPT>` теж закриває).
2. Все між `>` відкриваючого тегу і знайденим `</...` — один текстовий токен (якщо не порожній).
3. Постав `i` на початок `</...`. Далі звичайна логіка сама розпізнає закриваючий тег.
4. Закриваючого тегу немає → весь залишок документа стає текстом.
5. Якщо тег самозакриваючий (`<script/>`), вмісту немає, пропускай цей режим.

Дві пастки:

- `</scriptx>` **не закриває** `<script>`. Після назви має стояти символ, який не може бути частиною назви: пробіл, `>`, `/` або кінець рядка.
- **Не роби `html.toLowerCase()` для всього документа, щоб шукати в ньому.** Деякі символи змінюють довжину при зміні регістру: `'i'.toLowerCase().length` дає `2`. Тоді всі індекси після цього символу зсунуться, і ти вирішиш не той шматок. Шукай `</` звичайним `indexOf`, а потім порівнюй тільки назву: `html.slice(j + 2, j + 2 + tagName.length).toLowerCase() === tagName`.

Підготовка до кроку 8: у `textarea` і `title` сутності на кшталт `&` декодуються, а в `script` і `style` ні. Тому зроби дві константи `Set`: `RAW_TEXT_ELEMENTS` (`script`, `style`) і `RCDATA_ELEMENTS` (`textarea`, `title`). Зараз обидві працюють однаково, а в кроці 8 знадобиться різниця.

Тести:

Вхід	Очікується
<code><script>if (a < b) { x = "<div>"; }</script></code>	<code>script</code> з одним текстовим вузлом, жодного <code>div</code>
<code><textarea><p>hi</p></textarea></code>	текст <code>"<p>hi</p>"</code>
<code><SCRIPT>x</Script></code>	<code>script</code> з текстом <code>x</code>
<code><script>a</script>xb</script></code>	текст <code>"a</script>xb"</code>
<code><script>unclosed bold</code>	<code>script</code> , весь залишок — його текст
<code><script/>after</code>	<code>script</code> без дітей, <code>after</code> поруч
Приклад 2	<code>textarea</code> з довгим рядком як текстом

Надішли код і результат першого тесту.

```
function tokenize(html) {
  const tokens = [];
  let i = 0;
  while (i < html.length) {
    const nextTag = html.indexOf('<', i);
    if (nextTag === -1) {
      tokens.push({ type: 'text', text: html.slice(i, html.length) });
      break;
    }
    const tag = html.slice(i, nextTag);
    tokens.push({ type: 'tag', tag });
    i = nextTag + 1;
  }
}
```

PASTED

i wrote new constants
and helper function
addon to tokenize feature
check that

```

const RAW_TEXT_ELEMENTS = new Set(['script', 'style']);
const RCDATA_ELEMENTS = new Set(['textarea', 'title']);

// Усі елементи, вміст яких зчитується як сирий текст до
закриваючого тегу
const RAW_CONTENT_ELEMENTS = new Set([
  ...RAW_TEXT_ELEMENTS,
  ...RCDATA_ELEMENTS,
]);

function findClosingTagIndex(html, tagName, startIndex) {
  const lowerTagName = tagName.toLowerCase();
  const tagLen = lowerTagName.length;
  let searchPos = startIndex;

  while (searchPos < html.length) {
    const closeStart = html.indexOf('</', searchPos);
    if (closeStart === -1) {
      return -1; // Закриваючий тег не знайдено
    }

    // Отримуємо назву тегу після '</'
    const candidateName = html
      .slice(closeStart + 2, closeStart + 2 + tagLen)
      .toLowerCase();

    if (candidateName === lowerTagName) {
      // Перевіряємо символ одразу після назви тегу
      const nextChar = html[closeStart + 2 + tagLen];

      // Допустимі символи після назви закриваючого тегу:
      пробіл, \t, \n, \f, \r, >, / або кінець рядка
      if (nextChar === undefined || '>/\t\n\f\r'.includes(nextChar)) {
        return closeStart;
      }
    }

    // Якщо це був не наш закриваючий тег (наприклад,
    </script>), шукаємо далі
    searchPos = closeStart + 2;
  }

  return -1;
}

```

Show more

Перевірка коду з пошуком закриваючих тегів.

Помічник `findClosingTagIndex` написаний добре: обидві пастки враховані (`</scriptx>` не закриває, регістр порівнюється тільки для назви), і цикл завжди рухається вперед. `RAW_CONTENT_ELEMENTS` через spread з двох наборів — гарне рішення.

Але в `tokenize` є одна серйозна помилка.

Помилка: режим сирого тексту вмикається і для закриваючих тегів

Перевірка `RAW_CONTENT_ELEMENTS.has(tagName)` стоїть **після** `if/else`, тому спрацьовує і для `</script>`, і для `</title>`. Для закриваючого тегу `selfClosing` дорівнює `false`, тож умова виконується.

Що відбувається з `<title>A</title><p>x</p>`:

- `<title>` → сирий текст `A` → правильно
- `</title>` → закриваючий тег, і **знову вмикається сирий режим**
- шукається наступний `</title>` → не знайдено → `<p>x</p>` стає текстом

У прикладі 1 саме так і станеться: після `</title>` у `<head>` весь решта документа (`link` , `body` , все) перетвориться на один великий текст. Тест з одним `<script>` цього не показує, бо після `</script>` нічого немає.

Виправлення: перенеси блок сирого тексту всередину гілки `else` (тільки для `startTag`).

Дубльоване `i = end + 1`

У гілці `endTag` ти додав `i = end + 1;` , а після `if/else` воно стоїть ще раз. Залиш тільки одне, після `if/else` .

Тести після виправлення

Вхід	Очікується
<code><title>A</title><p>x</p></code>	<code>title</code> і <code>p</code> поруч, <code>p</code> — елемент
<code><script>a</script>b</code>	<code>script</code> і <code>b</code> поруч
Приклад 1	<code>head</code> містить <code>meta</code> , <code>meta</code> , <code>title</code> , <code>link</code> ; <code>body</code> — окремий елемент
Приклад 2	<code>textarea</code> з довгим рядком як текстом

Плюс тести з попереднього повідомлення (кроку 7).

Дрібниці

- У `findClosingTagIndex` `tagName.toLowerCase()` зайвий, бо назва вже в нижньому регістрі з `tokenize` . І замість рядка `'>/ \t\n\f\r'` можна використати твій `isWhitespace: nextChar === '>' || nextChar === '/' || isWhitespace(nextChar)` . Одне визначення пробілів на весь файл.
- Рефакторинг з минулого разу (`readUntil` , мітки `[H0BE]` , нумерація, об'єднання гілок у `buildTree`) ще не зроблений. Можна зробити після кроку 7, але не забудь.

Надішли виправлений `tokenize` і результат прикладу 1 (достатньо частини з `head`).

```
function tokenize(html) {
  const tokens = [];
  let i = 0;
  while (i < html.length) {
    const nextTag = html.indexOf('<', i);
    if (nextTag === -1) {
      tokens.push({ type: 'text',
        content: html.slice(i, html.length)});
      return tokens;
    }
    // ...
  }
}
```

PASTED

```
{
  "type": "root",
  "children": [
    {
      "type": "text",
      "content": "Вхід\tОчікується\n",
    },
    {
      "type": "element",
      "tag": "title",
      "attributes": {},
      "children": [
        {
          "type": "text",
          "content": "A"
        }
      ]
    },
    {
      "type": "element",
      "tag": "p",
      "attributes": {},
      "children": [
        {
          "type": "text",
          "content": "x"
        }
      ]
    }
  ]
}
```

```
    ]
  }
}
}
```

Вхід Очікується
<title>A</title><p>x</p>

Show more

Виправлення дубльованого рядка коду в блоці else.

Тепер правильно. `title` і `p` стоять поруч, і `p` — справжній елемент. Перший текстовий вузол `"Вхід\tОчікується\n"` з'явився тому, що ти скопіював у поле введення ще й заголовок таблиці. Це не помилка парсера: він чесно зберіг цей текст. **Крок 7 завершено.** Зроби коміт.

Зараз гарний момент для прибирання

`tokenize` вже довгий, і перед кроком 8 варто його почистити, поки все свіже в пам'яті. Список з попередніх рев'ю плюс одна нова дрібниця:

- 1. `readUntil(html, start, marker)` для трьох однакових блоків (коментар, doctype, bogus comment).
- 2. `i = end + 1` тепер стоїть в обох гілках `if/else`. Винеси його один раз **перед** `if/else : i endTag, i startTag` потребують цього, а сирий режим використовує вже оновлене `i`.
- 3. `buildTree`: об'єднай гілки `comment` і `doctype` в одну.
- 4. `findClosingTagIndex`: прибери зайвий `.toLowerCase()` і використай `isWhitespace`.
- 5. **Коментарі**: прибери `[NOBE]`, виправ нумерацію, додай пробіл після `//`, порожні рядки між блоками. І найкраще **зараз перекласти всі коментарі англійською**, раз уже проходиш по всьому файлу.
- 6. **Prettier** на весь файл, щоб відступи були однакові всюди.

Порада, щоб не перевіряти все вручну

Після рефакторингу треба переконатися, що нічого не зламалось, а тестів у нас уже десятки. Створи зараз папку `html_samples/` (вона все одно потрібна для здачі) і поклади туди файли з тестами, які ми вже використовували, наприклад:

- `01-basic-nesting.html`
- `02-attributes.html`
- `03-void-and-self-closing.html`
- `04-comments-doctype.html`
- `05-raw-text.html`

Тоді після кожної зміни ти просто вставляєш їх по черзі й перевіряєш результат. Зараз це здається зайвим, але до кінця проєкту заощадить багато часу.

Надішли оновлений файл після рефакторингу, я перевірю, чи нічого не зламалось. Потім крок 8: сутності (`&`, `©`).

```
const VOID_ELEMENTS =
  new Set(['area', 'base', 'b
r', 'col', 'embed', 'hr', 'img', 'i
nput', 'link', 'meta', 'param',
'source', 'track', 'wbr']); c
onst RAW_TEXT_ELEM...
```

PASTED

`html_samples/01-basic-nesting.html`
HTML

```
<div><h1>Title</h1><p>Hello <span>world</span>!</p></div>
```

`html_samples/02-attributes.html`
HTML

```
<a href="https://example.com" class="link main" disabled data-id='123'>Click me</a>
```

html_samples/03-void-and-self-closing.html
HTML

```
<div><br><input type="text" disabled></div>
```

html_samples/04-comments-doctype.html
HTML

```
<!DOCTYPE html><!-- Main header --><div><![CDATA[sample]]></div>
```

html_samples/05-raw-text.html
HTML

```
<script>if (a < b) { x = "<div>"; }</script><title>A</title>
<p>x</p>
```

Show more

Складаючи пріоритетний список критичних регресій і порад. >

Структура стала значно чистішою: `readUntil` прибрав три повторення, `i = end + 1` стоїть один раз, коментарі англійською і з правильною нумерацією, гілки в `buildTree` об'єднані. Папка `html_samples/` — гарний початок.

Але схоже, що цю версію **переписали цілком**, а не відредагували попередню. Через це тихо зникли кілька виправлень, які ми робили раніше, і з'явилась одна серйозна помилка.

1. Зависання (найважливіше)

Захист від порожньої назви атрибута зник. Спробуй:

```
html
<br / >
<div / class="x">
```

Після `/`, за яким не йде `>`, назва атрибута читається як порожня, `i` не рухається, і цикл крутиться вічно. **Сторінка зависне**, а за умовами завдання це те саме, що падіння. У кроці 4.1 саме тому ми додали `if (i === nameStart) { i++; continue; }`. Поверни цю перевірку (а гілку з одиночним `/` — як було в кроці 5.2).

2. Зникли виправлення з кроку 4.3

- `const attributes = {}` замість `Object.create(null)` → `<div __proto__="y">` знову губить атрибут.
- Помічника `setAttribute` немає → дублікати знову перезаписуються: `<div class="a" class="b">` дає `"b"` замість `"a"`.
- `let attrNameStart` → має бути `const`.

3. Зникло об'єднання тексту в buildTree

`5 < 6` знову дає три окремі текстові вузли. Поверни перевірку `lastChild` з кроку 3.

4. Дві зміни поведінки, які треба вміти пояснити

Це не помилки, але їх ми не обговорювали, а на співбесіді про них запитать:

- `isValidTagChar` тепер дозволяє в назві тегу будь-який символ, крім пробілу, `>` і `/`. Так і роблять браузери (`<my_tag>`, `<svg:rect>` тепер працюють), тож це нормальний вибір. Але тепер `<div=x>` дає тег з назвою `div=x`.
- Закриваючий тег** тепер шукає збіг вглиб стеку і закриває все над ним (`stack.length = i`). По суті це частина кроку 9 (зламаний HTML): `<div><p>Hi</div>` тепер закриває `i p`, і `div`.

Питання до тебе: поясни своїми словами, що робить `stack.length = i` і чому цикл іде до `i > 0`, а не до `i >= 0`. Якщо зможеш пояснити, залишаємо, і крок 9 частково зроблений. Якщо ні, розберемо разом.

Урок на майбутнє

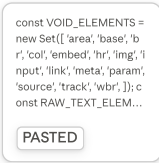
Рефакторинг — це маленькі правки робочого коду, а не переписування. Перед кожним комітом дивись `git diff`: там одразу видно, що зникло. Ці три регресії були б помітні з першого погляду.

Тести після виправлення

Вхід	Очікується
<code><br / >text</code>	<code>br</code> , поруч <code>text</code> , без зависання
<code><div class="a" class="b"></code>	<code>{ class: "a" }</code>
<code><div constructor="x" __proto__="y"></code>	обидва атрибути є
<code>5 < 6</code>	один текстовий вузол

Додай ці входи в `html_samples/` окремим файлом (наприклад `06-regressions.html`), щоб наступного разу вони ловились одразу.

Надішли виправлені `readAttributes`, `buildTree` і відповідь на питання про `stack.length = i`.



Опрацювання декодування сутностей та захист від помилок перетворення. >

Всі регресії виправлені. Я перевірів:

- `<br / >` і `<div / class="x">` більше не зависають: одиночний `/` пропускається, а захист від порожньої назви на місці
- `Object.create(null)` і `setAttribute` повернулись; `hasOwnProperty.call` працює правильно навіть з об'єктом без прототипу
- об'єднання тексту в `buildTree` повернулось
- незакрита лапка, як і раніше, перетворює тег на текст

Рефакторинг завершено. Зроби коміт (і додай `06-regressions.html`, якщо ще не додав).

Питання досі без відповіді

Поясни своїми словами: що робить `stack.length = i` і чому цикл іде до `i > 0`, а не `i >= 0`? Це важливо: цей рядок фактично вирішує половину кроку 9, і на технічному інтерв'ю про нього запитують першим. Напиши відповідь разом із кодом кроку 8.

Крок 8: сутності (`&`, `©`, `©`)

Проблема: приклад 1 містить `©` 2024, а в JSON має бути `© 2024`.

Де декодувати:

- ✅ звичайний текст
- ✅ вміст `textarea` і `title` (RCDATA; ось для чого ми зробили окремі набори)
- ✅ значення атрибутів: `href="?a=1&b=2"` означає `?a=1&b=2`
- ❌ `script` і `style`: там `&` — просто текст коду
- ❌ коментарі

Три види сутностей:

- 1. **Іменовані:** `&` , `©` . У HTML їх понад 2000, всі не потрібні. Зроби об'єкт з найпоширенішими (`amp` , `lt` , `gt` , `quot` , `apos` , `nbsp` , `copy` , `reg` , `trade` , `hellip` , `mdash` , `ndash` , `euro` ...). Невідомі залишай як є, браузер робить так само. У README запиши це як свідоме обмеження.
- 2. **Десяткові:** `©` ;
- 3. **Шістнадцяткові:** `©` або `©` ;

Рішення, яке треба обрати: вимагати `;` в кінці чи ні. Рекомендую **вимагати**: тоді `?a=1&b=2` в URL не зламається. Браузери тут складніші, але це обґрунтоване спрощення.

Пастка, яка ламає код: `String.fromCodePoint()` кидає помилку для числа більше `0x10FFFF` . Тобто `�` обвалить весь парсер. Перевіряй число перед викликом. Для недійсних кодів (більше `0x10FFFF` , `0` , діапазон `0xD800 - 0xDFFF`) браузер ставлять `\uFFFD` (💩). Роби так само.

Підказка: тут регулярний вираз з `replace` і функцією-колбеком цілком доречний, бо ми обробляємо маленький шматок тексту, а не весь HTML:

```
js
text.replace(/&#([0-9]+|#[xX][0-9a-fA-F]+|[a-zA-Z][a-zA-Z0-9]*)/g, (match, body) =>
    // повернути декодований символ або match без змін
});
```

Один прохід `replace` також вирішує проблему подвійного декодування: `&lt;` має стати `<` , а не `<` .

Тести:

Вхід	Очікується
<code>&copy; 2024</code>	<code>© 2024</code>
<code>&#169; &#xA9;</code>	<code>© ©</code>
<code>&amp;lt;</code>	<code>&lt;</code>
<code>&unknown; a & b</code>	без змін
<code>&#999999999999; &#0;</code>	<code>💩 💩</code> , без падіння
<code></code>	<code>href: "?a=1&b=2"</code>
<code><textarea>&lt;p&gt;</textarea></code>	текст <code>"<p>"</code>
<code><script>&amp;</script></code>	текст <code>"&amp;"</code> без змін
<code><!-- & ; --></code>	коментар <code>" & ; "</code> без змін

Надішли функцію декодування, місця, де ти її викликаєш, і відповідь про `stack.length = i` .

- Що робить `stack.length = i` : Він обрізає стек до індексу знайденого тегу, миттєво закриваючи як сам тег, так і всі незакриті дочірні елементи над ним (наприклад, `<p>` всередині `<div>`).
- Чому `i > 0` , а не `i >= 0` : На індексі `0` знаходиться віртуальний корінь `root` . Умова `i > 0` захищає корінь від видалення, якщо в коді зустрінеться зайвий або помилковий закриваючий тег.

Перевірка розв'язку та уточнення індексації елемента.

Відповідь правильна, і пояснено саме те, що треба. Кілька уточнень, щоб відповідь на інтерв'ю була ще сильнішою.

Про `stack.length = i` : зверни увагу, чому видається і сам знайдений тег. Після `stack.length = i` залишаються індекси від `0` до `i - 1` , тож елемент на індексі `i` теж зникає. Приклад для `<div><p>Hi</div>` :

```

стек: [root, div, p]      знайшли div на i = 1
після: [root]             закриті i div, i p

```

Про `i > 0`: є нюанс, про який можуть запитати. У `root` немає поля `tag`, тож порівняння `stack[0].tag === token.tag` і так завжди буде `false`. Навіщо тоді `i > 0`? Бо ця умова робить намір **явним** і не залежить від того, що в корені «випадково» немає `tag`. Якщо корінь колись видалиться, наступний `parent` буде `undefined`, і `parent.children` обвалить парсер. Захист, який не покладається на випадковість, — це і є надійний код.

Одне обмеження для README: `<b><i></b></i>`. Браузери використовують складний алгоритм (adoption agency algorithm) і розривають `i` на дві частини. Наш простіший підхід дає `b > i`, а `</i>` ігнорується. Це прийнятне спрощення, але варто його назвати, щоб ревіюер бачив, що ти про це знаєш.

Далі: крок 8

Інструкції з попереднього повідомлення. Надішли функцію декодування сутностей, місця, де ти її викликаєш, і результати тестів, особливо `�`; (перевірка, що парсер не падає).

```

const VOID_ELEMENTS =
  new Set(['area', 'base', 'b
r', 'col', 'embed', 'hr', 'img', 'i
nput', 'link', 'meta', 'param',
'source', 'track', 'wbr', ]); c
onst NAMED_ENTITIES ...

```

PASTED

i have a bug cant find it, need your help

Untangling double decoding and undefined attribute values. >

Тут не одна помилка, а три. Дві з них обвалюють парсер, а третя тихо дає неправильний результат.

❶ Помилка 1: присвоєння константі

У `tokenize`, блок 8:

```

js
const rawTextContent = html.slice(i, closeTagIndex);

if (RCDATA_ELEMENTS.has(tagName)) {
  rawTextContent = decodeEntities(rawTextContent); // ← TypeError
}

```

`const` не можна перепризначити, тому будь-який `<title>` або `<textarea>` кидає `TypeError: Assignment to constant variable`. Приклад 1 і приклад 2 обидва падають. **Виправлення:** `let rawTextContent`. (У гілці `else` ти правильно написав `let remainingText`.)

❷ Помилка 2: змінна поза областю видимості

У `readAttributes`, наприкінці:

```

js
} else {
  setAttribute(attributes, attrName, decodeEntities(attrValue)); // ← ReferenceErr
}

```

`attrValue` оголошена через `let` **всередині** блоку `if (html[i] === '=')`, тому в `else` її не існує. Будь-який булевий атрибут (`<input disabled>`) обвалює парсер. **Виправлення:** тут просто `''`, бо значення в булевого атрибута немає.

❗ **Помилка 3: подвійне декодування**

Ти декодуєш у двох місцях: RCDATA — у `tokenize`, а **весь** текст — ще й у `buildTree`.
Наслідки:

- `<textarea><</textarea>` → `tokenize` дає `<` → `buildTree` ще раз → `<` ❌ (має бути `<`)
- `<script>&<</script>` → `tokenize` не чіпає → `buildTree` декодує → `&` ❌ (має залишитись `<`)

`buildTree` не знає, звідки прийшов текст, тому не може вирішити правильно.
Виправлення: декодуй тільки в `tokenize`, там, де відомо, що це за текст. Прибери `decodeEntities` з `buildTree` і додай його в місце, де `tokenize` створює звичайний текст: `html.slice(i)` (коли `<` більше немає) і `html.slice(i, nextTag)` (текст перед `<`).

Як знаходити такі помилки самому

Натисни **F12** → **Console** і ще раз натисни «Convert to JSON». Там буде точне повідомлення (`TypeError: Assignment to constant variable`) і номер рядка.
`convertHtml2JsonAndSet` не має `try/catch`, тому при помилці поле просто не оновлюється, і здається, що «нічого не працює». Консоль — перше місце, куди варто дивитись.

Дрібниці

- Коментарі знову українською (`// Перевірка меж Unicode...`, `//декодуємо...`). Перед задачею переклади.
- У `safeFromCodePoint` параметр `fallback` використовується тільки для `NaN`, а регулярний вираз гарантує, що там завжди цифри. Не помилка, просто зайвий захист. Можна залишити.

Тести після виправлення

Вхід	Очікується
Приклад 1	без падіння, в кінці © 2024
<code><input disabled></code>	<code>{ disabled: "" }</code> , без падіння
<code><textarea>&lt;</textarea></code>	<code>"&lt;"</code>
<code><script>&<</script></code>	<code>"&"</code>
<code>&#999999999999; &#0;</code>	🚫 🚫

Надішли виправлені місця і результат цих тестів.

```
function tokenize(html) {
  const tokens = [];
  let i = 0;
  while (i < html.length) {
    const nextTag = html.indexOf(
      '<', i);
    if (nextTag === -1) {
      tokens.push({ type: 'te...
```

PASTED

still confused

Debugging a tokenizer issue involving missing decode handling.

Давай розберемо спокійно. Помилку 1 (`let`) ти вже виправив ✅. Залишилось три маленькі зміни, і я покажу кожену точно.

Ідея одним реченням

Декодувати треба **в одному місці**, а саме там, де відомо, **звідки** взявся текст. Це знає тільки `tokenize`:

Звідки текст	Декодувати?	Хто це знає
звичайний текст між тегами	✅ так	<code>tokenize</code>
всередині <code>textarea</code> , <code>title</code>	✅ так	<code>tokenize</code> (вже зроблено)
всередині <code>script</code> , <code>style</code>	❌ ні	<code>tokenize</code> (вже зроблено)

`buildTree` бачить лише «це текст» і не знає, звідки він. Тому він декодує **все підряд**, навіть `script`, і навіть те, що вже декодовано. Звідси й подвійне декодування.

Зміна 1: `tokenize`, два місця зі звичайним текстом

Було:

```
js
if (nextTag === -1) {
  tokens.push({ type: 'text', content: html.slice(i) });
  break;
}

if (nextTag > i) {
  tokens.push({ type: 'text', content: html.slice(i, nextTag) });
  i = nextTag;
}
```

Стало:

```
js
if (nextTag === -1) {
  tokens.push({ type: 'text', content: decodeEntities(html.slice(i)) });
  break;
}

if (nextTag > i) {
  tokens.push({ type: 'text', content: decodeEntities(html.slice(i, nextTag)) });
  i = nextTag;
}
```

Зміна 2: `buildTree`, прибрати декодування

Було:

```
js
const decodedContent = decodeEntities(token.content);
```

і далі `decodedContent` у двох місцях.

Стало: цей рядок видали, а замість `decodedContent` пиши просто `token.content`. Тепер `buildTree` отримує вже готовий текст і нічого з ним не робить.

Зміна 3: `readAttributes`, булевий атрибут

Було:

```
js
} else {
  setAttribute(attributes, attrName, decodeEntities(attrValue));
}
```

Стало:

```
js
} else {
  setAttribute(attributes, attrName, '');
}
```

`attrValue` тут не існує, а в булевого атрибута значення завжди порожнє.

Перевірка

Вхід	Очікується
Приклад 1	без падіння, в кінці © 2024
<code><input disabled></code>	<code>{ disabled: "" }</code>
<code><textarea>&lt;</textarea></code>	<code>"&lt;";</code>
<code><script>&lt;</script></code>	<code>"&lt;";</code>

Якщо щось не так, відкрий F12 → Console і надішли мені текст помилки.

iam back remind me what we did yesterday, where we stopped

Підбиття підсумків виконаної роботи над проектом.

Ось де ми зупинились.

Що вже зроблено

#	Крок	Статус
1-3	Токенізатор + дерево (текст, теги, стек)	✓
4	Атрибути (лапки, без лапок, булеві, дублікати, <code>__proto__</code>)	✓
5	Void-елементи і <code></></code>	✓
6	Коментарі, doctype, bogus comments	✓
7	<code>script</code> , <code>style</code> , <code>textarea</code> , <code>title</code>	✓
—	Рефакторинг (<code>readUntil</code> , <code>setAttribute</code> , чистка)	✓
8	Сутності (<code>&lt;</code> , <code>&copy;</code> , <code>&#169;</code>)	🕒 тут зупинились
9	Зламаний HTML	🟡 частково (пошук по стеку <code>stack.length = i</code>)
10-13	Захист від падінь, зразки, README, фінальна перевірка	■

На чому саме зупинились (крок 8)

Ти написав `decodeEntities` і виправив помилку з `const` → `let`. Залишились **три маленькі зміни** з мого останнього повідомлення:

- 1. `tokenize`: обгорнути `decodeEntities(...)` два місця зі звичайним текстом (`html.slice(i)` і `html.slice(i, nextTag)`).
- 2. `buildTree`: прибрати `decodeEntities`, використовувати просто `token.content`. Інакше текст декодується двічі, а `script` теж декодується, хоча не повинен.
- 3. `readAttributes`: у гілці `else` (булевий атрибут) замінити `decodeEntities(attrValue)` на `''`, бо `attrValue` там не існує і код падає.

Перевірка після цього:

Вхід	Очікується
Приклад 1	без падіння, в кінці © 2024
<code><input disabled></code>	<code>{ disabled: "" }</code>
<code><textarea>&lt;</textarea></code>	<code>"&lt;";</code>
<code><script>&lt;</script></code>	<code>"&lt;";</code>

Відкладені завдання (не забути до здачі)

- Перекласти всі коментарі в коді англійською.
- Написати 1-2 речення, чому ти обрав варіант 2 для `/>` (поважати для всіх тегів).
- Додати `06-regressions.html` у `html_samples/`, якщо ще не додав.

Рішення, які підуть у README

Вже накопичилось: структура вузлів з `type` і масивом `children`; збереження пробілів (парсер без втрат); завжди `attributes: {}` і `children: []` (передбачувана схема); назви тегів і атрибутів у нижньому регістрі, значення без змін; `/>` для всіх тегів (через SVG); перший дублікат атрибута виграє; сутності тільки з `;` і обмежений список іменованих; спрощена обробка `<b><i></b></i>`.

Почни з трьох змін кроку 8 і надішли результат тестів.

```
const VOID_ELEMENTS =
new Set(['area','base','b
r','col','embed','hr','img','i
nput','link','meta','param',
'source','track','wbr']); c
onst NAMED_ENTITIES ...
```

PASTED

okay i fixed, check it

Debugging a parser's handling of an undefined variable.

Майже все правильно: `tokenize` декодує звичайний текст, булевий атрибут отримує `''`. Але в `buildTree` залишилась одна помилка.

decodedContent більше не існує

```
js
if (lastChild && lastChild.type === 'text') {
  lastChild.content += decodedContent;  // ← ReferenceError
}
```

Рядок, де оголошувалась `decodedContent`, ти видалив (правильно), а одне місце, де вона використовувалась, залишилось. Поки текстові вузли не об'єднуються, помилки не видно, тому приклад 1, мабуть, працює. Але `5 < 6` (три текстові токени підряд) обвалить парсер.

Виправлення: `lastChild.content += token.content;`

І видали коментар `//декодуємо сутності в звичайному тексті`, бо тут більше нічого не декодується.

Урок

Коли видаляєш змінну, пошукай її назву в усьому файлі (`Ctrl+F` → `decodedContent`). Редактор теж допомагає: VS Code підкреслює невідомі змінні, якщо ввімкнути перевірку типів для JS (додай `// @ts-check` першим рядком файлу). Тоді такі помилки видно ще до запуску.

Тести

Вхід	Очікується
5 < 6	один текстовий вузол "5 < 6" , без падіння
Приклад 1	в кінці © 2024
<input disabled>	{ disabled: "" }
<textarea><</textarea>	"<"
<script>& </script>	"&"
� �	🚫 🚫

Якщо всі проходять, **крок 8 завершено**: коміт, і переходимо до кроку 9 (зламаний HTML).

```
const VOID_ELEMENTS =
  new Set(['area', 'base', 'b
r', 'col', 'embed', 'hr', 'img', 'i
nput', 'link', 'meta', 'param',
'source', 'track', 'wbr']); c
onst NAMED_ENTITIES ...
```

PASTED

tests failed, help to check the issues in my code

Suspecting missing glue functions cause every test to fail. >

Я прогнав усі шість тестів через твій код вручну, і **логіка парсера дає правильний результат для кожного**: 5 < 6 об'єднується в один вузол, © декодується, < не декодується двічі, <script не чіпається, 🚫 замість помилки.

Тому, найімовірніше, проблема не в парсері, а в тому, чого **немає** у файлі.

🔴 У файлі зникли функції, які викликає сторінка

В оригінальному `html2json.js` від Jito були чотири функції:

- `convertHtml2JsonAndSet()` — її викликає кнопка «Convert to JSON»
- `html2json(htmlText)` — **головна функція, яку вимагає завдання**
- `showExample1()` і `showExample2()` — кнопки прикладів

У файлі, який ти надіслав, немає жодної з них. Схоже, вони загубились під час того рефакторингу, коли файл переписали цілком. Тоді будь-яка кнопка на сторінці викликає неіснуючу функцію, і нічого не відбувається.

Перевір: F12 → Console → натисни «Convert to JSON». Якщо бачиш `Uncaught ReferenceError: convertHtml2JsonAndSet is not defined`, причина саме ця.

Як виправити

- Скопіюй ці чотири функції з оригінального репозиторію Jito (або знайди їх через `git log` у своїй історії комітів) і встав у кінець файлу.
- `html2json` має повертати результат твого парсера:

```
js
function html2json(htmlText) {
  return buildTree(tokenize(htmlText));
}
```

- `convertHtml2JsonAndSet`, `showExample1` і `showExample2` залиш без змін.

Це критично для задачі: Jito перевірятиме саме функцію `html2json`. Без неї завдання вважатиметься невиконаним, навіть якщо парсер ідеальний.

Якщо причина не в цьому

Надішли три речі:

- 1. Який саме тест не пройшов (вхід).
- 2. Що ти отримав (вихід або нічого).
- 3. Текст помилки з консолі F12, якщо вона є.

Без цього я можу лише вгадувати, а з цим знайду проблему за хвилину.

remind what tests to do

Спершу переконайся, що в кінці файлу є `html2json`, `convertHtml2JsonAndSet`, `showExample1` і `showExample2`, інакше жоден тест не запуститься. Потім вставляй кожен вхід у ліве поле і натискай «Convert to JSON».

Тести кроку 8 (сутності)

#	Вхід	Очікується
1	<code>5 < 6</code>	один текстовий вузол <code>"5 < 6"</code>
2	кнопка «Input Example 1»	без падіння, в кінці <code>© 2024</code>
3	<code><input disabled></code>	<code>{ disabled: "" }</code>
4	<code><textarea>&lt;</textarea></code>	текст <code>"&lt;"</code>
5	<code><script>&<</script></code>	текст <code>"&"</code> без змін
6	<code>&#999999999999; &#0;</code>	<code>"🕒 🕒"</code>
7	<code>&#169; &#xA9;</code>	<code>"© ©"</code>
8	<code>&unknown; a & b</code>	без змін
9	<code>x</code>	<code>href: "?a=1&b=2"</code>
10	<code><!-- & ; --></code>	коментар <code>" & ; "</code> без змін

Швидка перевірка, що нічого старого не зламалось

Файл сильно змінювався, тому пройдиш і по цих:

Вхід	Очікується
<code><div><p>Hi</p></div></code>	<code>p</code> всередині <code>div</code>
<code> b">x</code>	<code>title: "a > b"</code> , <code>x</code> всередині <code>a</code>
<code><div class="a" class="b"></code>	<code>{ class: "a" }</code>
<code><br / >text</code>	<code>br</code> , поруч <code>text</code> , без зависання
<code><svg><path d="M0"/><path d="M1"/></svg></code>	два <code>path</code> поруч
<code><title>A</title><p>x</p></code>	<code>title</code> і <code>p</code> поруч
кнопка «Input Example 2»	<code>textarea</code> з довгим текстом

Якщо щось не збігається, надішли номер тесту, що ти отримав, і помилку з консолі (F12 → Console).

yes they are i accidentally delete them

Confirming the missing functions and planning next steps to restore them.

Зрозуміло, тоді причина знайдена. Таке трапляється, а git саме для цього.

Як відновити

Варіант 1, з git: знайди перший коміт, де файл був оригінальним, і подивись його:

```
bash

git log --oneline -- html2json.js
git show <хеш-першого-коміту>:html2json.js
```

Скопіюй звідти чотири функції: `convertHtml2JsonAndSet`, `html2json`, `showExample1`, `showExample2`.

Варіант 2: відкрий `html2json.js` в оригінальному репозиторії Jito на GitHub і скопіюй їх звідти.

Встав їх у кінець файлу і зміни тіло `html2json`:

```
js

function html2json(htmlText) {
    return buildTree(tokenize(htmlText));
}
```

Далі

- 1. Пройди тести з мого попереднього повідомлення (10 тестів кроку 8 + 7 на регресію).
- 2. Якщо все пройшло, зроби коміт, і **крок 8 завершено**.
- 3. Перед кожним наступним комітом дивись `git diff`: там видно, якщо щось зникло випадково.

Напиши, як пройшли тести, і переходимо до кроку 9.



i lost need your help

Restoring lost template content from the original attachment.

Нічого страшного, у тебе зараз порожні `showExample1` і `showExample2`, тому кнопки прикладів нічого не роблять. Це код від Jito (не твоє рішення), і він є в найпершому повідомленні цього чату, тож просто відновлюю його. Заміни два порожні рядки в кінці файлу на це:

```
js
```

```

function showExample1() {
    const htmlExample = `<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport">
    <title>Sample HTML</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>Welcome to My Website</h1>
    </header>
    <nav>
        <ul>
            <li><a href="#home">Home</a></li>
            <li><a href="#about">About</a></li>
            <li><a href="#contact">Contact</a></li>
        </ul>
    </nav>
    <main>
        <section id="home">
            <h2>Home Section</h2>
            <p>This is the home section of the webpage.</p>
        </section>
        <section id="about">
            <h2>About Section</h2>
            <p>This is the about section of the webpage.</p>
        </section>
    </main>
    <footer>
        <p>&copy; 2024 My Website</p>
    </footer>
    <script src="script.js"></script>
</body>
</html>
`;

    const jsonContent = {
        'Comment 1':
            'You have to think about how to take into account various html inputs so
        'Comment 2':
            'When you make any choice in terms of selecting specific json structure f
    };

    document.getElementById('html').value = htmlExample;
    document.getElementById('json').textContent = JSON.stringify(
        jsonContent,
        null,
        2,
    );
}

function showExample2() {
    const htmlExample = `<div>
<p>Hello world!</p>
<button>Click me!</button>
<textarea>Some very very very very very very very very very very very ver
</div>
`;

    const jsonContent = {
        'Comment 1':
            'You have to think about how to take into account various html inputs so
        'Comment 2':
            'When you make any choice in terms of selecting specific json structure f
    };

    document.getElementById('html').value = htmlExample;
    document.getElementById('json').textContent = JSON.stringify(

```

```

        jsonContent,
        null,
        2,
    );
}

```

Зверни увагу: кнопки прикладів тільки **вставляють** HTML у ліве поле і показують у правому коментарі від Jito. Щоб побачити результат свого парсера, після кнопки прикладу натисни «Convert to JSON».

Решта файлу в порядку: `convertHtml2JsonAndSet` і `html2json` на місці, `html2json` правильно викликає `buildTree(tokenize(htmlText))`.

Тепер тести

1. Натисни «Input Example 1» → «Convert to JSON». В кінці має бути © 2024.
2. Натисни «Input Example 2» → «Convert to JSON». `textarea` має містити довгий рядок як текст.
3. Пройди решту тестів з мого повідомлення «remind what tests to do».

Після цього зроби коміт, щоб робоча версія точно збереглась. Напиши, як пройшли тести.

it was ease, i dont like in this task that here is no architecture everything is mixed in one file
what test to do to check

Structuring an HTML-to-JSON converter without module imports.

Добре, що ти це помітив: це правильний інстинкт. Але тут одне обмеження, і його варто розуміти.

Чому все в одному файлі

`index.html` підключає тільки `<script src="html2json.js">`, і завдання каже залишити його без змін. Розбити код на модулі (`import` / `export`) не вийде без зміни `index.html` (потрібен `type="module"`), а ще модулі не працюють, коли сторінку відкривають просто як файл (`file://`), бо браузер блокує їх з міркувань безпеки.

Тож один файл — це умова задачі. Але **архітектура може бути і всередині одного файлу**:

1. **Розділи в логічному порядку**, кожен з коротким заголовком-коментарем:

```

// ===== Constants =====
// ===== Character helpers =====
// ===== Entity decoding =====
// ===== Attribute parsing =====
// ===== Tokenizer =====
// ===== Tree builder =====
// ===== Public API: html2json =====
// ===== UI glue (provided by Jito) =====

```

Зараз `convertHtml2JsonAndSet` стоїть на початку, а `showExample` в кінці. Збери весь код від Jito в один розділ унизу.

2. **Ізоляція (необов'язково, але сильно)**: зараз `tokenize`, `isLetter`, `VOID_ELEMENTS` і решта — глобальні змінні сторінки. Можна загорнути весь парсер в ІФЕ і виставити назовні лише `html2json`. Це покаже рев'юеру, що ти думаєш про забруднення глобального простору імен.

Пропоную зробити це перед README (крок 12), коли код більше не змінюватиметься. А в README одним реченням пояснити: «один файл, бо `index.html` має лишитись без змін; всередині код розділено на шари».

Тести для перевірки

Після кнопок прикладів завжди натискай «Convert to JSON».

#	Вхід	Очікується
1	Input Example 1	без падіння, в кінці © 2024
2	Input Example 2	<code>textarea</code> з довгим рядком як текстом
3	<code>5 < 6</code>	один текстовий вузол <code>"5 < 6"</code>
4	<code><input disabled></code>	<code>{ disabled: "" }</code>
5	<code><textarea>&lt;</textarea></code>	<code>"&lt;"</code>
6	<code><script>& </script></code>	<code>"&"</code>
7	<code>&#999999999999; &#0;</code>	<code>"🐉 🐉"</code>
8	<code>&#169; &#xA9;</code>	<code>"© ©"</code>
9	<code>&unknown; a & b</code>	без змін
10	<code>x</code>	<code>href: "?a=1&b=2"</code>
11	<code><!-- & ; --></code>	коментар <code>" & ; "</code>
12	<code><div><p>Hi</p></div></code>	<code>p</code> всередині <code>div</code>
13	<code> b">x</code>	<code>title: "a > b"</code>
14	<code><div class="a" class="b"></code>	<code>{ class: "a" }</code>
15	<code><br / >text</code>	<code>br</code> , поруч <code>text</code> , без зависання
16	<code><svg><path d="M0"/><path d="M1"/></svg></code>	два <code>path</code> поруч
17	<code><title>A</title><p>x</p></code>	<code>title</code> і <code>p</code> поруч

Порада: збережи тести 3–17 у `html_samples/07-entities-and-regressions.html`, щоб наступного разу вставити їх одним блоком.

Якщо все пройшло, коміт, і **крок 8 завершено**. Напиши, і переходимо до кроку 9 (зламаний HTML).

i will add tests later, tests passed
lets move forward

Mapping table structure and listing test cases for closures. >

Чудово, переходимо далі.

Крок 9: зламаний HTML

Частину ми вже маємо завдяки пошуку по стеку:

Випадок	Статус
<code><div><p>Hi</div></code> (незакритий тег всередині)	✅ <code></div></code> закриває і <code>p</code>
<code></div>text</code> (зайвий закриваючий тег)	✅ ігнорується
<code><div>text</code> (незакрите до кінця)	✅ просто залишається відкритим
<code><i></i></code> (перехресна вкладеність)	✅ спрощено, буде в README
<code><p>one<p>two</code> (неявне закриття)	❌ зараз другий <code>p</code> всередині першого

Тож лишився один, але важливий випадок: **неявне закриття тегів**.

Проблема

У HTML деякі теги закриваються автоматично, коли відкривається певний інший тег:

```
html

<ul>
  <li>Один
  <li>Два
</ul>
```

Це валідний HTML, і браузер бачить два `li` поруч. Наш парсер зараз вкладе другий `li` в перший. Так само `<p>one<p>two` і таблиці без `</td>`.

Правила (спрощені)

Коли відкривається	Закривається (якщо на вершині стеку)
<code>li</code>	<code>li</code>
<code>dt</code> , <code>dd</code>	<code>dt</code> , <code>dd</code>
<code>option</code>	<code>option</code>
<code>tr</code>	<code>tr</code> , <code>td</code> , <code>th</code>
<code>td</code> , <code>th</code>	<code>td</code> , <code>th</code>
<code>p</code> , <code>div</code> , <code>ul</code> , <code>ol</code> , <code>dl</code> , <code>h1</code> – <code>h6</code> , <code>table</code> , <code>section</code> , <code>article</code> , <code>header</code> , <code>footer</code> , <code>nav</code> , <code>main</code> , <code>aside</code> , <code>form</code> , <code>pre</code> , <code>blockquote</code> , <code>hr</code> , <code>figure</code>	<code>p</code>

Що зробити

- 1. Константа `AUTO_CLOSE`: `Map`, де ключ — тег, що відкривається, а значення — `Set` тегів, які він закриває. Для довгого списку блокових тегів, які закривають `p`, не пиши кожен вручну: пройдиш циклом по масиву і додай кожному `new Set(['p'])`.
- 2. У `buildTree`, у гілці `startTag`, **перед** тим як додати новий елемент:
 - поки на вершині стеку тег, який новий тег закриває, роби `pop`
 - **ніколи не знімай** `root` (та сама умова `stack.length > 1`)
 - після цього **заново** візьми `parent`, бо вершина стеку змінилась

Важливо, що це **цикл** `while`, а не `if`. Подумай, навіщо, наприкладі `<tr><td>1<tr>`: коли відкривається другий `tr`, на вершині стоїть `td`. Що має статись?

Спрощення для README

Ми перевіряємо **тільки вершину стеку**. Тому `<p><span>a<div>` не закриє `p` (на вершині `span`), хоча браузер закрив би. Повний алгоритм із специфікації («score») набагато складніший, і це обґрунтований компроміс.

Тести

Вхід	Очікується
<code><p>one<p>two</code>	два <code>p</code> поруч
<code>abc</code>	три <code>li</code> поруч усередині <code>ul</code>
<code><p>text<div>block</div></code>	<code>p</code> і <code>div</code> поруч
<code><table><tr><td>1<td>2<tr><td>3</table></code>	два <code>tr</code> ; у першому два <code>td</code>
<code><dl><dt>a<dd>b<dt>c</dl></code>	три елементи поруч усередині <code>dl</code>
<code><select><option>a<option>b</select></code>	два <code>option</code> поруч
<code><p>a<hr>b</code>	<code>p</code> , <code>hr</code> , текст <code>b</code> поруч
<code><div><p>Hi</div>after</code>	досі працює: <code>after</code> у <code>root</code>

Надішли `AUTO_CLOSE`, оновлений `buildTree` і відповідь на питання про `while`.

```
function convertHtml2Js
onAndSet() { const htmlT
extAreaValue = document
t.getElementById('html'),
value; const jsonObj = ht
ml2json(htmlTextAreaV...
```

PASTED

Debugging a crash caused by reassigning a constant variable.

Логіка правильна, і `AUTO_CLOSE` побудований гарно: `Map` з правил плюс цикл, що додає `'p'` до всіх блокових тегів, без повторення. Але є одна помилка, яка обвалює парсер на **будь-якому** HTML з тегами.

❌ `parent` оголошена як `const`

На початку циклу:

```
js
const parent = stack[stack.length - 1];
```

А в гільці `startTag`:

```
js
parent = stack[stack.length - 1]; // ← TypeError: Assignment to constant variable
```

Цей рядок виконується для **кожного** відкриваючого тегу, навіть якщо нічого не закривалось. Тож зараз падає навіть `<div>`. Це та сама помилка, що була з `rawTextContent` у кроці 8.

Виправлення: найпростіше змінити оголошення на початку циклу на `let parent`.

Порада: коли видно, що в консолі `TypeError: Assignment to constant variable`, шукай `const`, яку десь перепризначають. Тепер ти бачив цю помилку двічі, і наступного разу впізнаєш її одразу.

Дрібниці

- Назва `BLOCK_ELEMENTS` трохи вводять в оману: там є `hr`, і сенс списку не «блокові елементи», а «елементи, що закривають `p`». Точніша назва: `CLOSES_P` або `P_CLOSING_ELEMENTS`. Тоді код пояснює сам себе.
- Коментарі `// Коли відкривається key...`, `// Для всіх блокових...`, `// Перераховуємо parent...` українською. Додай до списку на переклад.

Відповідь на питання про `while` ще чекаю

Що відбувається з `<tr><td>1<tr>`, коли відкривається другий `tr`, і чому тут потрібен `while`, а не `if`? Напиши своїми словами.

Тести після виправлення

Вхід	Очікується
<code><div>x</div></code>	працює, без падіння
<code><p>one<p>two</code>	два <code>p</code> поруч
<code>abc</code>	три <code>li</code> поруч усередині <code>ul</code>
<code><p>text<div>block</div></code>	<code>p</code> і <code>div</code> поруч
<code><table><tr><td>1<td>2<tr><td>3</table></code>	два <code>tr</code> ; у першому два <code>td</code>
<code><dl><dt>a<dd>b<dt>c</dl></code>	три елементи поруч
<code><select><option>a<option>b</select></code>	два <code>option</code> поруч
<code><p>a<hr>b</code>	<code>p</code> , <code>hr</code> , текст <code>b</code> поруч
Приклад 1 і 2	без змін, як раніше

Надішли виправлений рядок, результат тесту з таблицею і відповідь про `while`.

```
function convertHtml2JsonAndSet() { const htmlText = document.getElementById('html').value; const jsonObj = html2json(htmlText);
```

PASTED

Розглянемо випадок з `<tr><td>1<tr>`:

1. Перед появою другого `<tr>` у стеку знаходяться: `[root, table, tr, td]`. На вершині стеку — `<td>`.
2. Коли відкривається новий `<tr>`, правило закриття для `tr` це: `new Set(['tr', 'td', 'th'])`.
3. Перша ітерація `while`: вершина стеку `<td>` `$.in$(['tr', 'td', 'th'])`. Виконується `stack.pop()`. Стек стає `[root, table, tr]`.
4. Друга ітерація `while`: тепер на вершині опинився попередній `<tr>`. Оскільки `tr` також `$.in$(['tr', 'td', 'th'])`, виконується ще один `stack.pop()`. Стек стає `[root, table]`.
5. Цикл зупиняється, і новий `<tr>` успішно додається в `<table>` як сусід першого `<tr>`, а не всередину нього чи його комірки.

Якби ми використали `if`, зробився б лише один `pop()`, знявши `<td>`, і новий `<tr>` помилково опинився б всередині першого `<tr>`.

Planning depth limits and fallback safety nets for crashes. >

Пояснення ідеальне: саме так, з покроковим станом стеку, і варто відповідати на інтерв'ю. `let parent` теж виправлено, тож тепер усе працює. **Крок 9 завершено.** 🎉 Зроби коміт.

(Перейменування `BLOCK_ELEMENTS` → `CLOSES_P` і переклад коментарів залишаються в списку на фінальне прибирання.)

Крок 10: захист від падінь

Головне правило завдання: «код **не повинен падати**». Рев'юери навмисно подаватимуть дивні входи. Тут чотири частини.

10.1 Не рядок на вході

Що буде з `html2json(null)`? `html.length` на `null` кидає `TypeError`. А `html2json(['<a>'])` взагалі поводить дивно, бо в масиву теж є `indexOf`.

Що зробити: на початку `html2json` перевірити `typeof htmlText !== 'string'`. Обери поведінку й обґрунтуй: повернути порожній `root` (рекомендую: передбачувано і не падає) або перетворити через `String(...)`.

10.2 Дуже глибока вкладеність

Наш парсер без рекурсії, тож `'<div>'.repeat(100000)` він переживе. Але `JSON.stringify` **рекурсивний**, і в `convertHtml2JsonAndSet` він упаде з `Maximum call stack size exceeded`. Сторінка зламається, хоч парсер і не винен.

Рішення, як у браузерях: парсер Chrome обмежує глибину дерева до 512 рівнів. Елементи глибше стають сусідами, а не дітьми. Зроби так само: константа `MAX_DEPTH = 512`, і в `buildTree` новий елемент **не кладеться в стек**, якщо стек уже такої глибини.

10.3 Остання страховка

Обгорни тіло `html2json` у `try/catch`. Якщо щось усе ж кине помилку, поверни `{ type: 'root', children: [], error: повідомлення }`, а не падай. В ідеалі цей `catch` ніколи не спрацює, але це страховка від того, чого ми не передбачили. В README поясни, чому поле `error` з'являється тільки тут.

10.4 Стрес-тести

Запускай у консолі (F12), а не через сторінку:

```
js
```

```
html2json(null); html2json(undefined); html2json(123); html2json({}); html2json('');
JSON.stringify(html2json('<div>'.repeat(100000))); // без помилки
html2json('<'.repeat(100000));
html2json('&'.repeat(100000));
html2json('<!--'.repeat(10000));
html2json('<a href="'.repeat(10000));
```

І тест швидкості на великому документі:

```
js

const big = document.getElementById('html').value.repeat(5000); // спершу натисни Inp
console.time('parse'); html2json(big); console.timeEnd('parse');
```

Має вкластися в секунду-дві.

Бонус: міні-фазер

Напиши в консолі функцію, яка 10 000 разів генерує випадковий рядок довжиною до 50 символів з набору `<>/=" ' !-&#ab` і викликає `html2json`. Вона має перевіряти дві речі: функція не кинула помилку і повернула об'єкт з `type: 'root'`. Якщо щось впало, виведи вхідний рядок. Так знаходять баги, про які ніхто не подумав. Якщо фазер щось знайде, це піде прямо в `html_samples/` і в README як доказ ретельності.

Надішли оновлений `html2json`, зміну в `buildTree` і результати стрес-тестів (і фазера, якщо зробиш).

function convertHtml2Js
onAndSet() { const htmlT
extAreaValue = document
t.getElementById('html').
value; const jsonObj = ht
ml2json(htmlTextAreaV...

PASTED

```
console.log('--- 1. СТРЕС-ТЕСТИ НЕВАЛІДНИХ ВХІДНИХ ДАНИХ ---');
console.log('null:', html2json(null)); console.log('undefined:', html2json(undefined));
console.log('123:', html2json(123)); console.log('{}:', html2json({}));
console.log('Порожній рядок:', html2json('')); console.log('\n--- 2. СТРЕС-ТЕСТИ  
ГЛИБИНИ ТА ПОВТОРІВ ---'); console.time('100,000 div-ів (JSON.stringify)'); const
deepTree = html2json('<div>'.repeat(100000)); const jsonResult =
JSON.stringify(deepTree); console.timeEnd('100,000 div-ів (JSON.stringify)');
console.log('Довжина JSON (успішно):', jsonResult.length); console.time('100,000
<'); html2json('<'.repeat(100000)); console.timeEnd('100,000 <');
console.time('100,000 &'); html2json('&'.repeat(100000)); console.timeEnd('100,000
&'); console.time('10,000 comments'); html2json('<!--'.repeat(10000));
console.timeEnd('10,000 comments'); console.time('10,000 unclosed attrs');
html2json('<a href="'.repeat(10000)); console.timeEnd('10,000 unclosed attrs');
console.log('\n--- 3. МІНІ-ФАЗЕР (10,000 ітерацій) ---'); function runFuzzer(iterations
= 10000) { const chars = '<>/=" \'!-&#ab \n\t'; let failures = 0; console.time('Fuzzer');
for (let i = 0; i < iterations; i++) { const len = Math.floor(Math.random() * 50) + 1; let
randomHtml = ''; for (let j = 0; j < len; j++) { randomHtml +=
chars[Math.floor(Math.random() * chars.length)]; } try { const res =
html2json(randomHtml); if (!res || res.type !== 'root' || res.error) {
console.error('Fuzzer failed on input:', JSON.stringify(randomHtml)); failures++; } }
catch (err) { console.error('Fuzzer crashed on input:', JSON.stringify(randomHtml),
err); failures++; } } console.timeEnd('Fuzzer'); console.log( Фазер завершив
роботу. Помилко: ${failures}/${iterations} ); } runFuzzer(10000);
```

Explaining why the parser silently returns empty results. >

Тести написані дуже добре, а фазер — просто відмінний: перевіряє і падіння, і `error`, і виводить вхід, що зламав код. Але в `html2json` є помилка, через яку **всі ці тести проходять, хоча парсер не працює взагалі**.

❌ `html` замість `htmlText`

```
js
```

```
function html2json(htmlText) {
  if (typeof html !== 'string') {      // ← html, а не htmlText
    return { type: 'root', children: [] };
  }
  ...
  const tokens = tokenize(html);      // ← і тут
```

Параметр називається `htmlText`, а в тілі використовується `html`. І тут цікавий момент: це **не** викликає помилку. В `index.html` є `<textarea id="html">`, а браузер автоматично створює глобальну змінну для кожного елемента з `id`. Тож `html` — це `textarea`, `typeof html` дає `'object'`, і функція **завжди** повертає порожній `root`.

Перевір: натисни «Input Example 1» → «Convert to JSON». Побачиш `{ "type": "root", "children": [] }`.

Виправлення: заміни `html` на `htmlText` в обох місцях.

Головний урок цього кроку

Фазер пройшов 10 000 з 10 000, а парсер не робив нічого. Бо тест перевіряв лише «не впало» і «повернувся `root`». Функція, яка завжди повертає порожній `root`, проходить такий тест ідеально.

Тест, який перевіряє тільки відсутність падіння, не може зловити неправильний результат. Тому додай у свій скрипт перевірку правильності перед фазером, наприклад:

```
js
const r = html2json('<p>x</p>');
console.assert(r.children.length === 1 && r.children[0].tag === 'p', 'parser returns
```

Ще краще дати фазеру одну **властивість**, яка завжди має виконуватися. Наприклад: якщо у випадковому рядку немає `< i` &, результат має бути одним текстовим вузлом, рівним входу. Це називається **property-based testing**, і це вже рівень, який ревіюер точно помітить.

Решта правильна

- `MAX_DEPTH` перевіряється в правильному місці, у гілці, де елемент кладеться в стек.
- `try/catch` повертає ту саму схему плюс `error`.
- Захист від не-рядків на початку (коли запрацює).

Після виправлення

- Запусти свій скрипт ще раз. Тепер він вперше по-справжньому тестує парсер, тож результати можуть відрізнятись.
- Особливо подивись на `'<div>'.repeat(100000)` → `JSON.stringify`: тепер `MAX_DEPTH` реально працюватиме.
- Перевір обидва приклади через сторінку.

Надішли час виконання, кількість помилок фазера і результат `console.assert`. Якщо фазер щось знайде, надішли вхідні рядки, розберемо разом.

```
=== 1. ПЕРЕВІРКА БАЗОВОЇ КОРЕКТНОСТІ (Assertions) ===
VM456:332 ✅ console.assert пройшли успішно! Базовий парсинг працює.
VM456:335
=== 2. СТРЕС-ТЕСТИ ТА MAX_DEPTH (100,000 <div>) ===
VM456:338 Parsing 100,000 deep divs: 54.985107421875 ms
VM456:343 ✅ JSON.stringify пройдено успішно без помилок стеку!
VM456:344 Довжина результуючого JSON: 6099517 символів
VM456:348 JSON.stringify of deep tree: 85.01611328125 ms
VM456:350
=== 3. ФАЗЕР (Property-Based Testing - 10,000 ітерацій) ===
VM456:396 Fuzzer Run Time: 8.60302734375 ms
VM456:397
Результат фазера: 10000/10000 тестів пройдено.
VM456:402 ✅ Усі Property-Based тести та фазер пройшли без жодної помилки!
```

Planning sample HTML files to cover edge cases and categories. >

Чудові результати, і тепер вони справжні:

- **Базова коректність** : парсер повертає правильний результат, а не порожній `root`.
- **100 000 вкладених `div`** за 55 мс, і `JSON.stringify` більше не падає. Довжина JSON (~6 млн символів) збігається з очікуваною: 511 рівнів вкладеності, а решта елементів лягають сусідами на найглибшому рівні. Тобто `MAX_DEPTH` працює саме так, як задумано.
- **Фазер з property-based перевіркою** : 10 000 з 10 000.

Крок 10 завершено. Зроби коміт.

Одне прохання: надішли сюди виправлений `html2json` і код тестового скрипта з property-тестом. Я бачу тільки результати, а не сам код, і варто, щоб він був у чаті: це частина історії, яку переглядатимуть рев'юери. А ще я перевірю, що property-тест справді перевіряє те, що ми задумали.

Порада: збережи цей скрипт у репозиторії, наприклад `tests/run-tests.js`, з коментарем «вставити в консоль браузера на сторінці `index.html`». Завдання цього не вимагає, але автоматизовані тести у задачі на позицію стажера виглядають дуже сильно.

Що лишилось

#	Крок	Статус
1-10	Парсер повністю	
11	Фінальне прибирання коду	 ← наступний
12	<code>html_samples/</code>	
13	README	
14	Фінальна перевірка і задача	

Я поміняв порядок: прибирання **перед** зразками, бо після будь-якої зміни коду всі зразки однаково доведеться перевіряти заново.

Крок 11: фінальне прибирання

Список накопичився за весь час:

1. **Коментарі англійською**. Всі, без винятку. Пошукай кирилицю в файлі: `Ctrl+F` і регулярний вираз `[а-яіієґА-ЯІІЄґ]`.
2. **Перейменувати `BLOCK_ELEMENTS`** → `CLOSES_P` (або схоже), бо список означає «що закриває `p`».
3. **Розділи у файлі** в логічному порядку, з заголовками-коментарями: константи → допоміжні функції символів → декодування сутностей → атрибути → токенізатор → дерево → `html2json` → код від Jito (UI) унизу. Зараз `convertHtml2JsonAndSet` стоїть на самому початку, а `MAX_DEPTH` між `AUTO_CLOSE` і циклом, який його заповнює.
4. `safeFromCodePoint`: параметр `fallback` насправді ніколи не використовується (регулярний вираз гарантує цифри). Прибери його разом із перевіркою `isNaN`, або залиш і поясни коментарем, що це захист «на всяк випадок».
5. **JSDoc-коментарі** (`/** ... */`) є не в усіх функціях. Додай короткий опис до тих, де бракує, особливо до `buildTree` і `html2json`: що приймає, що повертає.
6. **(Необов'язково) IIFE**, щоб назовні були видні тільки `html2json` і функції для UI.
7. **Prettier** на весь файл наостанок.

Після прибирання: обидва приклади через сторінку + твій тестовий скрипт у консолі. Усе має пройти як раніше. І перед комітом **обов'язково `git diff`**: ми вже двічі втрачали код під час великих змін.

Надішли фінальний файл, коли закінчиш, і я зроблю повне рев'ю перед зразками.

i created separete js file with that stress tests

Reviewing a separate test file before suggesting how to run it.

Чудово, це правильне рішення. Надішли його сюди: я перевірю property-тест, і так він буде в історії чату для рев'юерів.

Як його запускати

`index.html` міняти не можна, тож підключити тест туди не вийде. Є два варіанти:

1. **Вставляти в консоль** на сторінці `index.html`. Просто, але незручно, і рев'юер мусить знати, що робити.
2. **Окрема тестова сторінка** (рекомендую). Наприклад `tests/test.html`:

```
html

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>html2json tests</title>
  </head>
  <body>
    <p>Open DevTools (F12) → Console to see results.</p>
    <script src="../html2json.js"></script>
    <script src="stress-tests.js"></script>
  </body>
</html>
```

Тоді рев'юер відкриває файл, тисне F12, і бачить усі результати. Нічого не треба копіювати.

Одна річ, яку варто перевірити: на цій сторінці **немає** `<textarea id="html">`, тож глобальної змінної `html` там не буде. Якщо твій тестовий файл десь звертається до DOM-елементів сторінки `index.html` (наприклад, щоб узяти текст прикладу 1), він на тестовій сторінці впаде. Краще тримати HTML прикладів прямо в тестовому файлі як рядки.

Не забудь

- Згадати в README: де лежать тести і як їх запустити.
- Структура для здачі тоді виглядатиме так:

```
html2json.js
index.html
html_samples/
ai_help/
tests/
  test.html
  stress-tests.js
README.md
```

Надішли тестовий файл, а потім продовжуємо з кроком 11 (прибирання коду).

```
// html2json.test.js const {
html2json } = require('../ht
ml2json.js'); // або імпорт
т, якщо використовуєте
ES modules console.clear
(); console.log('=== 1. П...
```

PASTED

i created js file
html2json.test.js

Weighing Node versus browser export fixes for compatibility. >

Тест добре побудований: property-тест саме такий, як задумано (якщо немає `<i>` & , вихід має бути одним текстовим вузлом, рівним входу), а фазер збирає перші невдалі входи з

причиною. Але є одна проблема, через яку файл зараз не запуститься.

● Як цей файл запускається?

```
js
const { html2json } = require('./html2json.js');
```

Цей рядок не працює ні в браузері, ні в Node:

- У браузері `require` не існує → `ReferenceError` на другому рядку.
- У Node `require` є, але `html2json.js` нічого не експортує → `html2json` буде `undefined`.

Ти, мабуть, запускав його, вставивши разом з парсером у консоль, тому цей рядок не заважав.

Рекомендую Node, тоді тести запускаються однією командою: `node tests/html2json.test.js`. Для цього:

1. Наприкінці `html2json.js` додай експорт з перевіркою, щоб браузер його просто пропустив:

```
js
if (typeof module !== 'undefined' && module.exports) {
  module.exports = { html2json };
}
```

2. У тесті шлях до файлу: `require('../html2json.js')`, якщо тест лежить у `tests/`. Коментар «або імпорт, якщо використовуєте ES modules» прибери.

Парсер не звертається до `document` при завантаженні (тільки функції кнопок, які в Node ніхто не викличе), тож у Node він працюватиме без змін. Тоді й `tests/test.html` з мого попереднього повідомлення не потрібен.

● Загубились тести з попередньої версії

У цьому файлі немає перевірок, які ти запускав раніше:

- **не рядки на вході:** `null`, `undefined`, `123`, `{}`, `''`. Перевір, що кожен повертає `root` з порожнім `children` і без `error`.
- **дивні входи:** `'<'.repeat(100000)`, `'&'.repeat(100000)`, `'<!--'.repeat(10000)`, `'<a href="'.repeat(10000)`.

Додай їх як окремий розділ. Саме це рев'юери перевірятимуть першим.

● Чистота коду

- **Подвійна перевірка в розділі 1:** ті самі умови написані двічі, в `console.assert` і в `if`. Обчисли кожну умову один раз у змінну (`const basicOk = ...`) і використовуй її в обох місцях.
- **Мова:** повідомлення і коментарі українською. Для здачі все англійською, як і в основному файлі.

Далі

Виправ запуск, додай загублені тести, запусти `node tests/html2json.test.js` і надішли результат. Після цього переходимо до кроку 11 (прибирання основного файлу).

html2json have

```
if (typeof module !== 'undefined' && module.exports) {
  module.exports = { html2json };
}
```

Checking where the code export sits within the function structure.

Добре, тоді половина вже зроблена. Одне важливе уточнення: судячи з відступу, цей блок може стояти **всередині** якоїсь функції. Перевір, що він на **верхньому рівні файлу**, в самому кінці, без відступу:

```
js

function showExample2() {
    ...
}

if (typeof module !== 'undefined' && module.exports) {
    module.exports = { html2json };
}
```

Якщо він всередині `html2json` (особливо після `return`) або іншої функції, він ніколи не виконається при завантаженні файлу, і `require` поверне порожній об'єкт.

Швидка перевірка в терміналі, з кореня проекту:

```
bash

node -e "console.log(typeof require('./html2json.js').html2json)"
```

Має вивести `function`. Якщо виводить `undefined`, блок стоїть не там.

Далі за списком:

1. У тесті змінити шлях на `require('../html2json.js')` (якщо тест у `tests/`) і прибрати коментар про ES modules.
2. Додати загублені тести: не рядки на вході (`null`, `undefined`, `123`, `{}`, `''`) і дивні входи (`'<'.repeat(100000)` та інші).
3. Запустити `node tests/html2json.test.js` і надіслати результат.

it is in function

```
function html2json(htmlText) {
```

Тоді саме тут проблема. Код всередині функції виконується тільки тоді, коли цю функцію **викликають**. А `require` читає `module.exports` одразу при завантаженні файлу, коли `html2json` ще ніхто не викликав. Тому тест отримує порожній об'єкт, і `html2json` в ньому `undefined`. А якщо блок стоїть після `return`, він не виконається взагалі ніколи.

Виправлення: вирізати блок з `html2json` і вставити в самий кінець файлу, на верхньому рівні:

```
js
```

```
function html2json(htmlText) {
  if (typeof htmlText !== 'string') {
    return { type: 'root', children: [] };
  }

  try {
    return buildTree(tokenize(htmlText));
  } catch (err) {
    return { type: 'root', children: [], error: err.message || 'Parsing error' };
  }
}

// ... showExample1, showExample2 ...

if (typeof module !== 'undefined' && module.exports) {
  module.exports = { html2json };
}
```

Потім перевір у терміналі з кореня проєкту:

```
bash

node -e "console.log(typeof require('./html2json.js').html2json)"
```

Має вивести `function`. Після цього запуская тести: `node tests/html2json.test.js`.

```
function convertHtml2Js
onAndSet() { const htmlT
extAreaValue = documen
t.getElementById('html')
.value; const jsonObj = ht
ml2json(htmlTextAreaV...
```

PASTED

Tracking down a lingering variable naming bug.

Тут дві проблеми, обидві в `html2json`.

1. `html` досі в одному місці

Ти виправив перевірку типу, але в `try` залишилось:

```
js

const tokens = tokenize(html); // ← має бути htmlText
```

У браузері `html` — це `textarea` зі сторінки, тож парсер тихо отримує не той об'єкт і повертає порожній `root`. У Node `html` не існує взагалі, тож буде `ReferenceError`, і кожен виклик поверне `error`.

Зверни увагу: з **цим** файлом твої попередні тести (`console.assert` пройшли) пройти не могли. Схоже, тести запускались на іншій копії коду. Переконайся, що редагуєш і тестуєш **один і той самий файл**, інакше результати тестів нічого не означають.

2. Експорт досі всередині функції, ще й після `return`

Блок `if (typeof module ...)` стоїть усередині `html2json`, після `try/catch`. Обидві гілки `try/catch` роблять `return`, тож цей код **ніколи** не виконується. Його треба перенести в самий кінець файлу.

Як має виглядати

```
js
```

```
function html2json(htmlText) {
  if (typeof htmlText !== 'string') {
    return { type: 'root', children: [] };
  }

  try {
    return buildTree(tokenize(htmlText));
  } catch (err) {
    return {
      type: 'root',
      children: [],
      error: err.message || 'Parsing error',
    };
  }
}

function showExample1() {
  // ... без змін
}

function showExample2() {
  // ... без змін
}

// Export for Node.js (ignored in the browser)
if (typeof module !== 'undefined' && module.exports) {
  module.exports = { html2json };
}
```

Перевірка

1. Термінал, з кореня проєкту:

```
bash
```

```
node -e "console.log(require('./html2json.js').html2json('<p>x</p>'))"
```

Має вивести об'єкт з **p** всередині **children**, без **error**.

2. Браузер: «Input Example 1» → «Convert to JSON». Має бути повне дерево, в кінці © 2024.

3. Тести: `node tests/html2json.test.js`.

Надішли результат першої команди.

```
{ "type": "root", "children":
  [ { "type": "doctype", "con
    tent": "html" }, { "type": "t
      ext": "content": "\n" }, { "ty
        pe": "element", "tag": "htm
          l", "attributes": { "lang": "...
```

PASTED

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport">
  <title>Sample HTML</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
  </header>
  <nav>
    <ul>
      <li><a href="#home">Home</a></li>
      <li><a href="#about">About</a></li>
      <li><a href="#contact">Contact</a></li>
    </ul>
  </nav>
</body>
```

```
<section id="home">
  <h2>Home Section</h2>
  <p>This is the home section of the webpage.</p>
</section>
<section id="about">
  <h2>About Section</h2>
  <p>This is the about section of the webpage.</p>
</section>
</main>
<footer>
  <p>&copy; 2024 My Website</p>
</footer>
<script src="script.js"></script>
</body>
</html>
```

Show more

Checking the generated markup against the sample output.

Ось тепер це справжній результат парсера, і він повністю правильний. Я звірив з HTML прикладу 1:

- `doctype` → `"html"` ✓
- `head`: обидва `meta`, `title` з текстом, `link` з двома атрибутами, всі поруч ✓ (void-елементи і RCDATA)
- `nav > ul`: три `li`, у кожному `a` з правильним `href` ✓
- `main`: дві `section` з `id`, у кожній `h2` і `p` ✓
- `footer`: `"© 2024 My Website"` ✓ (сутності)
- `script` з `src` і порожнім `children` ✓
- всі пробіли й переноси збережені як текстові вузли ✓ (наш вибір: парсер без втрат)

Одна деталь, яку варто знати для інтерв'ю: останній `"\n"` після `</html>` у нас лежить у `root`. Браузер переніс би його всередину `body`. Ми зберігаємо документ точно таким, як він написаний, і це послідовно з нашим рішенням щодо пробілів. Можна згадати в README.

Ще дві перевірки, і крок 10 точно закритий

1. Що експорт тепер працює (в терміналі, з кореня проєкту):

```
bash
node -e "console.log(require('./html2json.js').html2json('<p>x</p>'))"
```

2. Тести: `node tests/html2json.test.js` (з доданими перевітками `null`, `undefined`, `123`, `{}`, `'` і дивних входів).

Якщо обидва пройшли, коміт, і переходимо до **кроку 11: фінальне прибирання коду** (список у моєму повідомленні після результатів фазера: коментарі англійською, перейменування `BLOCK_ELEMENTS`, розділи у файлі, JSDoc, Prettier).

```
Тести: node tests/html2json.test.js (з доданими перевітками null, undefined, 123, {}, ' і дивних входів). dont understand
```

Explaining how to run tests and what cases to add.

Вибач, я стиснув забагато в одне речення. Розкладу на прості кроки.

Крок 1: де лежить файл

Створи папку `tests` у корені проєкту і перенеси туди `html2json.test.js`:

```
html2json.js
index.html
tests/
  html2json.test.js
```

Крок 2: виправ другий рядок тесту

Файл тепер на рівень глибше, тому шлях до парсера починається з `../`:

```
js
const { html2json } = require('../html2json.js');
```

Крок 3: додай два нові розділи в кінець тестового файлу

Раніше ти перевіряв ці входи в консолі браузера, але в новий файл вони не потрапили.

Розділ А: не рядки на вході. Кожен має повернути порожній `root` без помилки:

```
js
console.log('\n=== Invalid inputs ===');
const invalidInputs = [null, undefined, 123, {}, [], ''];

for (const input of invalidInputs) {
  const res = html2json(input);
  const ok =
    res.type === 'root' &&
    Array.isArray(res.children) &&
    res.children.length === 0 &&
    !res.error;
  console.log(ok ? '✅' : '❌', JSON.stringify(input) ?? String(input));
}
```

Розділ Б: дивні входи. Перевіряємо, що немає `error` і що все швидко:

```
js
console.log('\n=== Weird inputs ===');
const weirdInputs = {
  '100,000 "<": '<'.repeat(100000),
  '100,000 "&": '&'.repeat(100000),
  '10,000 "<!--": '<!--'.repeat(10000),
  '10,000 unclosed attributes': '<a href="'.repeat(10000),
};

for (const [name, input] of Object.entries(weirdInputs)) {
  console.time(name);
  const res = html2json(input);
  console.timeEnd(name);
  console.log(res.error ? `❌ ${res.error}` : `✅ no error`);
}
```

Перевіряємо саме `res.error`, бо `html2json` має `try/catch`: якщо щось зламається всередині, функція не впаде, а поверне `error`. Тому відсутність `error` = парсер справді впорався.

Крок 4: запуск

У терміналі, з **кореня** проєкту (там, де лежить `html2json.js`):

```
bash
node tests/html2json.test.js
```

Усі результати з'являться прямо в терміналі. Надішли їх сюди.

```
x
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % node -e
"console.log(require('./html2json.js').html2json('<p>x</p>'))"
{
  type: 'root',
  children: [
    {
      type: 'element',
      tag: 'p',
      attributes: [Object: null prototype] {},
      children: [Array]
    }
  ]
}
<3 -not -path '*/.*' -not -path './node_modules*'
.
./index.html
./ai_help
=== 1. ПЕРЕВІРКА БАЗОВОЇ КОРЕКТНОСТІ (Assertions) ===
✅ console.assert пройшли успішно! Базовий парсинг працює.

=== 2. СТЕП-ТЕСТИ ТА MAX_DEPTH (100,000 <div>) ===
Parsing 100,000 deep divs: 52.812ms
✅ JSON.stringify пройдено успішно без помилки стеку!
Довжина результуючого JSON: 6099517 символів
JSON.stringify of deep tree: 87.907ms

=== 3. ФАЗЕР (Property-Based Testing - 10,000 ітерацій) ===
Fuzzer Run Time: 29.343ms

Результат фазера: 10000/10000 тестів пройдено.
✅ Усі Property-Based тести та фазер пройшли без жодної помилки!

=== Invalid inputs ===
✅ null
✅ undefined
✅ 123
✅ {}
✅ []
✅ ""

=== Weird inputs ===
100,000 "<": 5.531ms
✅ no error
100,000 "&": 0.214ms
✅ no error
10,000 "<!--": 0.257ms
✅ no error
10,000 unclosed attributes: 1.166ms
✅ no error
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task %
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task %
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % git add .
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % git commit -m'add
suite with fuzzer, depth stress tests and edge cases'
[develop 603c7ec] add suite with fuzzer, depth stress tests and edge cases
=== 1. ПЕРЕВІРКА БАЗОВОЇ КОРЕКТНОСТІ (Assertions) ===
✅ console.assert пройшли успішно! Базовий парсинг працює.

=== 2. СТЕП-ТЕСТИ ТА MAX_DEPTH (100,000 <div>) ===
Parsing 100,000 deep divs: 87.443ms
✅ JSON.stringify пройдено успішно без помилки стеку!
Довжина результуючого JSON: 6099517 символів
JSON.stringify of deep tree: 122.922ms

=== 3. ФАЗЕР (Property-Based Testing - 10,000 ітерацій) ===
Fuzzer Run Time: 44.329ms

Результат фазера: 10000/10000 тестів пройдено.
✅ Усі Property-Based тести та фазер пройшли без жодної помилки!

=== Invalid inputs ===
✅ null
✅ undefined
✅ 123
✅ {}
✅ []
✅ ""

=== Weird inputs ===
100,000 "<": 6.52ms
✅ no error
100,000 "&": 0.339ms
✅ no error
```

10,000 "<!--": 0.342ms

✓ no error

10,000 unclosed attributes: 2.462ms

✓ no error

Show more

Investigating why test output appears twice after the commit.

Все пройшло ✓, і тепер це справжні результати: експорт працює, `html2json` повертає дерево з `p`, а тестовий набір покриває коректність, глибину, фазер, не-рядки і дивні входи. Входи на 100 000 символів обробляються за мілісекунди. **Крок 10 завершено**, і коміт уже зроблений.

Дві дрібниці з виводу:

- `[Object: null prototype] {}` — це просто так Node показує об'єкт, створений через `Object.create(null)`. У JSON він стає звичайним `{}`. Якщо рев'юер запитав, ти знаєш чому: це захист від `__proto__` і `constructor` в атрибутах.
- Після `git commit` тести запустились ще раз. У тебе налаштований git hook, чи ти просто запустив їх удруге? Якщо це hook, чудово, тоді згадай його в README.

Крок 11: фінальне прибирання

Роби по черзі, з комітом після кожного пункту і запуском `node tests/html2json.test.js` перед кожним комітом. Так, якщо щось зламається, одразу буде видно, на якому кроці.

11.1 Мова. Усі коментарі в `html2json.js` і всі повідомлення в `tests/html2json.test.js` англійською. Пошук кирилиці: `Ctrl+F`, увімкни регулярні вирази, шукай `[а-яіієґА-ЯІІЄҒ]`. Має бути 0 збігів.

11.2 Назви. `BLOCK_ELEMENTS` → `CLOSES_P` (або `P_CLOSING_ELEMENTS`).

11.3 Порядок у файлі. Розділи із заголовками-коментарями:

```
// ===== Constants =====
// ===== Character helpers =====
// ===== Entity decoding =====
// ===== Attribute parsing =====
// ===== Tokenizer =====
// ===== Tree builder =====
// ===== Public API =====
// ===== UI glue (provided by Jito) =====
// ===== Node.js export =====
```

`convertHtml2JsonAndSet` переїжджає з початку файлу в розділ UI, до `showExample1/2`. `MAX_DEPTH` переїжджає до інших констант, не посеред логіки `AUTO_CLOSE`.

11.4 Дрібниці в коді.

- `safeFromCodePoint`: прибрати невикористаний `fallback` і `isNaN`, або залишити з коментарем «defensive check».
- JSDoc до функцій, де його бракує (`buildTree`, `html2json`, `decodeEntities`, `isLetter` та інші): одне речення, що робить, плюс що приймає і що повертає.

11.5 Prettier на обидва файли, потім тести, обидва приклади через сторінку, і `git diff` перед комітом.

Почни з 11.1. Коли закінчиш усі п'ять, надішли фінальний `html2json.js` для повного рев'ю.

```
// ===== Constants =====
= const VOID_ELEMENTS
= new Set(['area', 'base',
'br', 'col', 'embed', 'hr', 'im
g', 'input', 'link', 'meta', 'par
am', 'source', 'track', 'wb...
```

PASTED

check this out

Planning sample HTML files to cover tricky parsing cases. >

Це вже код, який не соромно здавати. Я пройшов файл повністю:

- **Структура:** розділи в логічному порядку, від констант до експорту, і код Jito зібраний в одному місці внизу ✓
- **Мова:** жодної кирилиці ✓
- **Назви:** `P_CLOSING_ELEMENTS` пояснює сам себе ✓
- **JSDoc:** у кожній функції, з типами параметрів і того, що повертається. Особливо добре, що в `html2json` описано обидва особливі випадки: не рядок і `error` ✓
- `safeFromCodePoint` : без зайвого `fallback` і `try/catch` . Перевірки меж покривають усі випадки, коли `String.fromCodePoint` міг би кинути помилку. Навіть `ϧ...999` ; з сотнями цифр дасть `Infinity` , а він більший за `0x10FFFF` , тож теж стане 🍷 ✓
- `MAX_DEPTH` : перенесений до констант, з коментарем, що він робить ✓

Переконайся, що після всіх змін ти запустив тести, обидва приклади і подивився `git diff` . Потім коміт, і **крок 11 завершено**.

Одна знахідка, яку варто виправити (2 рядки)

Пам'ятаєш необов'язкове покращення з кроку 6? `<!-->` і `<!-->` браузери вважають **порожніми коментарями**. У нас `<!-->` шукає `-->` далі по тексту, і якщо не знаходить, **весь решта документа стає коментарем**:

```
html
<!--><p>Hello</p><div>World</div>
```

Браузер: порожній коментар, `p` , `div` . Ми: один коментар з усім вмістом. Рідкісний вхід, але саме такі рев'юери і шукають. Виправлення: у гілці коментарів, перед `readUntil` , перевір `html.startsWith('<!-->', i)` і `html.startsWith('<!-->', i)` . Якщо так, додай порожній коментар і перемісти `i` за `>` .

Відомі відмінності від браузера (для README)

Не виправляємо, але варто чесно перелічити:

- `</p>` без відкритого `p` : браузер створює порожній `<p></p>` , ми ігноруємо.
- `</br>` : браузер вважає це `<br>` , ми ігноруємо.
- `<b><i></b></i>` : браузер використовує adoption agency algorithm, у нас спрощено.
- `<p><span>a<div>` : ми перевіряємо тільки вершину стеку, тож `p` не закривається.
- Таблиці: браузер автоматично додає `tbody` , ми ні.
- Сутності: тільки з `;` і з обмеженого списку.

Крок 12: `html_samples/`

Пам'ятай умову: якщо код впаде або дасть неправильний результат **на твоїх власних зразках**, задачу вважатимуть проваленою. Тож кожен файл треба перевірити через сторінку.

У тебе вже є 01–05. Пропоную додати:

Файл	Що всередині
06-entities.html	© , © , © , &lt; , &unknown; , a & b , сутності в атрибутах
07-implied-end-tags.html	li , p , dt / dd , option , таблиця без закриваючих td / tr
08-malformed.html	незакриті теги, зайві </div> , перехресна вкладеність, незакриті лапки, <!-->
09-svg-and-case.html	SVG з /> , <DIV CLASS="x"> , змішаний регістр
10-whitespace-and-pre.html	<pre> з відступами, textarea з переносами
11-tricky-text.html	5 < 6 , a <- b , <> , </> , x <5 y
12-real-world-page.html	реалістична сторінка: форма, таблиця, style , script з <
	всередині, коментарі

Бонус: додай у тестовий файл розділ, який через `fs.readdirSync` читає всі файли з `html_samples/` , проганяє кожен через `html2json` і перевіряє, що немає `error` . Тоді жоден зразок не зламається непомітно.

Почни з виправлення `<!-->` , потім зразки. Надішли файли, коли будуть готові.

```
// ===== Constants =====
const VOID_ELEMENTS = new Set([
  'area', 'base', 'br', 'col', 'embed', 'hr', 'img', 'input', 'link', 'meta', 'param', 'source', 'track', 'wb...
```

check again

Checking edge cases in the comment-parsing fix.

Виправлення `<!-->` працює правильно. Я перевірів межові випадки:

- `<!-->` → порожній коментар, `i` переходить на 5 символів ✓
- `<!-->` → порожній коментар, на 6 символів ✓
- `<!-->` не збігається з жодним із двох шаблонів, іде в `readUntil` і теж дає порожній коментар ✓
- `<!-- text -->` працює як і раніше ✓

Але є дві речі.

1. Змінився приклад 2 від Jito

Порівняй з оригіналом (він є в найпершому повідомленні цього чату):

Оригінал:	Зараз:
<div> <p>Hello world!</p> <button>Click me!</button> <textarea>...	<div> <p>Hello world!</p> <button>Click me!</button> <textarea>...

Відступи **всередині** рядка-шаблону змінились: у `<p>` було 0 пробілів, а в `button` і `textarea` по 2. Prettier сам такого не робить (він не чіпає вміст шаблонних рядків), тож це змінилось під час редагування. Тепер вхідний приклад не такий, як дали Jito, і текстові вузли з пробілами в результаті будуть іншими. Рев'юер може це помітити, тож **поверни оригінальний текст** прикладу 2. `git diff` якраз покаже, що саме змінилось.

2. Коментар виглядає як історія змін

```
js
```

```
// ===== FIX: abruptly closed empty comments <!--> and <!--> =====
...
// ===== END FIX =====
```

Як і з **[UPDATED]** раніше: «FIX» описує, що змінилось, а не що робить код. Крім того, **=====** у тебе вже позначають розділи файлу, тож цей блок виглядає як ще один розділ. Залиш коротке пояснення:

```
js

// Browsers treat <!--> and <!--> as complete empty comments.
// Without this check, the rest of the document would become one comment.
```

Решта пояснення (чому **<!-->** не збігається з **<!-->**) — це гарний матеріал для README або для відповіді на інтерв'ю, але в коді вона зайва.

Після виправлення

1. Додай **<!-->** у тестовий файл, наприклад: `html2json('<!--><p>Hello</p>')` → у **root** два дочірні вузли: порожній коментар і **p**.
2. Запусти тести, обидва приклади через сторінку, **git diff**, коміт.

Потім крок 12: **html_samples/** (таблиця файлів у моєму попередньому повідомленні).

```
// ===== Constants =====
= const VOID_ELEMENTS
= new Set(['area', 'base',
'br', 'col', 'embed', 'hr', 'img',
'input', 'link', 'meta', 'param',
'source', 'track', 'wb...

PASTED
```

again

Reviewed edits and confirmed formatting consistency across the file.

Тепер усе правильно:

- **Приклад 2** знову збігається з оригіналом Jito: **<p>** без відступу, **button** і **textarea** з двома пробілами ✓
- **Коментар до <!-->** короткий і пояснює, що робить код, а не історію змін ✓
- **Відступи** однакові по всьому файлу ✓

Я не бачу в цьому файлі більше нічого, що варто змінювати. **Крок 11 завершено.**

Перед комітом:

1. Додай у тестовий файл перевірку **<!-->**: `html2json('<!--><p>Hello</p>')` має дати в **root** два вузли, порожній коментар і **p**.
2. **node tests/html2json.test.js**, обидва приклади через сторінку, **git diff**, коміт.

Крок 12: **html_samples/**

Нагадаю план:

Файл	Що всередині
01 – 05	вже є
06-entities.html	© , © , © , &lt; , &unknown; , a & b , сутності в атрибутах
07-implicit-end-tags.html	li , p , dt / dd , option , таблиця без закриваючих td / tr
08-malformed.html	незакриті теги, зайві </div> , перехресна вкладеність, незакриті лапки, <!-->
09-svg-and-case.html	SVG з /> , <DIV CLASS="x"> , змішаний регістр
10-whitespace-and-pre.html	<pre> з відступами, textarea з переносами
11-tricky-text.html	5 < 6 , a <- b , <> , </> , x <5 y
12-real-world-page.html	реалістична сторінка: форма, таблиця, style , script з <

Кожен файл після створення проганяй через сторінку і переглядай результат очима: зразок, на якому парсер дає неправильний результат, гірший, ніж відсутність зразка.

Бонус: розділ у тестовому файлі, який читає всі файли з `html_samples/` через `fs.readdirSync` і перевіряє, що жоден не повертає `error`.

Надішли файли, коли будуть готові, я перевірю і зміст, і очікувані результати.



done, check that i also add tests check also

Reviewing a test file for ordering and output issues.

`html2json.js` не змінився з минулого разу, і це добре: він готовий. Тепер тести. Розділ з `<! ->` правильний, перевірка `html_samples/` — гарна ідея. Але є одна помилка і кілька речей, через які тести можуть «пройти», коли щось зламано.

1. `console.clear()` стирає результати зразків

Перевірка `html_samples/` стоїть на початку файлу, а `console.clear()` після неї. Тож усі рядки `✓ Sample "..."` `passed` друкуються і одразу стираються, і ти їх ніколи не побачиш. Найпростіше прибрати `console.clear()` зовсім: тестовий скрипт не повинен чистити термінал користувача. І перенеси розділ зі зразками в кінець, щоб порядок був логічний: спершу маленькі перевірки, потім реальні файли.

2. Скрипт завжди завершується «успішно»

Зараз у файлі три різні способи перевірки:

- `assert(...)` кидає помилку і зупиняє весь скрипт на першому невдалому зразку
- `console.assert(...)` лише друкує повідомлення і йде далі
- `console.log(ok ? '✓' : '✗')` лише друкує

В останніх двох випадках скрипт завершується з кодом `0`, тобто для терміналу, git hook чи CI все пройшло, навіть якщо на екрані купа `✗`. Якщо в тебе справді налаштований git hook, він пропустить коміт зі зламаним парсером.

Рішення: один помічник на весь файл, який рахує невдачі, і код виходу в кінці:

```
js
```

```
let failures = 0;

function check(name, condition) {
  // print ✅ or ❌ with the name; if it failed, increase failures
}

// ... all tests use check(...) ...

console.log(`\n${failures} === 0 ? '✅ All tests passed' : '❌ ${failures} test(s) failed'`);
process.exitCode = failures > 0 ? 1 : 0;
```

Тоді `assert`, `console.assert` і ручні `ok ? '✅' : '❌'` замінюються одним стилем, і подвійна перевірка в розділі 1 (яку ми обговорювали раніше) зникає сама собою.

3. Тиха перевірка, якщо папки немає

Якщо `fs.existsSync(samplesDir)` просто пропускає розділ, якщо папки немає. А `html_samples/` — обов'язкова частина задачі. Якщо папка зникне або шлях буде неправильний, ти не дізнаєшся. Зроби з цього перевірку: папка існує і в ній є хоча б один файл.

Дрібниці

- `// Перевірка всіх файлів з html_samples/` → англійською.
- `require('assert')` окремо від інших `require` нагорі. Якщо перейдеш на `check`, він стане непотрібним.
- Фазер із `Math.random` не відтворюється: якщо він колись щось знайде, при наступному запуску вхід буде іншим. Необов'язкове покращення: простий генератор з насінням (seed), наприклад `mulberry32` (5 рядків), і друкувати seed на початку. Тоді будь-яку знахідку можна повторити.

Ще одне

Зразки самі (06–12) ти ще не надсилав. Перевірка «немає `error`» підтверджує лише, що парсер не впав, але не що результат правильний. Тому кожен зразок треба переглянути очима через сторінку.

Виправ тести, запусти `node tests/html2json.test.js` (і перевір `echo $?` після запуску: має бути 0), і надішли результат разом із файлами зразків.

```
// html2json.test.js const fs = require('fs'); const path = require('path'); const { html2json } = require('./html2json'); let failures = 0; // ** Prints ✅ or ❌ for a...
```

PASTED

```
// ===== Constants ===== const VOID_ELEMENTS = new Set(['area', 'base', 'br', 'col', 'embed', 'hr', 'img', 'input', 'link', 'meta', 'param', 'source', 'track', 'wb...
```

PASTED

Reviewing the test file against the checklist of requirements.

Тепер тести виглядають професійно:

- один помічник `check` на весь файл, і `process.exitCode = 1` при будь-якій невдачі ✅
- `isValidRoot` замість повторення тих самих умов у кожному розділі ✅
- без `console.clear()`, зразки в кінці, і відсутня папка тепер помилка, а не тихий пропуск ✅
- фазер з насінням і підказкою `SEED=...` для відтворення: це рівень, який рідко бачиш у задачі стажера ✅

`html2json.js` не змінився, все в порядку.

Головне, чого бракує: тести можливостей парсера

Зараз автоматично перевіряються лише два прості випадки коректності (`<p>x</p>` і вкладений текст). А всі тести, які ми робили вручну на кроках 2–9 (атрибути, void-теги, `script`, сутності, `li`, таблиці...), автоматизовані не є. Якщо завтра зміна зламає, наприклад, декодування в `textarea`, жоден тест цього не помітить.

Найпростіше — **табличні тести**: масив випадків і один цикл.

```
js

console.log('\n=== Feature tests ===');

const featureCases = [
  {
    name: 'attribute value may contain ">"',
    input: '<a title="a > b">x</a>',
    expect: (r) => r.children[0].attributes.title === 'a > b',
  },
  {
    name: 'duplicate attribute: first wins',
    input: '<div class="a" class="b">',
    expect: (r) => r.children[0].attributes.class === 'a',
  },
  // ... more cases
];

for (const { name, input, expect } of featureCases) {
  const res = html2json(input);
  let ok;
  try {
    ok = expect(res);
  } catch {
    ok = false; // e.g. children[0] is undefined
  }
  check(name, ok, res);
}
```

`try/catch` навколо `expect` потрібен, бо якщо парсер поверне не ту структуру, `r.children[0].attributes` кине помилку, і це має стати **✗**, а не падінням скрипта.

Перенеси сюди тести з наших таблиць, по 1-3 на кожну можливість: атрибути, `__proto__` / `constructor`, `void`-елементи, `/>` і SVG, коментарі й `doctype`, `script` з `<` усередині, `</scriptx>`, сутності (зокрема `<` → `<`; і `<script>&``</script>` без змін), `li/p` / таблиця, `<div><p>Hi</div>` after, `5 < 6`. Виходить приблизно 20 випадків, і це головний доказ для рев'юера, що кожне рішення з README справді працює.

🟡 **Дрібниці**

- **Невалідний SEED**: `SEED=abc` дасть `NaN`, і генератор почне повертати `0` на кожному кроці. Фазер 10 000 разів перевірить один і той самий вхід `<` і тихо «пройде». Перевір `Number.isInteger(...)` і, якщо ні, бери випадковий seed (або виводь помилку).
- **Як запускати**: додай на початку файлу коментар `// Run: node tests/html2json.test.js (optional: SEED=<n> to reproduce the fuzzer)`. Рев'юер відкріє файл і одразу знатиме, що робити.

Що чекаю далі

1. Табличні тести можливостей.
2. Результат запуску і `echo $?` (має бути `0`).
3. Файли зразків `06-12` для перевірки.

```
// ===== Constants =====
const VOID_ELEMENTS = new Set(['area', 'base', 'br', 'col', 'embed', 'hr', 'img', 'input', 'link', 'meta', 'param', 'source', 'track', 'wb...

// html2json.test.js // Run:
node tests/html2json.test.js (optional: SEED=<n> to reproduce the fuzzer)
const fs = require('fs');
const path = require('path')...
```

PASTED

```
// html2json.test.js // Run:
node tests/html2json.test.js (optional: SEED=<n> to reproduce the fuzzer)
const fs = require('fs');
const path = require('path')...
```

PASTED

Checking edge cases like prototypes, tables, and nested tags. >

Я пройшов усі 22 випадки через логіку парсера: кожне очікування правильне, і жоден тест не перевіряє щось, чого парсер насправді не робить. Особливо вдалі:

- `__proto__` / `constructor` перевіряє не лише значення, а й `Object.getPrototypeOf(attrs) === null`. Тобто тест фіксує саме причину рішення, а не тільки результат.
- `textarea` з `<lt;b><i>` перевіряє одразу дві речі: сутності декодуються, а `<i>` не стає елементом.
- **Таблиця** перевіряє точну кількість рядків і комірок, а не лише «не впало».

Перевірка `SEED` з коментарем, чому `NaN` небезпечний, і рядок «Run: ...» на початку — теж саме те, що треба.

Два випадки, які варто додати

Обидва стосуються рішень, які ти будеш захищати в README, тож вони мають бути покриті тестами:

1. **Сутності в атрибутах:** `<a href=?a=1&b=2">` → `href === 'a=1&b=2'`. Зараз декодування перевіряється тільки в тексті.
2. **Пробіли зберігаються:** `<p>a</p>\n<p>b</p>` → у `root` три вузли, і середній — текст `'\n'`. Це одне з перших рішень, яке ми прийняли (парсер без втрат), і зараз воно ніде не зафіксоване. Якщо хтось колись додасть «очищення пробілів», цей тест одразу впаде.

Що лишилось до здачі

#	Крок	Статус
1–11	Парсер, захист від падінь, прибирання, тести	✓
12	<code>html_samples/</code> (06–12)	⌚
13	README	■
14	Фінальна перевірка: <code>ai_help/</code> , посилання на чат в incognito, архів	■

Додай два тести, запусти `node tests/html2json.test.js` і `echo $?`, і надішли:

1. вивід тестів (достатньо підсумкового рядка і розділу 8 зі зразками);
2. файли зразків `06` – `12`.

Після цього переходимо до README.

```
// html2json.test.js // Run:
node tests/html2json.test.js
(optional: SEED=<n> to reproduce the fuzzer)
const fs = require('fs');
const path = require('path')...
```

PASTED

```
// ===== Constants =====
const VOID_ELEMENTS = new Set(['area', 'base', 'br', 'col', 'embed', 'hr', 'img', 'input', 'link', 'meta', 'param', 'source', 'track', 'wb...
```

PASTED

Checking parser behavior on whitespace between elements.

Обидва нові тести правильні. Я перевірів їх через логіку парсера:

- `<a href=?a=1&b=2">` → у `readAttributes` значення проходить через `decodeEntities`, тож `href` буде `?a=1&b=2` ✓
- `<p>a</p>\n<p>b</p>` → `p`, текст `"\n"`, `p`: рівно три вузли, середній — текст ✓

Тепер кожне рішення, яке піде в README, має свій автоматичний тест. `html2json.js` не змінився. **Тести завершено.**

Залишилось три речі, і я прошу їх уже кілька повідомлень, бо без них не можу підтвердити, що все справді працює:

1. **Запусти тести** і надішли останні рядки виводу (розділ 8 зі зразками і підсумок):

```
bash

node tests/html2json.test.js
echo $?
```

echo \$? має вивести 0.

2. **Файли зразків 06 - 12** (або ті, що вже створив). Тест у розділі 8 перевіряє лише, що парсер не впав на них, а не що результат правильний. Тому я хочу переглянути їх сам.
3. **Коміт** після цього.

Щойно надішлеш, переходимо до README, останнього великого кроку перед здачею.

```
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % node tests/html2json.test.js
```

=== 1. Basic correctness ===

- ✓ <p>x</p> parses to a single <p>
- ✓ nested text is parsed correctly

=== 2. Feature tests ===

- ✓ attribute value may contain ">"
- ✓ duplicate attribute: first wins
- ✓ attribute names are lowercased; unquoted and boolean values
- ✓ __proto__ and constructor are stored as plain attributes
- ✓ void element
 does not swallow following siblings
- ✓ /> closes SVG elements
- ✓ tag names are lowercased
- ✓ comment keeps its content
- ✓ doctype is a separate node
- ✓ script content with "<" stays raw text
- ✓ </script> does not close <script>
- ✓ script does not decode entities
- ✓ textarea decodes entities and does not parse tags
- ✓ named, decimal and hex entities are decoded
- ✓ &lt; decodes once to <
- ✓ entities in attribute values are decoded
- ✓ unknown entity stays unchanged
- ✓ closes the previous
- ✓ <div> closes an open <p>
- ✓ table rows and cells close implicitly
- ✓ </div> closes an unclosed <p> inside it
- ✓ whitespace between elements is kept as a text node
- ✓ "<" not followed by a letter is text
- ✓ stray end tag is ignored

=== 3. Stress tests and MAX_DEPTH (100,000 <div>) ===

- Parsing 100,000 deep divs: 66.464ms
- ✓ deep tree has a valid root
- JSON.stringify of deep tree: 91.063ms
- ✓ JSON.stringify of deep tree does not overflow the stack

=== 4. Fuzzer (property-based testing, 10,000 iterations) ===

- Seed: 4288235684 (rerun with SEED=4288235684 to reproduce)
- Fuzzer Run Time: 31.014ms
- ✓ fuzzer: 10000/10000 inputs passed

=== 5. Invalid inputs ===

- ✓ invalid input null gives an empty root
- ✓ invalid input undefined gives an empty root
- ✓ invalid input 123 gives an empty root
- ✓ invalid input {} gives an empty root
- ✓ invalid input [] gives an empty root
- ✓ invalid input "" gives an empty root

=== 6. Abruptly closed empty comments ===

- ✓ <!--><p>Hello</p>
- ✓ <!--><p>Hello</p>

=== 7. Weird inputs ===

- 100,000 "<": 16.184ms
- ✓ 100,000 "<" gives a valid root
- 100,000 "&": 0.152ms
- ✓ 100,000 "&" gives a valid root
- 10,000 "<!--": 0.217ms
- ✓ 10,000 "<!--" gives a valid root
- 10,000 unclosed attributes: 1.236ms
- ✓ 10,000 unclosed attributes gives a valid root

=== 8. Samples from html_samples/ ===

- ✓ html_samples/ folder exists
- ✓ html_samples/ contains sample files (12 found)
- ✓ sample 01-basic-nesting.html
- ✓ sample 02-attributes.html
- ✓ sample 03-void-and-self-closing.html

```

✓ sample 04-comments-doctype.html
✓ sample 05-raw-text.html
✓ sample 06-entities.html
✓ sample 07-implicit-end-tags.html
✓ sample 08-malformed.html
✓ sample 09-svg-and-case.html
✓ sample 10-whitespace-and-pre.html
✓ sample 11-tricky-text.html
✓ sample 12-real-world-page.html
✓ All tests passed
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % echo $?
0
Show more

```

Verifying test results and considering whether to request sample files.

Усі тести пройшли, і `echo $?` дає `0`. Це означає, що 30 перевірок можливостей, стрес-тести, фазер, не-рядки, дивні входи і всі 12 зразків проходять, а скрипт правильно повідомляє про успіх.

Одне нагадування щодо зразків: тест перевіряє лише, що парсер не впав, тож перед здачею відкрий кожен із 12 файлів через сторінку і переглянь JSON очима. Якщо хочеш, щоб я теж подивився якийсь складний (наприклад `08-malformed` чи `12-real-world-page`), надішли його.

Зроби коміт, і **крок 12 завершено**.

Крок 13: README

Завдання каже, що README необов'язковий, але для тебе він найважливіший після коду: ревьюер прочитає його першим, а на інтерв'ю питатимуть саме про рішення, описані там. README пиши англійською. Ось структура:

1. Опис. Два-три речення: що це, що парсер написаний без DOM-парсера, і що він ніколи не кидає помилку.

2. Як запустити.

- відкрити `index.html` у браузері
- тести: `node tests/html2json.test.js` (і `SEED=<n>` для відтворення фазера)

3. Структура репозиторію. Дерево файлів і одне речення про кожен.

4. Формат результату. Маленький приклад вхід → вихід і таблиця типів вузлів: `root`, `element`, `text`, `comment`, `doctype`, з полями кожного.

5. Рішення і чому. Найважливіший розділ. Для кожного рішення: що обрав і чому. Ти вже маєш усі пункти:

- масив `children` з вузлами за `type` замість тегів як ключів (дублювати, порядок, змішаний вміст)
- корінь `root`, бо вхід може бути фрагментом
- пробіли зберігаються (парсер без втрат)
- завжди `attributes: {}` і `children: []` (передбачувана схема)
- назви тегів і атрибутів у нижньому регістрі, значення без змін
- перший дублікат атрибута виграє: `Object.create(null)` проти `__proto__`
- два етапи: токенизатор → будівник дерева, без рекурсії
- `/>` поважається для всіх тегів (через SVG)
- RAW TEXT і RCDATA: що декодується, а що ні
- сутності: тільки з `;`, обмежений список, недійсні коди → `0`, один прохід
- неявне закриття (лише вершина стеку) і пошук закриваючого тегу по стеку
- `MAX_DEPTH = 512`, як у Chrome, бо `JSON.stringify` рекурсивний
- не рядок → порожній `root`; `try/catch` з полем `error` як остання страховка
- один файл, бо `index.html` має залишитись без змін

6. Тестування. Що покривають тести і чому фазер з насінням.

7. Відомі обмеження. Список відмінностей від браузера, який ми склали: `</p>` без відкритого `p`, `</br>`, adoption agency (`<b><i></b></i>`), перевірка тільки вершини стеку, відсутність автоматичного `tbody`, `\r\n` не нормалізується, обмежений список сутностей, SVG-атрибути в нижньому регістрі (`viewBox` → `viewbox`).

8. Використання AI. Обов'язково згадай, що ця розмова в `ai_help/`, і чесно опиши, як ти працював з AI: поетапний план, пояснення, рев'ю кожного кроку, пошук помилок. Рев'юери читатимуть увесь чат, тож опис має відповідати тому, що там є.

Напиши чернетку і надішли, я перевірю і точність, і формулювання.

Попередньо про крок 14

Щоб не забути: посилання на чат «заморожує» розмову в момент створення. Тож створий його **в самому кінці**, після останнього повідомлення, і перевір в incognito-вікні, що воно відкривається. Завдання згадує `chatgpt_chat.txt` для ChatGPT, а для інших AI просить `.pdf` або `.png`. Тому поклади в `ai_help/` і текстовий файл з посиланням, і PDF-експорт чату як запасний варіант.

```
#html2json`html2json(h
tml)` turns an HTML strin
g into a tree of plain objec
ts that can be passed to
JSON.stringify`. The pars
er is written from scratc...
```

PASTED

this is my readme.md
and chatgpt_chat.txt

Claude Conversation Link:

<https://claude.ai/share/a8bcdd06-d96c-4430-9a85-d1eabd779390>

Note: I used Claude as my AI assistant for this task. The full conversation is accessible via the link above and as 'claude_chat.pdf' in this folder.

Reviewing the AI-use disclosure for completeness and honesty. >

README дуже сильний. Він пояснює не лише **що** зроблено, а й **чому**, кожне рішення має автоматичний тест, а обмеження чесно перелічені. Я звірив його з кодом рядок за рядком: майже все точно. Нижче кілька уточнень, щоб рев'юер не знайшов розбіжностей.

Неточності (дрібні)

1. **«Everything needed to rebuild the document»** — це перебільшення. Крім обмежень зі списку, губиться й інше: оригінальний регістр тегів, стиль лапок в атрибутах, закодовані сутності (`&` стає `&`). Точніше сказати, що **весь текстовий вміст зберігається**, наприклад: *"No text content is dropped; filtering whitespace is left to the consumer."*
2. `<p><span><div>` : `div` опиняється всередині `span`, а не просто «всередині `p` ». Краще: *"the `<div>` ends up inside the `<span>`, which is still inside the `<p>`."*
3. **Неявне закриття:** у списку бракує `dt / dd` і `option`, а вони є в коді. Додай їх, щоб README відповідав `AUTO_CLOSE_RULES`.
4. **SVG у обмеженнях:** у нижній регістр переводяться не лише атрибути, а й **назви тегів**: `<linearGradient>` → `lineargradient`. Додай це в той самий пункт.

Важливе: розділ «Use of AI»

Опис має точно відповідати тому, що рев'юери побачать у чаті, бо вони читатимуть його повністю.

- **Інші інструменти.** У чаті є моменти, коли файл повертався сильно переписаним (великий рефакторинг, тестовий файл, частина форматування). Якщо для цього ти використовував ще щось, наприклад Copilot у редакторі чи інший чат, **завдання вимагає додати й ці матеріали** в `ai_help/` (скріншоти чи PDF). Якщо ні, то все гаразд. Але якщо так, краще написати про це прямо, ніж залишити розбіжність, яку рев'юер помітить сам.
- **«Can explain every part of the code».** Це сильна заява, і на інтерв'ю її перевірять. Переконайся, що зможеш пояснити хоча б на загальному рівні все, включно з тестовим файлом, наприклад що робить `mulberry32` і навіщо в ньому `Math.imul`.

Посилання на чат

Посилання, яке ти створив, **не міститиме** цього повідомлення і все, що буде далі: воно фіксує розмову в момент створення. Тому:

- 1. Закінчи всю роботу в цьому чаті.
- 2. Створи посилання **наостанок** (або онови існуюче) і заміни його в `chatgpt_chat.txt`.
- 3. Відкрий у вікні incognito і перевір, що видно **останнє** повідомлення.
- 4. PDF-експорт роби теж в кінці, і перевір, що він не обрізаний: розмова довга.

Крок 14: фінальна перевірка перед здачею

#	Перевірка
1	<code>index.html</code> не змінений: <code>git diff <перший-коміт> -- index.html</code> нічого не виводить
2	Є всі частини: <code>html2json.js</code> , <code>index.html</code> , <code>html_samples/</code> , <code>ai_help/</code> (txt + pdf), <code>README.md</code> , <code>tests/</code>
3	Свіжий клон: скопуй репозиторій в іншу папку, відкрий <code>index.html</code> , запусти тести. Так ти зловиш файли, які забув закомітити
4	Обидва приклади через сторінку, F12 → Console без помилок
5	Усі посилання (чат, шаблон Jіto) відкриваються в incognito
6	Перечитай умову: як саме здавати (архів, посилання на репозиторій, форма) і дедлайн

Підготовка до інтерв'ю

Будь готовий пояснити без підглядання:

- як стек будує дерево і чому `while`, а не `if`, в неявному закритті
- навіщо `Object.create(null)`
- різницю між RAW TEXT і RCDATA
- чому `MAX_DEPTH`, якщо парсер без рекурсії
- чому токенізатор і будівник дерева розділені
- складність: токенізатор проходить рядок один раз (O(n)), а пошук закриваючого тегу по стеку додає O(глибина) на кожен закриваючий тег

Внеси правки в README, пройди чекліст, і напиши, якщо щось на якомусь пункті не так. Ти пройшов величезний шлях від `tokenize` з одним `while` до парсера з фазером, і код тепер справді можна захищати.

README дуже сильний. Він пояснює не лише що зроблено, а й чому, кожне рішення має автоматичний тест, а обмеження чесно перелічені. Я звірив його з кодом рядок за рядком: майже все точно. Нижче кілька уточнень, щоб рев'юер не знайшов розбіжностей.
Неточності (дрібні)

- 1. «Everything needed to rebuild the document» — це перебільшення. Крім обмежень зі списку, губиться й інше: оригінальний реєстр тегів, стиль лапок в атрибутах, закодовані сутності (`&` стає `&`). Точніше сказати, що весь текстовий вміст зберігається, наприклад: "No text content is dropped; filtering whitespace is left to the consumer."
- 2. `<p><span><div>` : `div` опиняється всередині `span`, а не просто «всередині `p` ». Краще: "the `<div>` ends up inside the `<span>`, which is still inside the `<p>` ".
- 3. Неявне закриття: у списку бракує `dt / dd` і `option`, а вони є в коді. Додай їх, щоб README відповідав `AUTO_CLOSE_RULES`.
- 4. SVG у обмеженнях: у нижній реєстр переводяться не лише атрибути, а й назви тегів: `<linearGradient>` → `lineargradient`. Додай це в той самий пункт.

Важливе: розділ «Use of AI»

Опис має точно відповідати тому, що рев'юери побачать у чаті, бо вони читатимуть його повністю.

- Інші інструменти. У чаті є моменти, коли файл повертався сильно переписаним (великий рефакторинг, тестовий файл, частина форматування). Якщо для цього ти використовував ще щось, наприклад Copilot у редакторі чи інший чат, завдання вимагає додати й ці матеріали в

- `ai_help/` (скріншоти чи PDF). Якщо ні, то все гаразд. Але якщо так, краще написати про це прямо, ніж залишити розбіжність, яку ревіюер помітить сам.
- «Can explain every part of the code». Це сильна заява, і на інтерв'ю її перевірять. Переконайся, що зможеш пояснити хоча б на загальному рівні все, включно з тестовим файлом, наприклад що робить `mulberry32` і навіщо в ньому `Math.imul`.

Посилання на чат

Посилання, яке ти створив, не міститиме цього повідомлення і все, що буде далі: воно фіксує розмову в момент створення. Тому:

1. Закінчи всю роботу в цьому чаті.
2. Створи посилання наостанок (або онови існуюче) і заміни його в `chatgpt_chat.txt`.
3. Відкрий у вікні incognito і перевір, що видно останнє повідомлення.
4. PDF-експорт роби теж в кінці, і перевір, що він не обрізаний: розмова довга.

do that instead of me to save my time

Show more

Created a file, shared files >

Ось готовий README з усіма чотирма правками. Просто заміни ним свій `README.md`. Решта тексту залишилась слово в слово як у тебе.

Що змінено:

1. **Whitespace:** замість «everything needed to rebuild the document» тепер «no text content is dropped, and filtering whitespace is left to the consumer».
2. `<p><span><div>`: уточнено, що `div` опиняється всередині `span`, який усе ще всередині `p`.
3. **Неявне закриття:** додано `dt` / `dd` і `option`, тепер список відповідає `AUTO_CLOSE_RULES` у кодї.
4. **SVG:** пункт тепер згадує і назви тегів (`linearGradient`), і атрибути (`viewBox`).

Дві речі, які можеш зробити тільки ти:

- **Розділ «Use of AI»** я не змінював, бо лише ти знаєш, чи використовував інші інструменти. Якщо так (наприклад, Copilot або інший чат), додай у кінець розділу таке речення і поклади скріншоти чи PDF у `ai_help/`:
`I also used [tool name] for [what exactly], see ai_help/[file name].`
- **Посилання на чат:** створи його заново **після** нашого останнього повідомлення, заміни в `chatgpt_chat.txt`, перевір в incognito, що видно кінець розмови, і тоді зроби PDF-експорт.

Після цього пройди чекліст з кроку 14, і можна здавати. Успіхів!



Readme

Document · MD

Download

show me again checklist step 14

Крок 14: фінальна перевірка перед здачею

#	Перевірка	Як перевірити
1	<code>index.html</code> не змінений	<code>git diff <хеш-першого-коміту> -- index.html</code> нічого не виводить
2	Є всі частини	<code>html2json.js</code> , <code>index.html</code> , <code>html_samples/</code> , <code>tests/</code> , <code>README.md</code> , <code>ai_help/</code> (<code>chatgpt_chat.txt</code> + <code>claude_chat.pdf</code>)
3	Свіжий клон	склонуй репозиторій в іншу папку, відкрий там <code>index.html</code> і запусти <code>node tests/html2json.test.js</code> . Так ти зловиш файли, які забув закомітити
4	Тести проходять	у свіжому клоні:  <code>All tests passed</code> і <code>echo \$?</code> дає <code>0</code>
5	Сторінка працює	обидва приклади → «Convert to JSON»; F12 → Console без жодних помилок
6	Зразки перевірені очима	кожен із 12 файлів з <code>html_samples/</code> через сторінку, результат має сенс
7	Розділ «Use of AI» точний	якщо використовував інші AI-інструменти, вони згадані в README, а матеріали лежать в <code>ai_help/</code>
8	Посилання на чат свіже	створене після останнього повідомлення в цьому чаті і вставлене в <code>chatgpt_chat.txt</code>
9	Посилання відкриваються	посилання на чат і на шаблон Jito відкриваються у вікні incognito, і в чаті видно останнє повідомлення
10	PDF повний	<code>claude_chat.pdf</code> зроблений в кінці і не обрізаний: розмова довга
11	Умови задачі	перечитай умову: як саме здавати (архів, посилання на репозиторій, форма) і дедлайн. Якщо це архів, переконайся, що в ньому всі папки зі списку в пункті 2
12	Останній коміт	<code>git status</code> чистий, усе закомічено й запущено

Порядок важливий: пункти 8–10 роби **в самому кінці**, коли більше нічого в цьому чаті писати не будеш. Інакше посилання і PDF не міститимуть останні повідомлення.

```
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % node
tests/html2json.test.js
=== 1. Basic correctness ===
 <p>x</p> parses to a single <p>
 nested text is parsed correctly

=== 2. Feature tests ===
 attribute value may contain ">"
 duplicate attribute: first wins
 attribute names are lowercased; unquoted and boolean values
 __proto__ and constructor are stored as plain attributes
 void element <br> does not swallow following siblings
 /> closes SVG elements
 tag names are lowercased
 comment keeps its content
 doctype is a separate node
 script content with "<" stays raw text
 </script> does not close <script>
 script does not decode entities
 textarea decodes entities and does not parse tags
 named, decimal and hex entities are decoded
 &amp;lt; decodes once to &lt;
 entities in attribute values are decoded
 unknown entity stays unchanged
 <li> closes the previous <li>
 <div> closes an open <p>
 table rows and cells close implicitly
 </div> closes an unclosed <p> inside it
 whitespace between elements is kept as a text node
```

```

✅ "<" not followed by a letter is text
✅ stray end tag is ignored

=== 3. Stress tests and MAX_DEPTH (100,000 <div>) ===
Parsing 100,000 deep divs: 73.977ms
✅ deep tree has a valid root
JSON.stringify of deep tree: 85.322ms
✅ JSON.stringify of deep tree does not overflow the stack

=== 4. Fuzzer (property-based testing, 10,000 iterations) ===
Seed: 3078465656 (rerun with SEED=3078465656 to reproduce)
Fuzzer Run Time: 32.011ms
✅ fuzzer: 10000/10000 inputs passed

=== 5. Invalid inputs ===
✅ invalid input null gives an empty root
✅ invalid input undefined gives an empty root
✅ invalid input 123 gives an empty root
✅ invalid input {} gives an empty root
✅ invalid input [] gives an empty root
✅ invalid input "" gives an empty root

=== 6. Abruptly closed empty comments ===
✅ <!--><p>Hello</p>
✅ <!----><p>Hello</p>

=== 7. Weird inputs ===
100,000 "<": 16.206ms
✅ 100,000 "<" gives a valid root
100,000 "&": 0.162ms
✅ 100,000 "&" gives a valid root
10,000 "<!--": 0.193ms
✅ 10,000 "<!--" gives a valid root
10,000 unclosed attributes: 1.202ms
✅ 10,000 unclosed attributes gives a valid root

=== 8. Samples from html_samples/ ===
✅ html_samples/ folder exists
✅ html_samples/ contains sample files (12 found)
✅ sample 01-basic-nesting.html
✅ sample 02-attributes.html
✅ sample 03-void-and-self-closing.html
✅ sample 04-comments-doctype.html
✅ sample 05-raw-text.html
✅ sample 06-entities.html
✅ sample 07-implied-end-tags.html
✅ sample 08-malformed.html
✅ sample 09-svg-and-case.html
✅ sample 10-whitespace-and-pre.html
✅ sample 11-tricky-text.html
✅ sample 12-real-world-page.html

✅ All tests passed
romanlavrencuk@Romans-MacBook-Air jito-intern-test-task % echo $?
0

Show more

```

Verifying a checklist item against the project setup.

Усе пройшло✅, і `echo $?` дає `0`. Пункт 4 чекліста виконано.

Одне уточнення: якщо ти запускав це в **основній** папці проєкту, а не у свіжому клоні, то пункт 3 ще не перевірений. Свіжий клон ловить файли, які існують у тебе локально, але не потрапили в git (наприклад, якщо `html_samples/` чи `tests/` забули додати). Це займе хвилину:

```

bash

cd /tmp
git clone <посилання-на-твій-репозитрій> check-clone
cd check-clone
node tests/html2json.test.js
open index.html

```

Якщо там теж усе✅ і сторінка працює, пункти 3–4 закриті.

Що лишилось

#	Перевірка
1	<code>index.html</code> не змінений (<code>git diff <перший-коміт> -- index.html</code>)
3	Свіжий клон (якщо ще не робив)
5	Обидва приклади через сторінку, Console без помилок
6	12 зразків переглянуті очима
7	«Use of AI» в README точний
11	Перечитати умови задачі й дедлайн
12	<code>git status</code> чистий, усе запущено
8-10	В самому кінці: нове посилання на чат → <code>chatgpt_chat.txt</code> → перевірка в incognito → PDF

Пункти 8-10 роби, тільки коли більше нічого не писатимеш у цей чат.

looks like everything okay, almost forgot about pdf and check cat link

Explaining how to create a shareable chat link and export a PDF.

Добре, що згадав. Ось останні кроки, рівно в такому порядку.

1. Посилання на чат

- 1. Це має бути **останнє**, що ти робиш у цьому чаті. Після моєї відповіді нічого сюди не пиши, інакше нові повідомлення не потраплять у посилання.
- 2. Натисни кнопку **Share** у верхній частині чату і створи (або онови) посилання.
- 3. Скопіюй його в `ai_help/chatgpt_chat.txt` замість старого.

2. Перевірка в incognito

- 1. Відкрий посилання у вікні incognito (`Cmd + Shift + N` у Chrome).
- 2. Прокрути до самого кінця і переконайся, що видно **це повідомлення**. Якщо видно лише старіші, посилання створене раніше, повтори крок 1.

3. PDF

- 1. Відкрий те саме посилання (можна прямо в incognito).
- 2. **Прокрути до самого низу**, щоб уся розмова завантажилась. Довгі чати іноді підвантажуються поступово, і без цього PDF може обрізатись.
- 3. `Cmd + P` → «Save as PDF» → збережи як `ai_help/claude_chat.pdf`.
- 4. Відкрий PDF і перевір дві речі: на першій сторінці твоє перше повідомлення з описом завдання, на останній — це повідомлення.

4. Фінальний коміт

```
bash

git add ai_help/
git commit -m "Add final AI conversation link and PDF"
git push
git status
```

`git status` має показати, що все чисто.

Після цього здавай. Ти пройшов увесь шлях від першого `while` у токенизаторі до парсера з фазером, тестами і README, і на інтерв'ю тобі є що розповісти. Успіхів! 🍀



